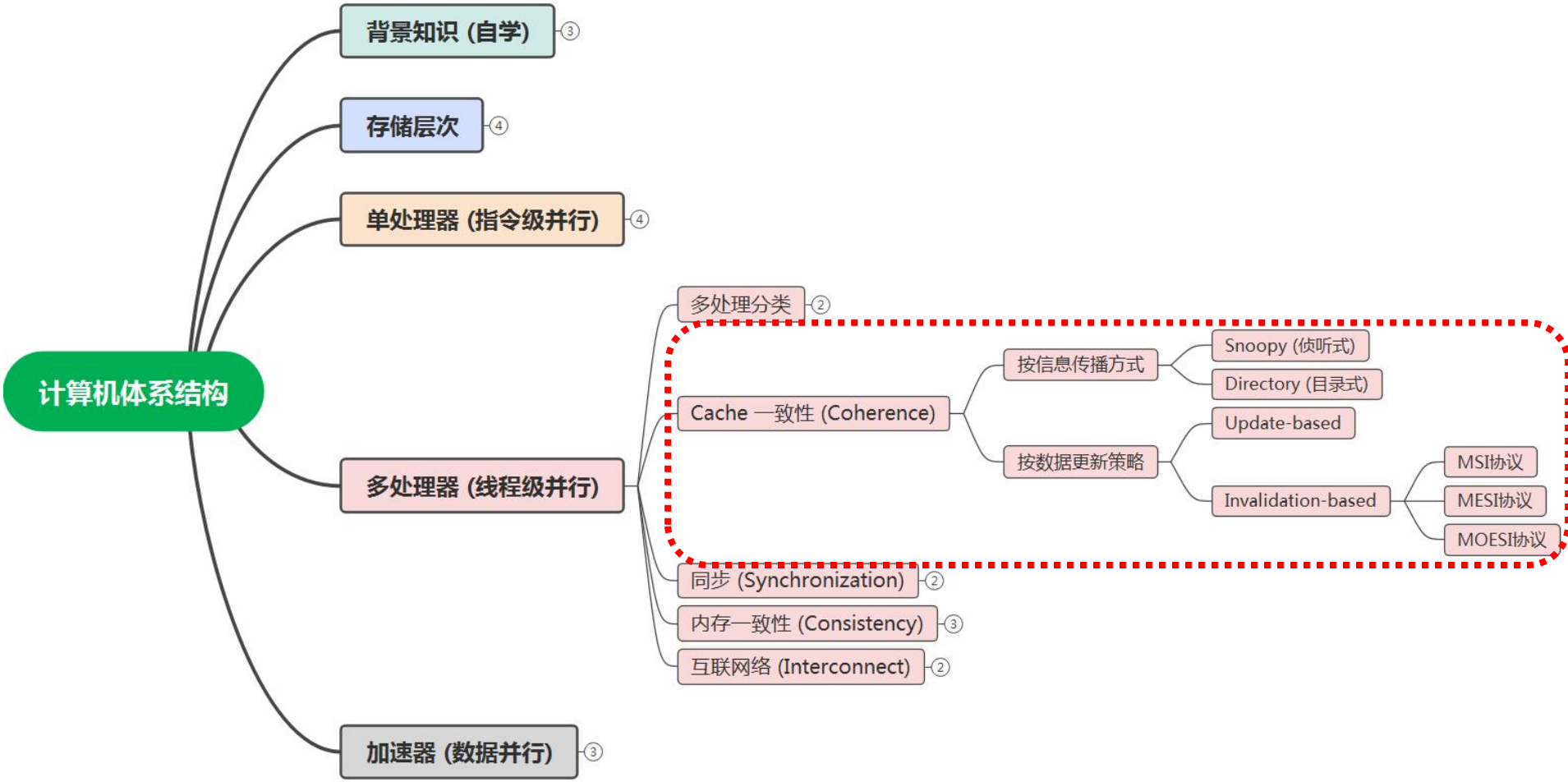
计算机体系结构

L06-2 多处理器缓存一致性

**本课程PPT参考了MIT, CMU, ETH, 中科院, 华科, 台湾清华, 宾大等国内外高校课程, 仅供上课使用, 切勿传播*


计算机体系结构
背景知识 (自学) 3
存储层次 4
单处理器 (指令级并行) 4
多处理器 (线程级并行)
多处理分类 2
Cache 一致性 (Coherence)
按信息传播方式
Snoopy (侦听式)
Directory (目录式)
按数据更新策略
Update-based
Invalidation-based
MSI协议
MESI协议
MOESI协议
同步 (Synchronization) 2
内存一致性 (Consistency) 3
互联网络 (Interconnect) 2
加速器 (数据并行) 3

中心化共享内存
(Centralized Shared Memory)

中心化共享内存架构

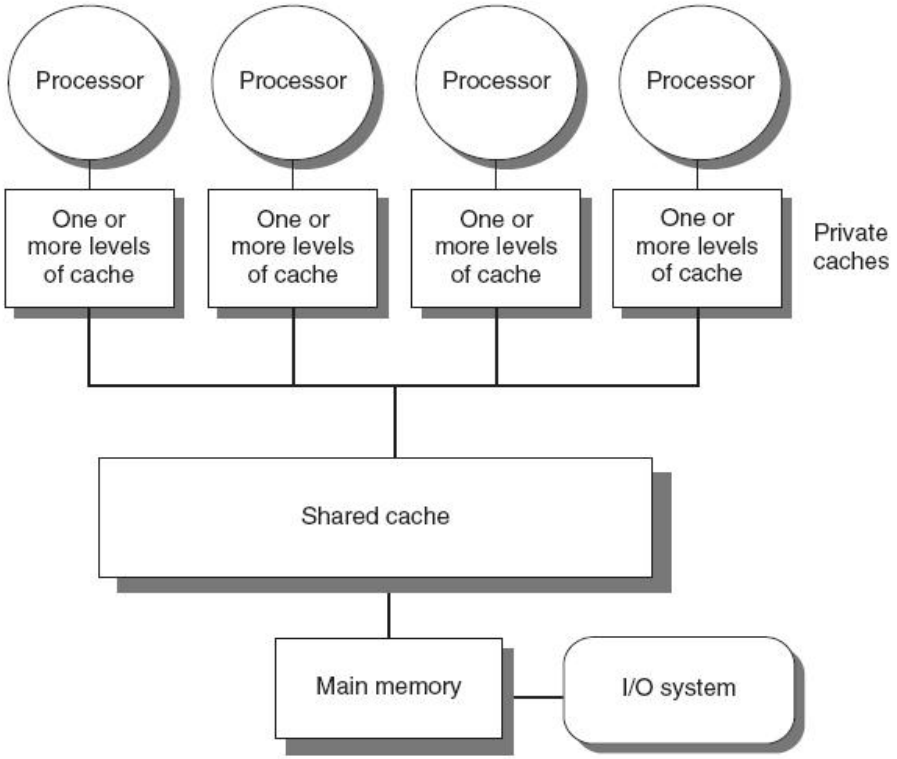
- **Key insight: 大容量多级缓存可以显著降低处理器对内存带宽的需求**

→ **多个处理器共享内存**

- 支持共享数据和私有数据
- 共享数据

  → 通过读写共享内存(cache)实现处理器之间的通信
- 私有数据: 1个处理器使用

  → 出于性能考虑，每个处理器有私有cache

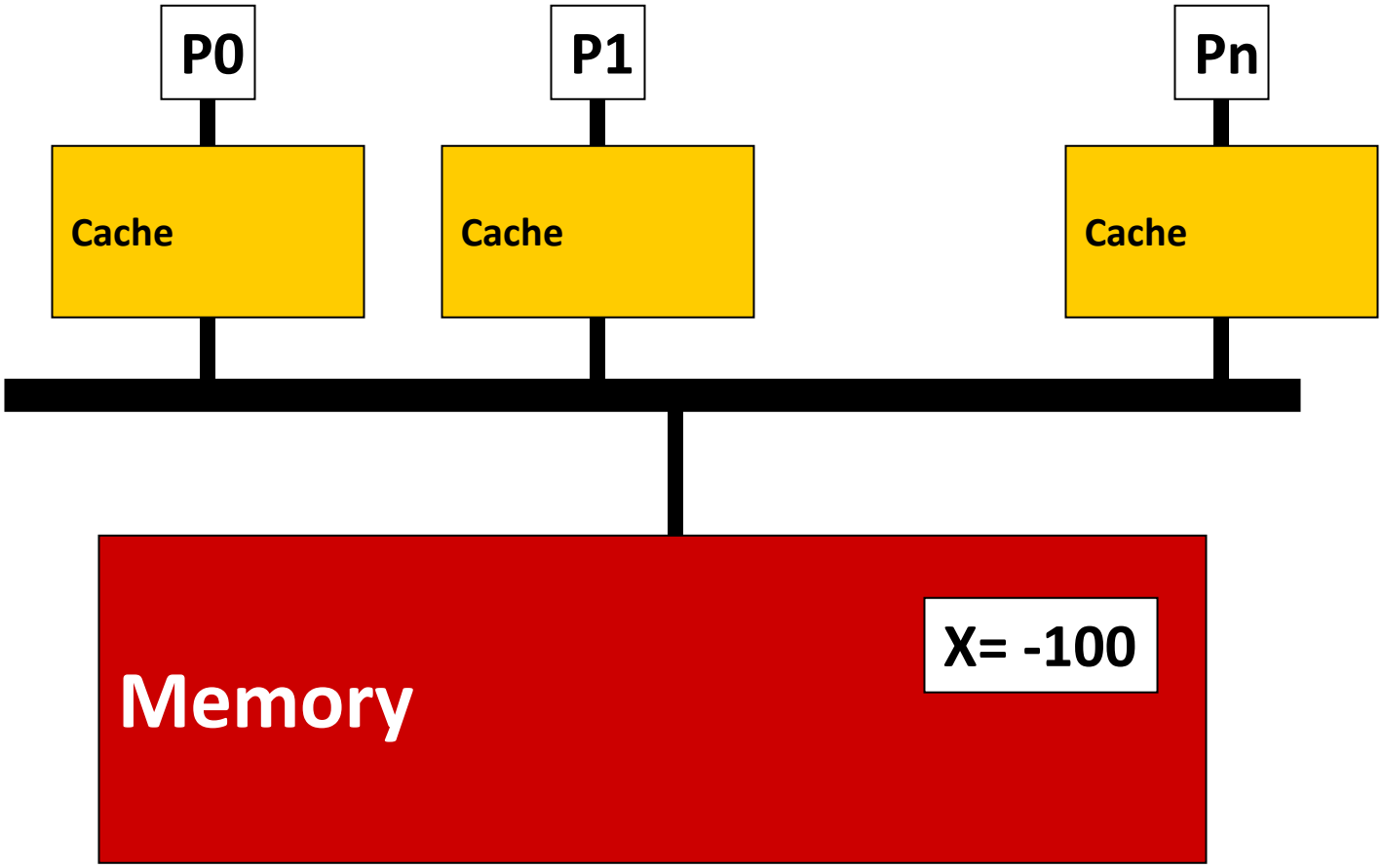


Cache一致性问题

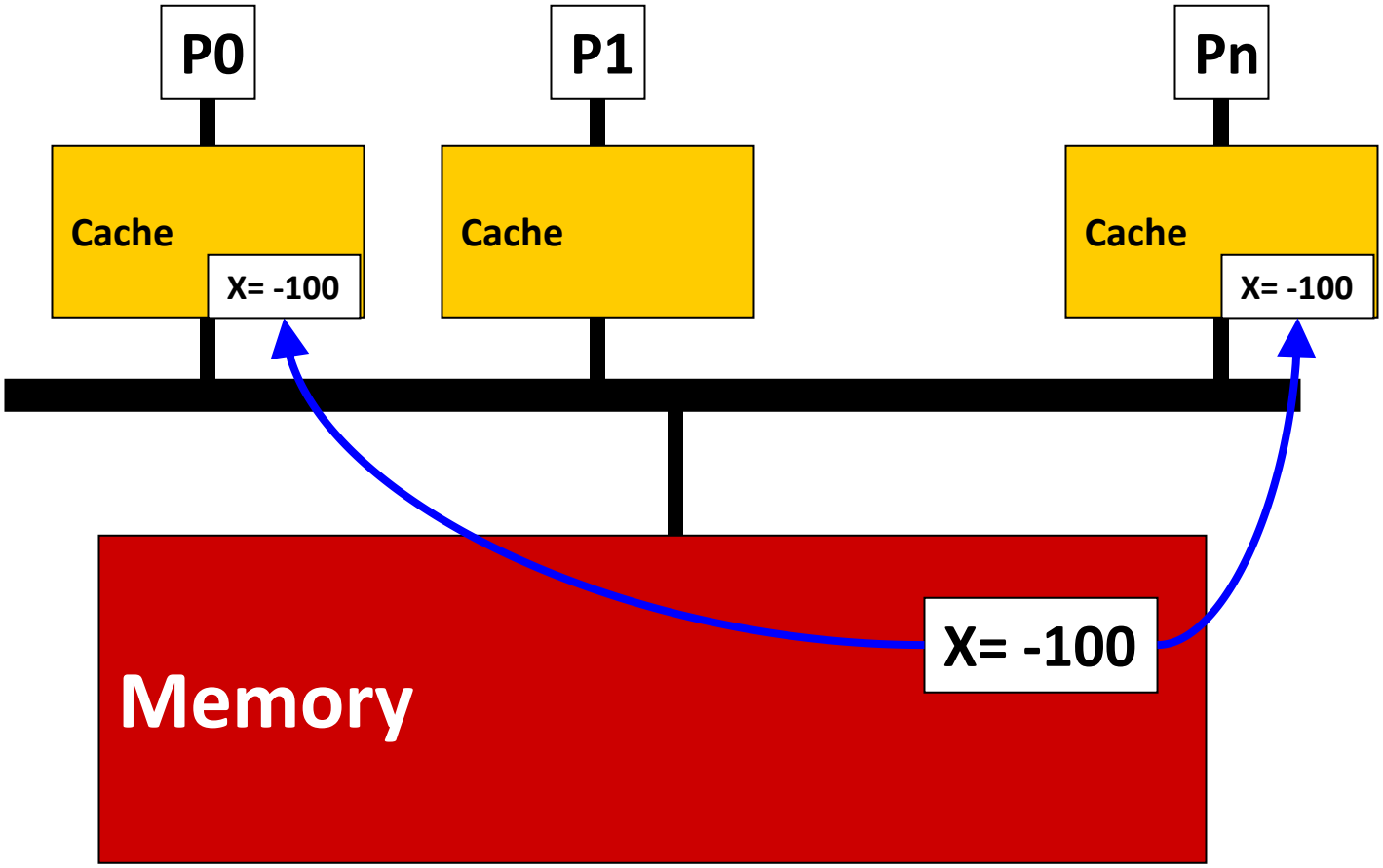
- **问题: 如果多个处理器同时拥有同一个cache block, 如何保证一致性(cache coherence)?**

- **本地更新会导致cache状态不一致**
  - ✓write-through和writeback caches都存在该问题

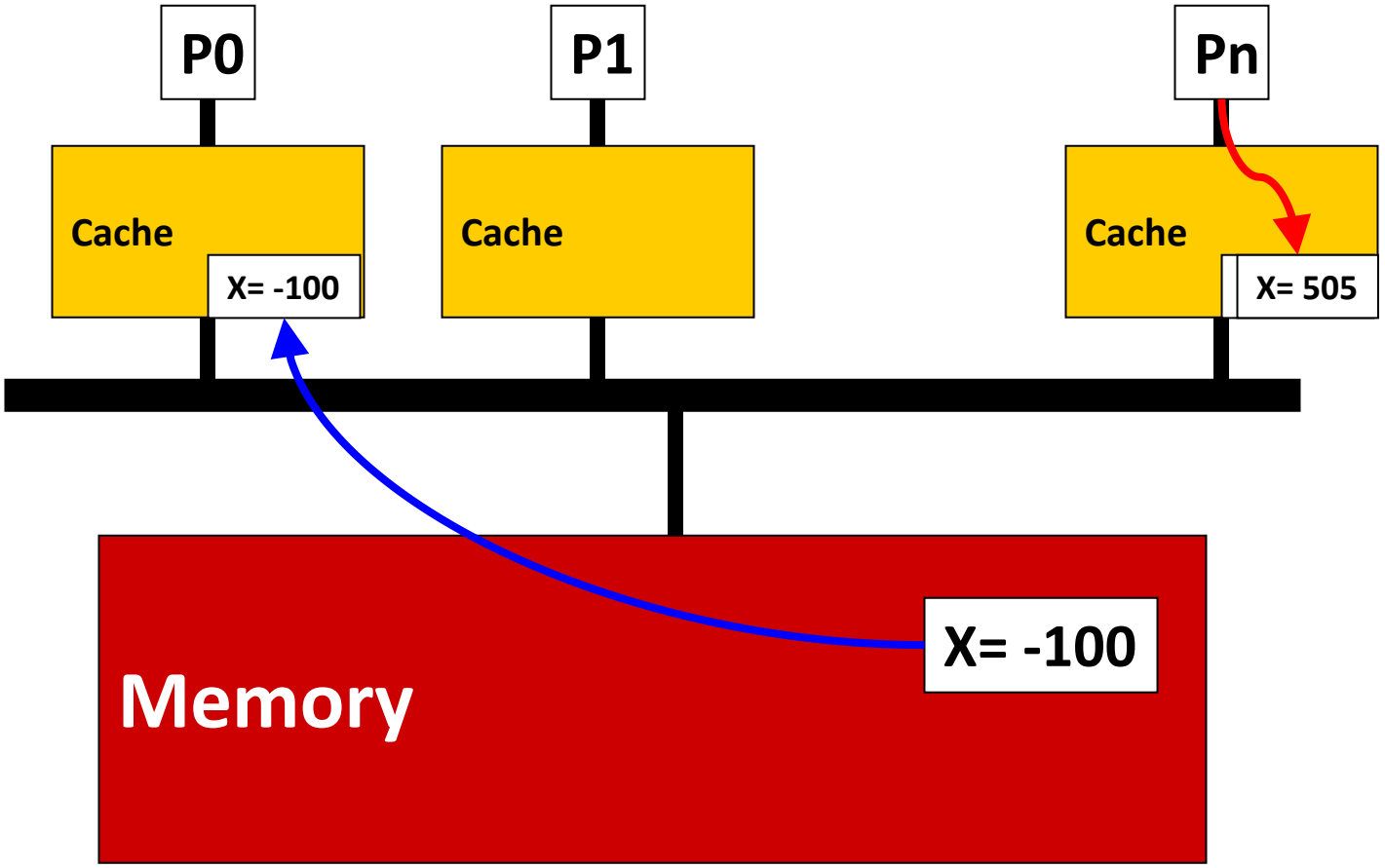
Example: Write-Back Cache



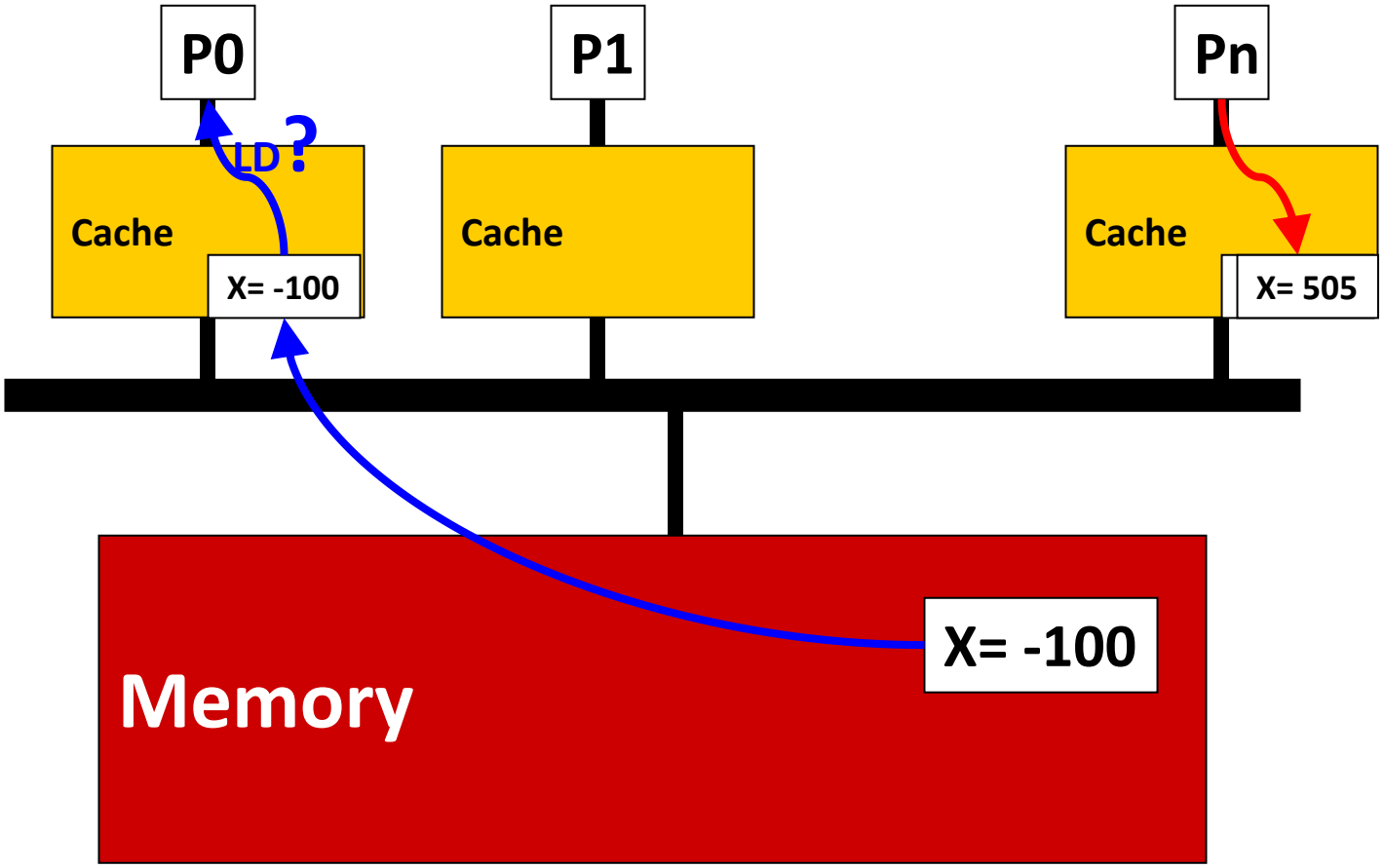
Example: Write-Back Cache



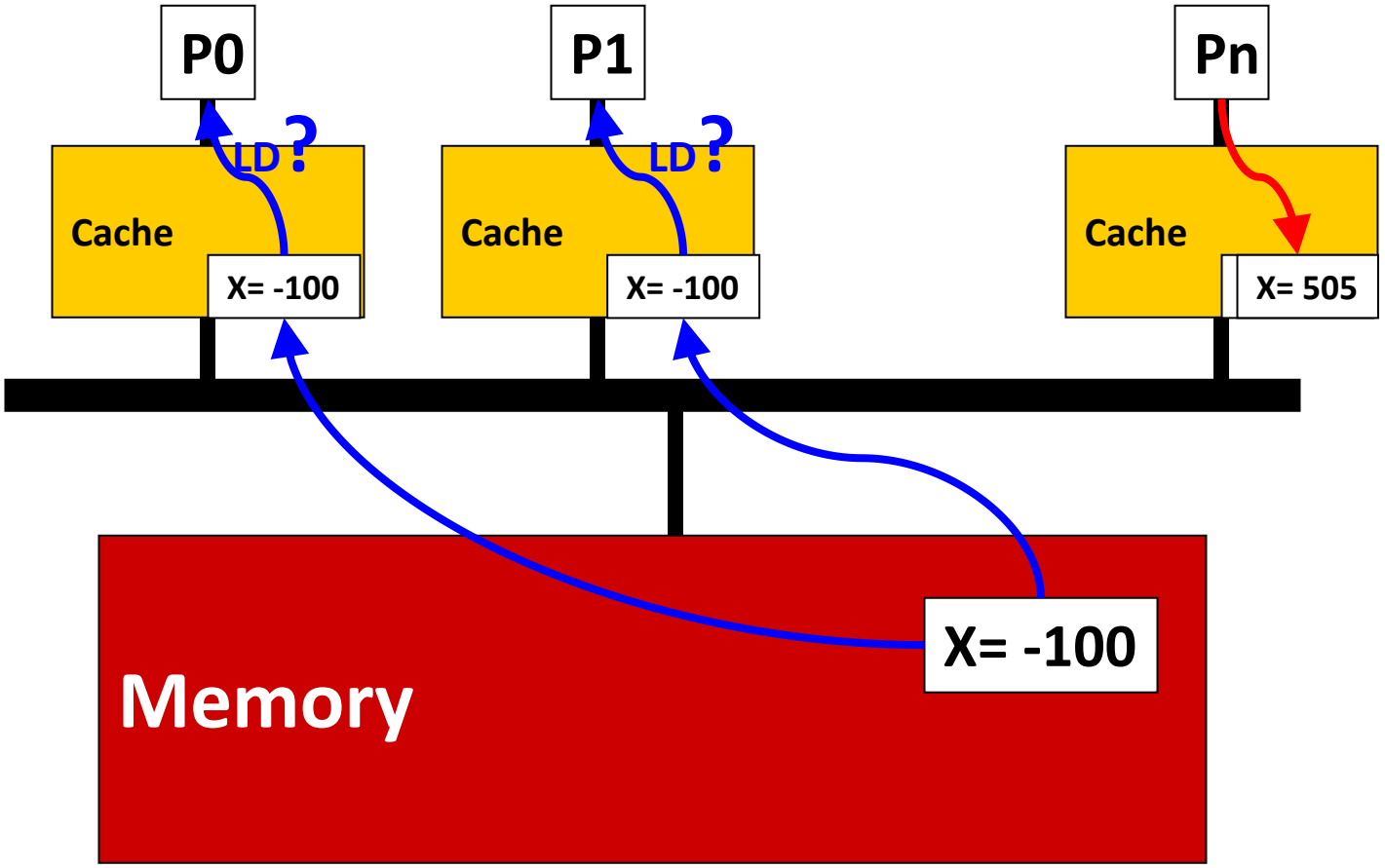
Example: Write-Back Cache



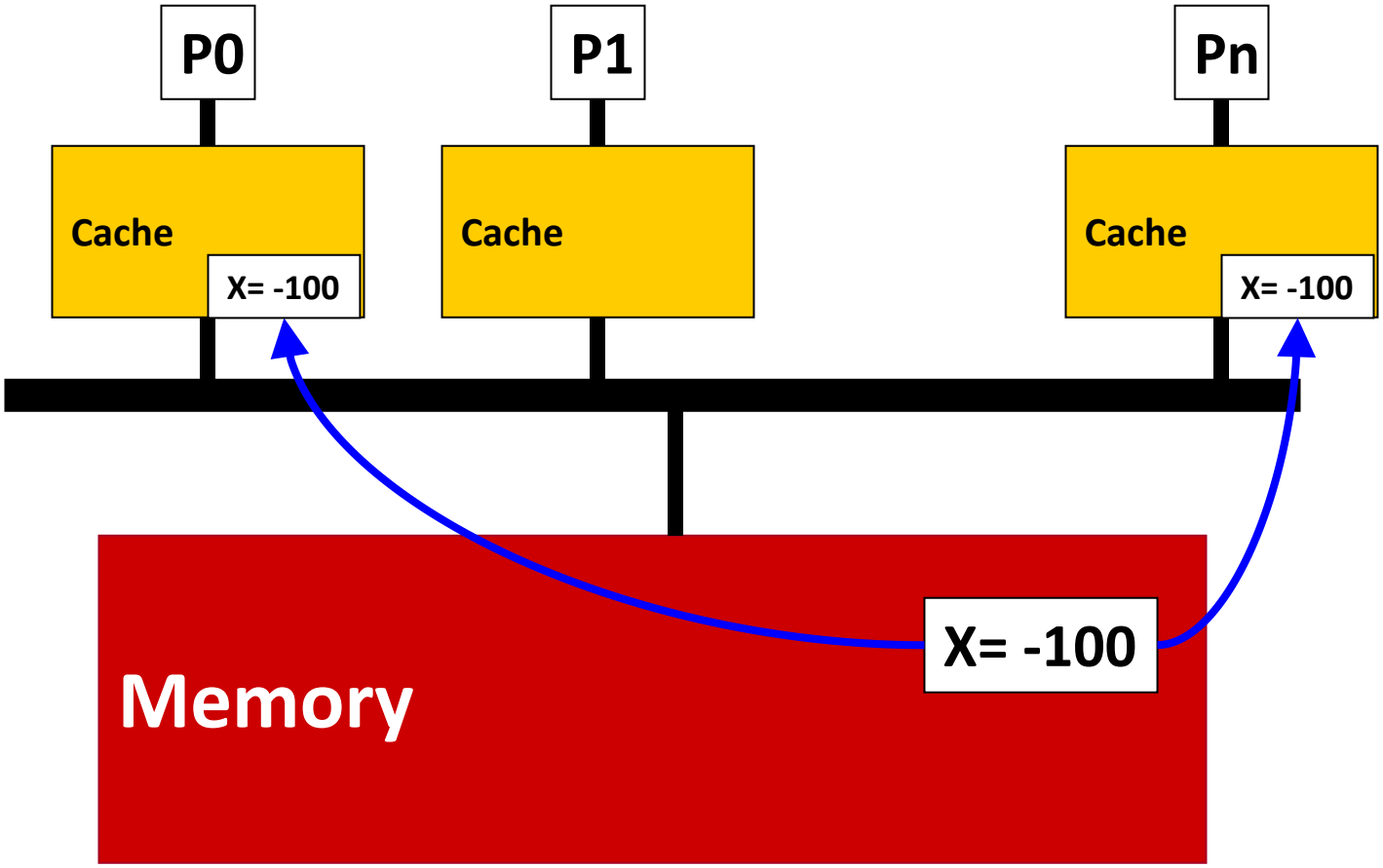
Example: Write-Back Cache



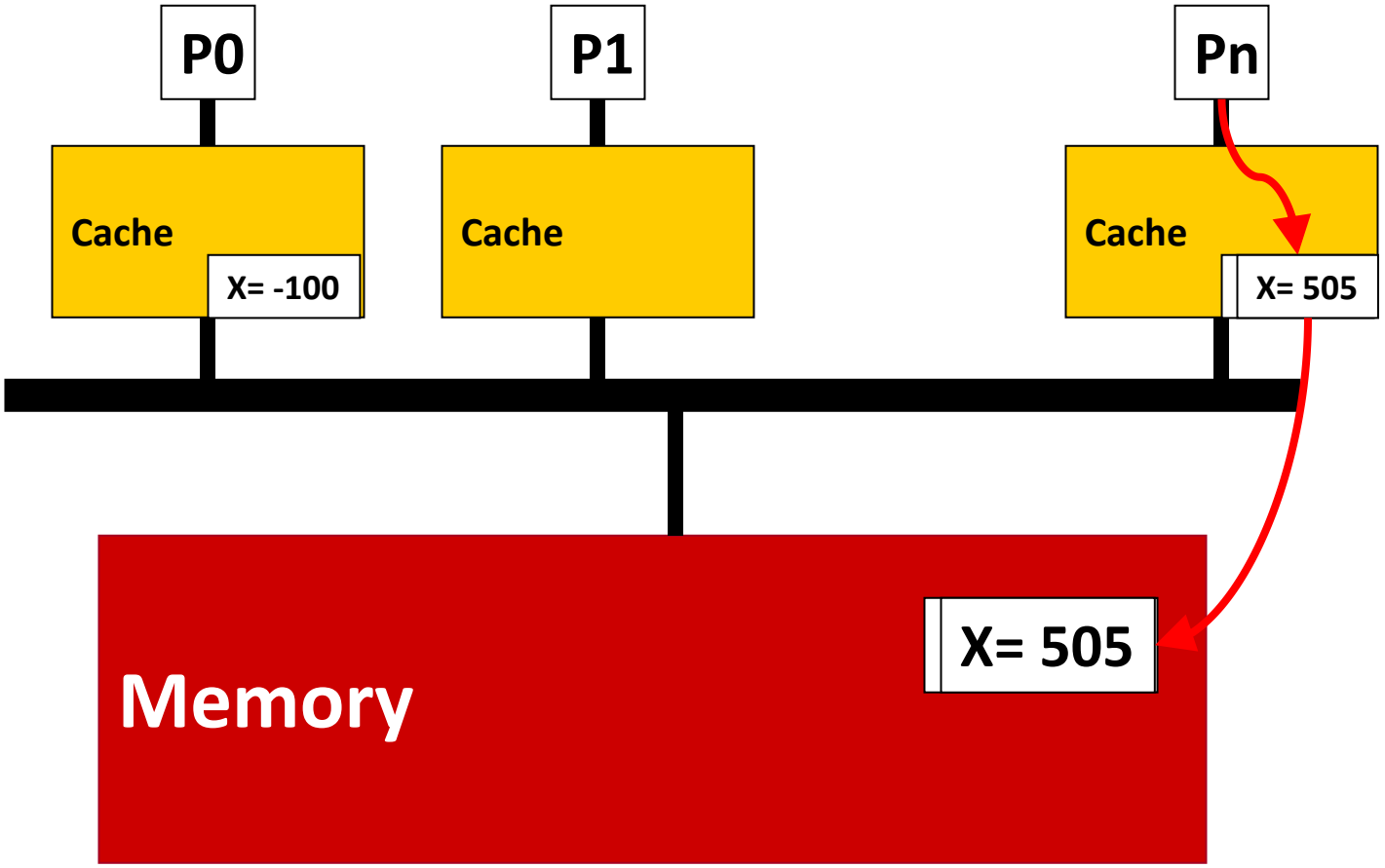
Example: Write-Back Cache



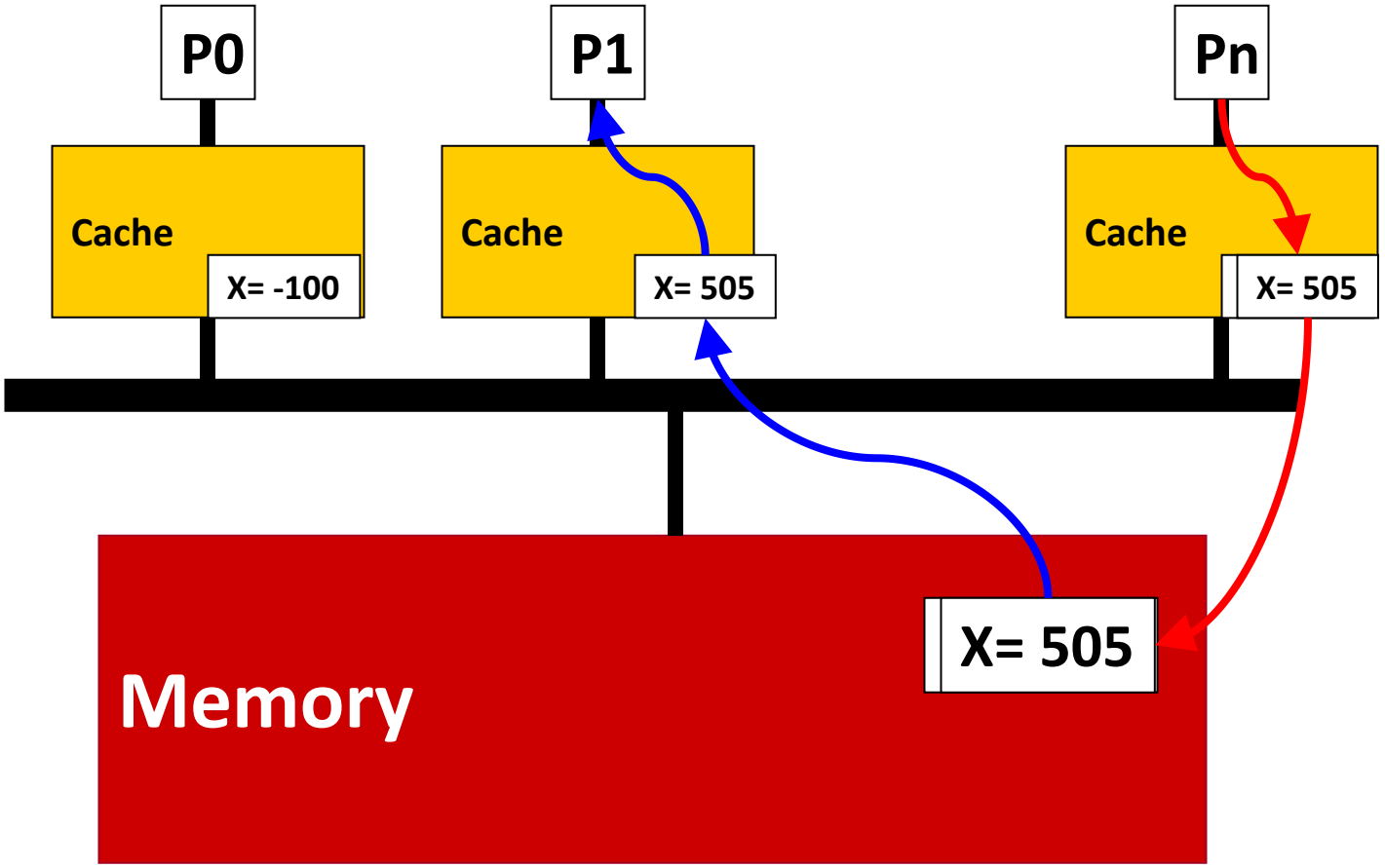
Example: Write-Through Cache



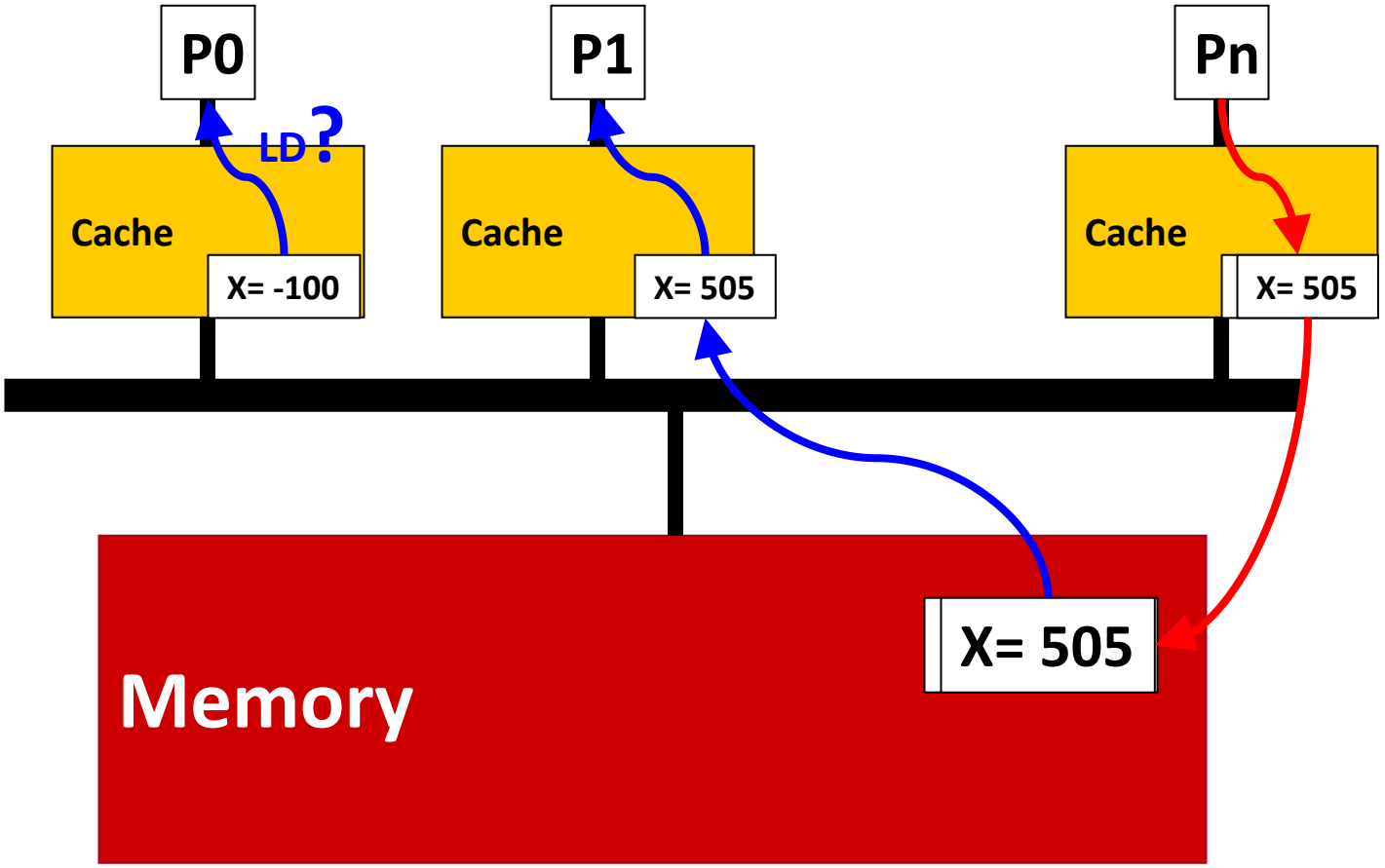
Example: Write-Through Cache



Example: Write-Through Cache



Example: Write-Through Cache



维护Cache一致性(Coherence)

- **一致性定义：**对于相同的内存位置，所有处理器看到的值是一致的

- **基本策略：**将最新值(**写操作**)及时更新到所有cache copy

- **写操作**的两种处理方式
  - **Option 1 (update protocol):** 将新数据广播给所有cache copy
  - **Option 2 (invalidate protocol):** 让所有其他copy都无效，只保留一份有效的本地copy, 并更新它

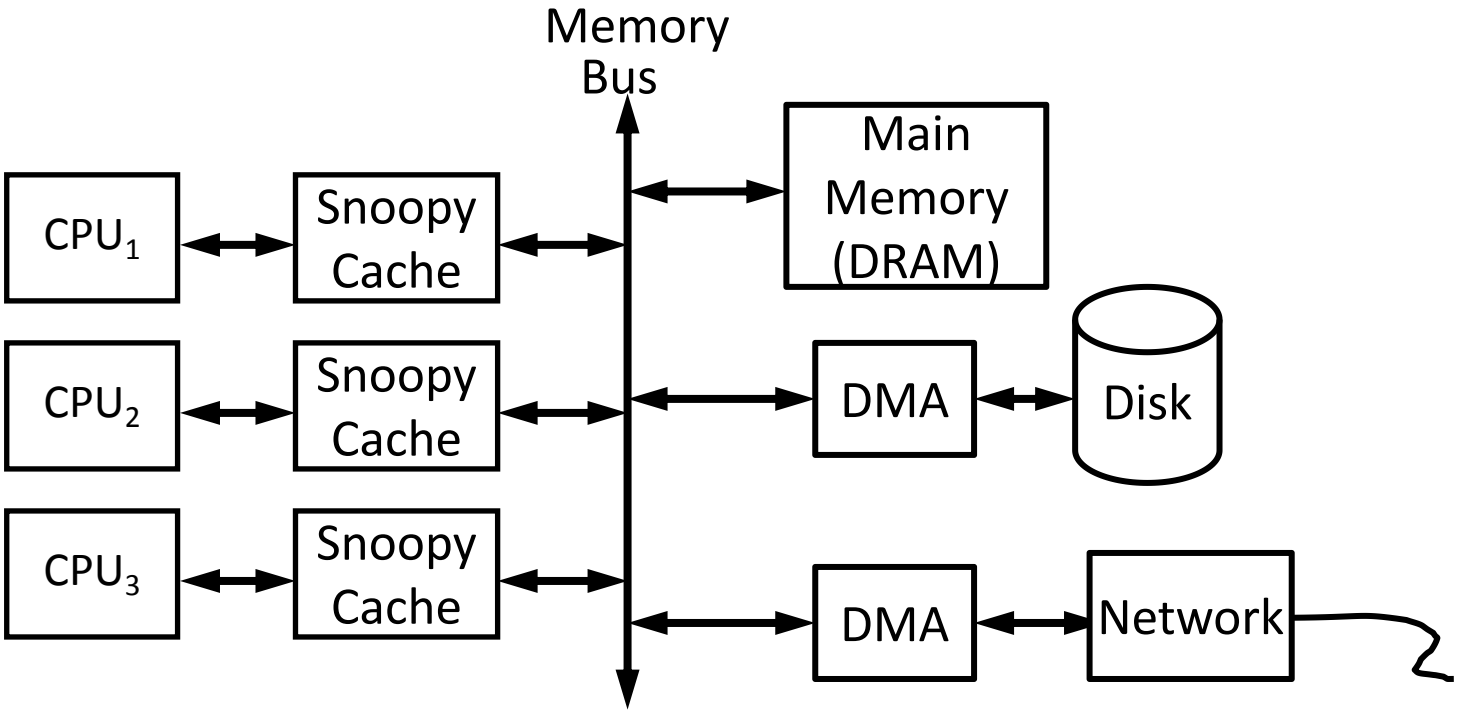
消息如何传播？

- **Snoopy bus: (侦听式)**
  - 处理器在总线上广播数据更新
  - 每个处理器都侦听其他处理器的动向
  - 所有请求通过总线实现序列化→ **完全按顺序**

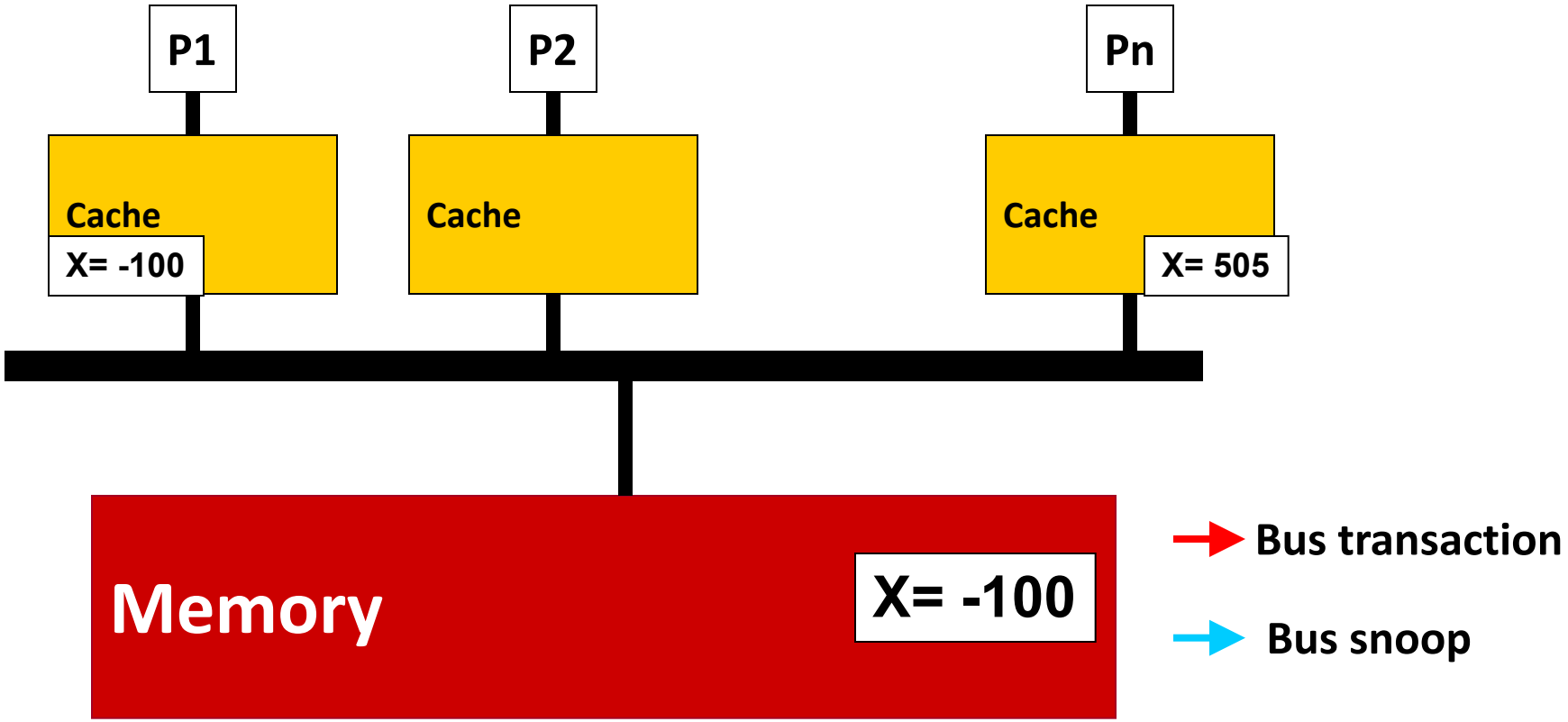
- **Directory: (目录式)**
  - 通过目录跟踪每个block的所有权
  - 处理器通过目录显式地请求数据
  - 目录协调invalidation/update动作
  - 充当为乱序请求”排序”的角色

侦听式Cache Coherence

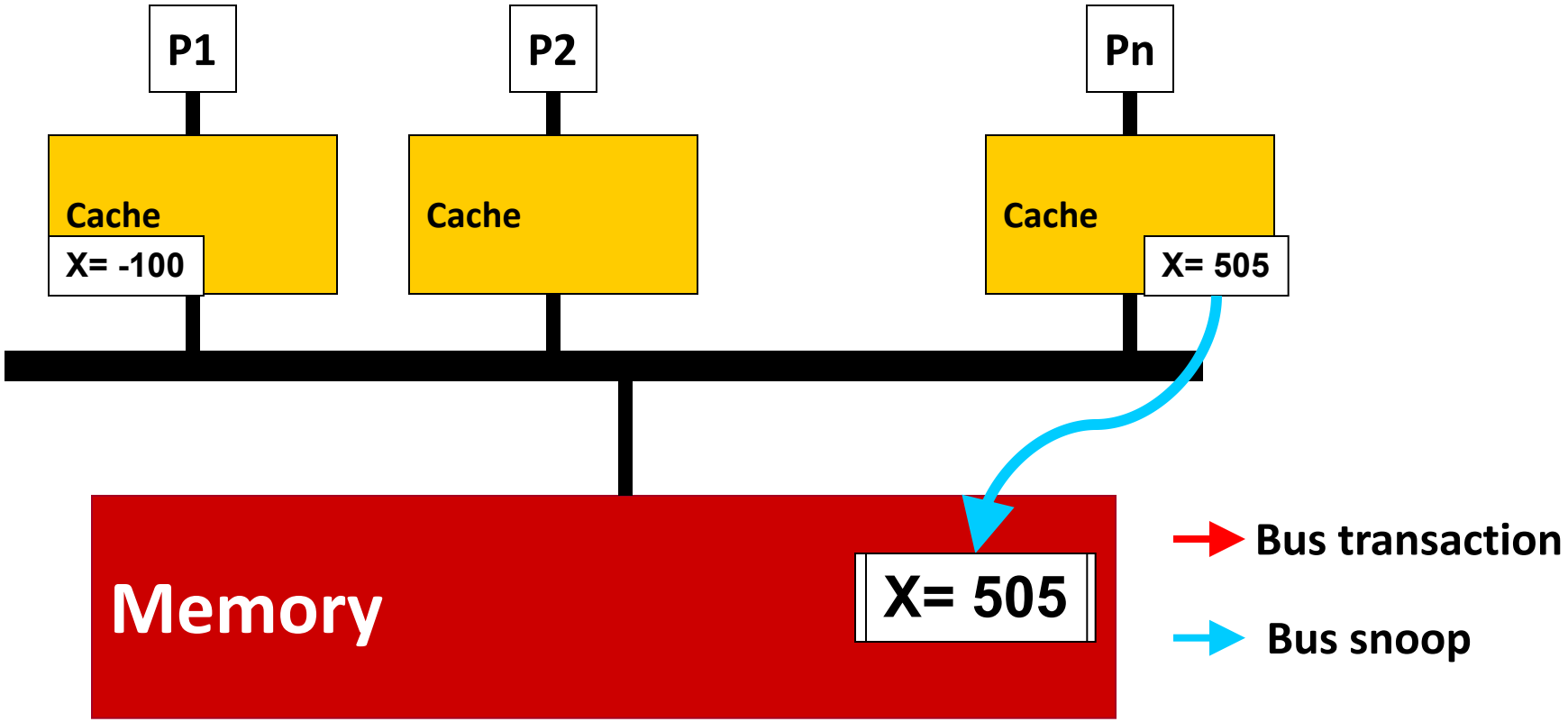
- **如果所有cache共享总线，很容易实现**
  - 每个cache在总线上广播其read/write操作
  - 每个cache block的有限状态机(FSM)比较简单
  - 适合小规模多处理器



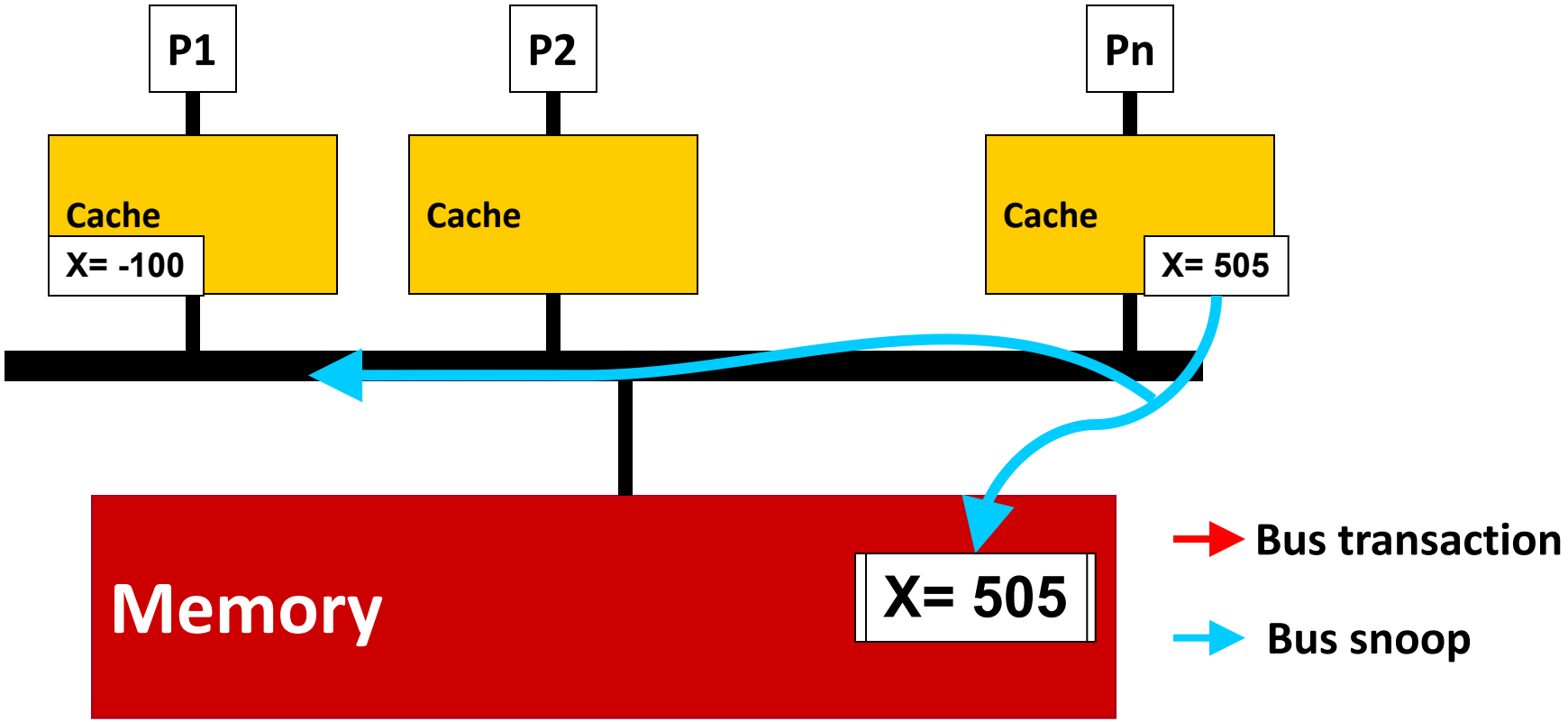
侦听式协议：Write-Through Cache + Update-based



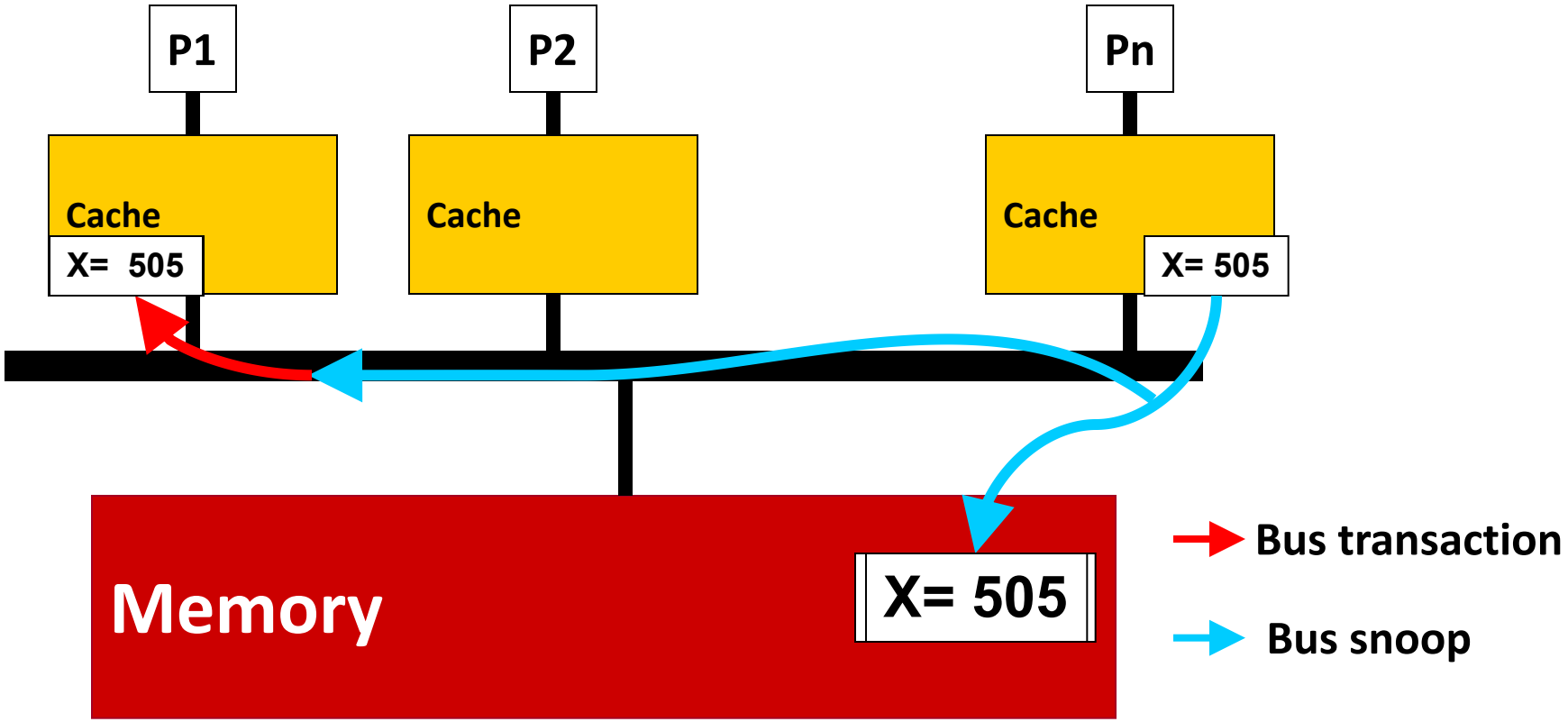
侦听式协议：Write-Through Cache + Update-based



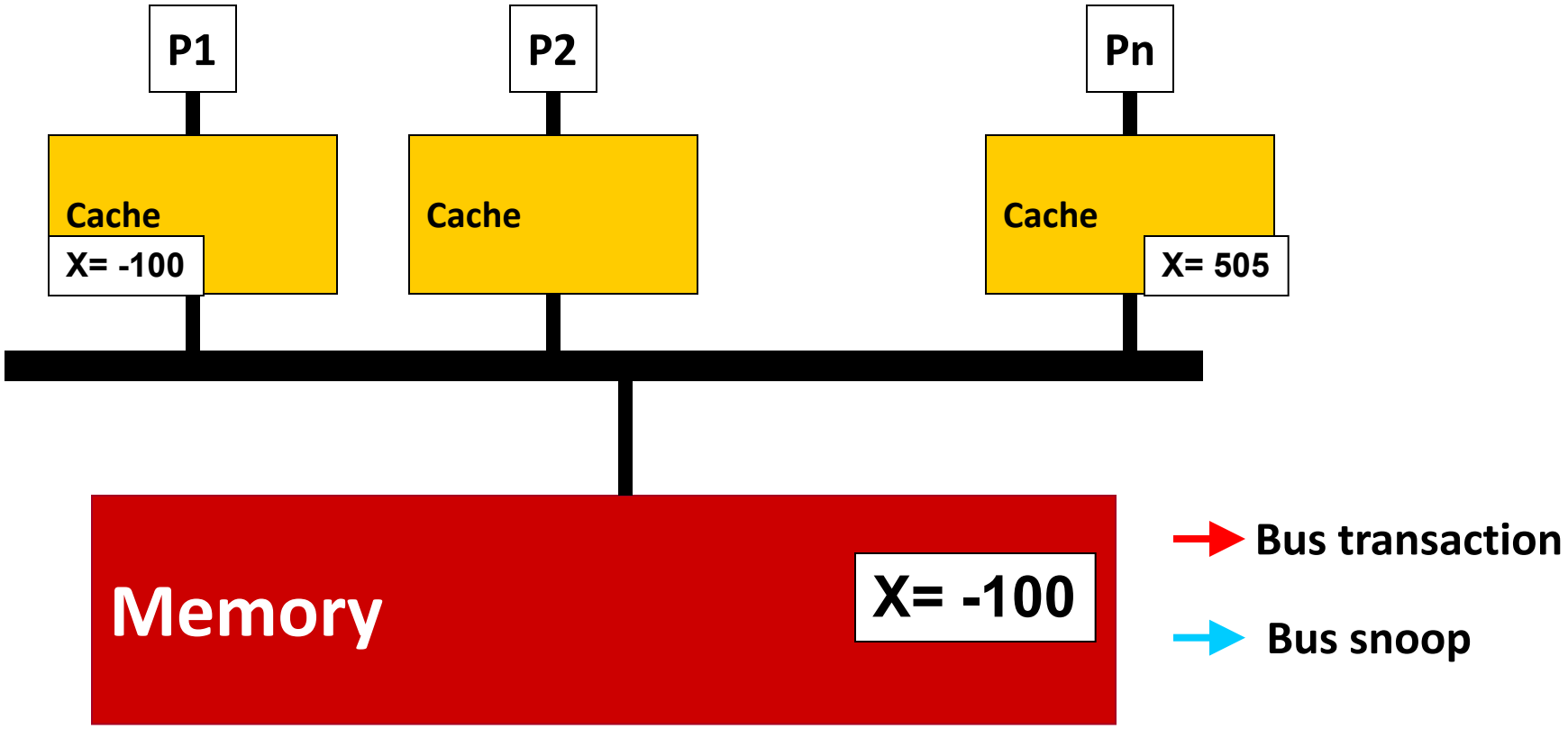
侦听式协议：Write-Through Cache + Update-based



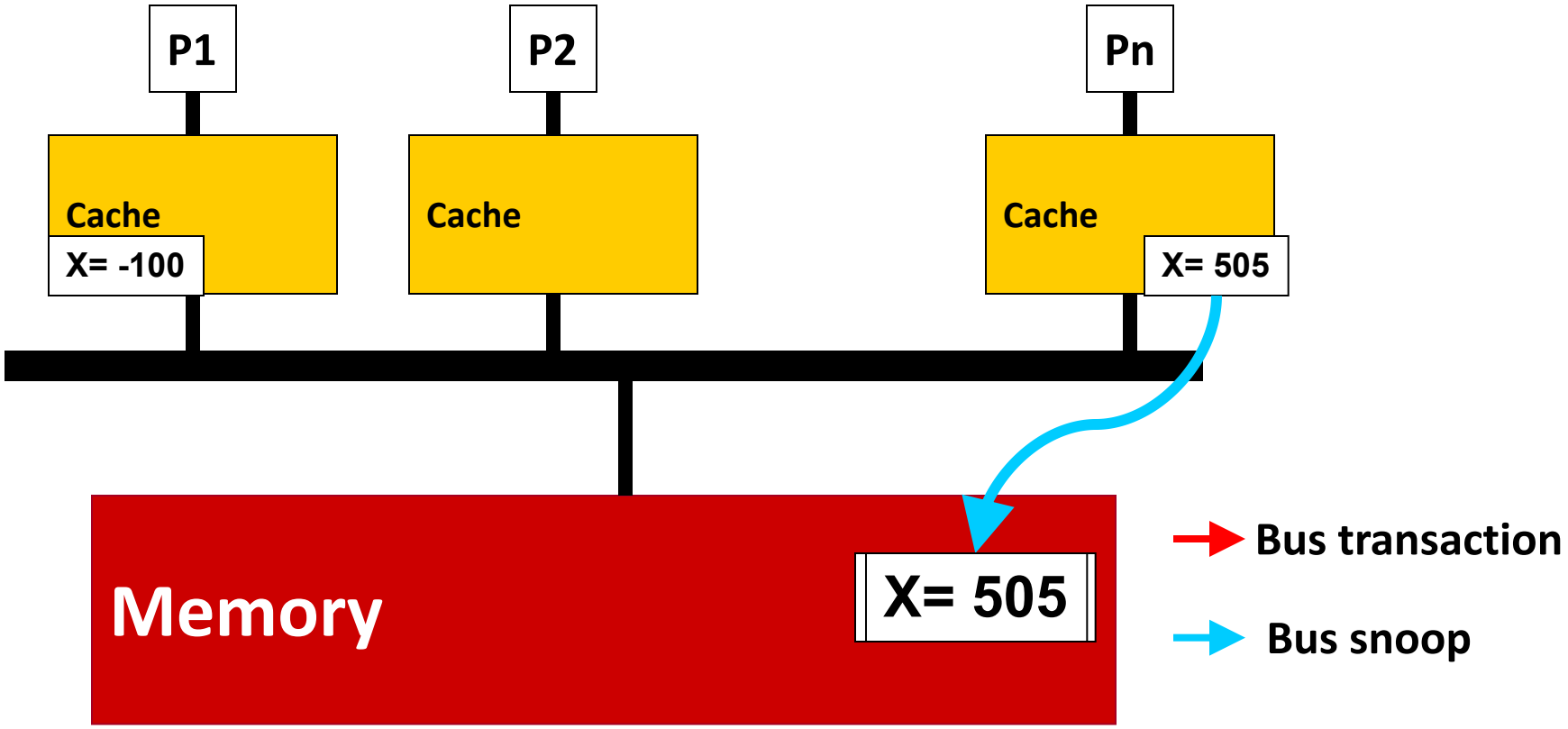
侦听式协议：Write-Through Cache + Update-based



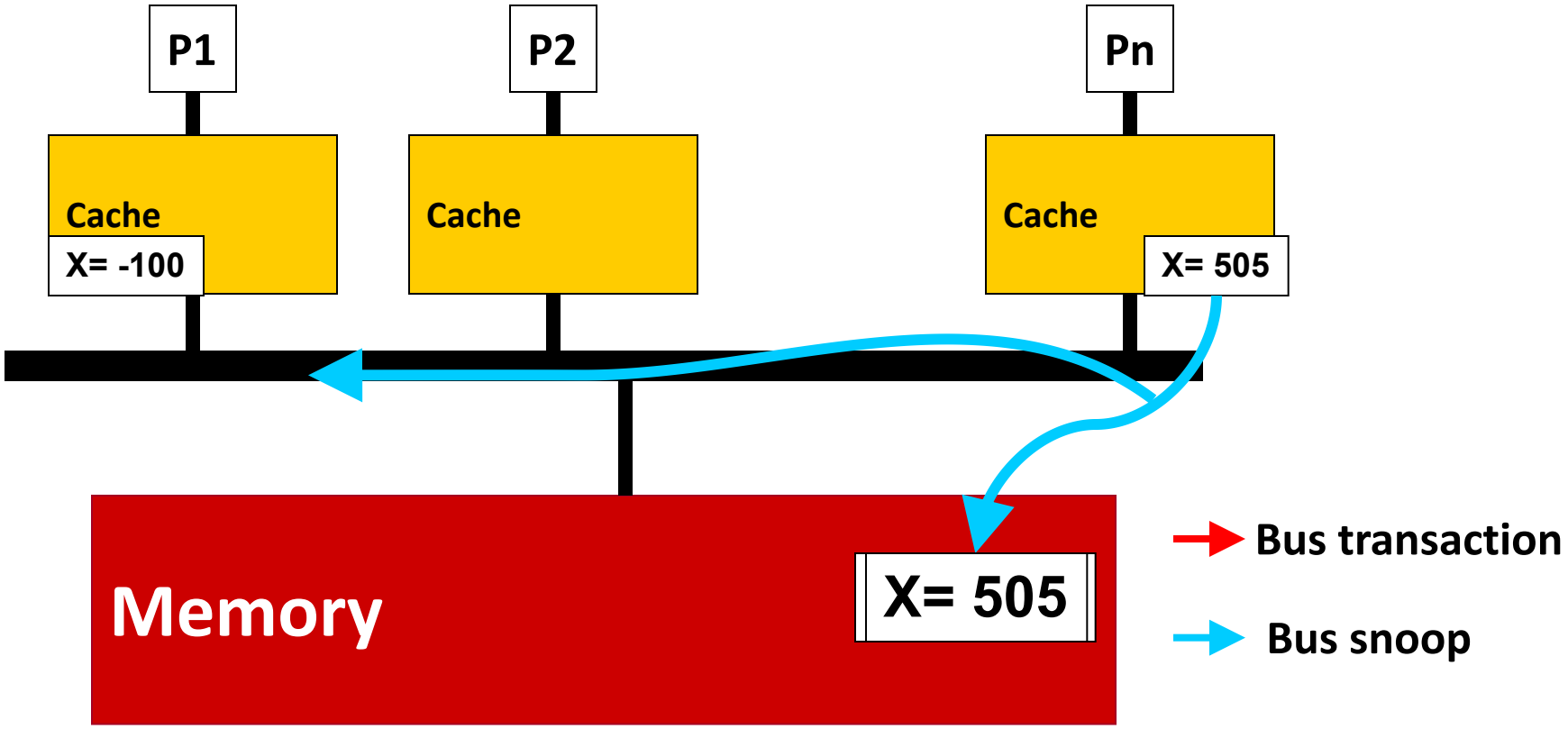
侦听式协议：Write-Through Cache + Invalidation-based



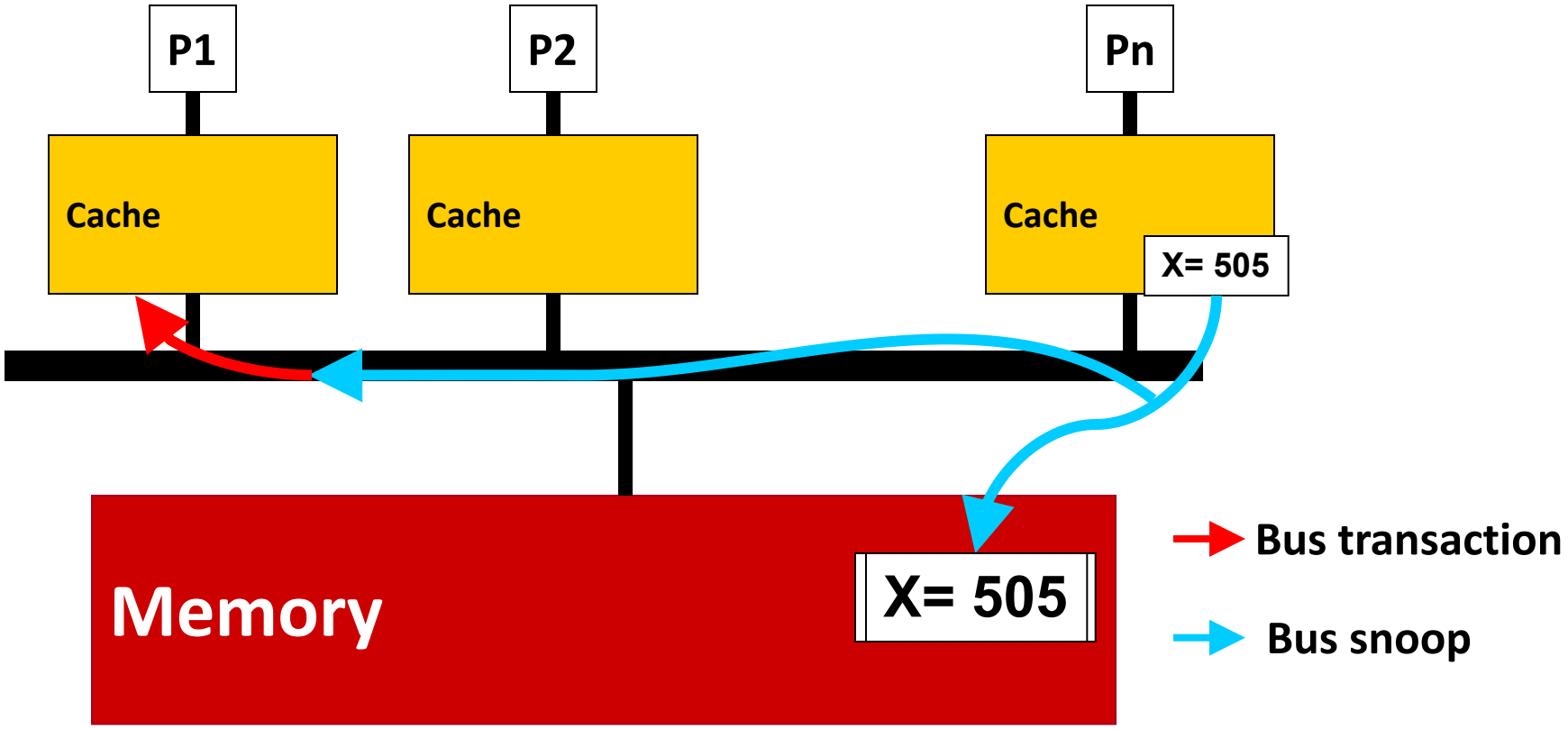
侦听式协议：Write-Through Cache + Invalidation-based



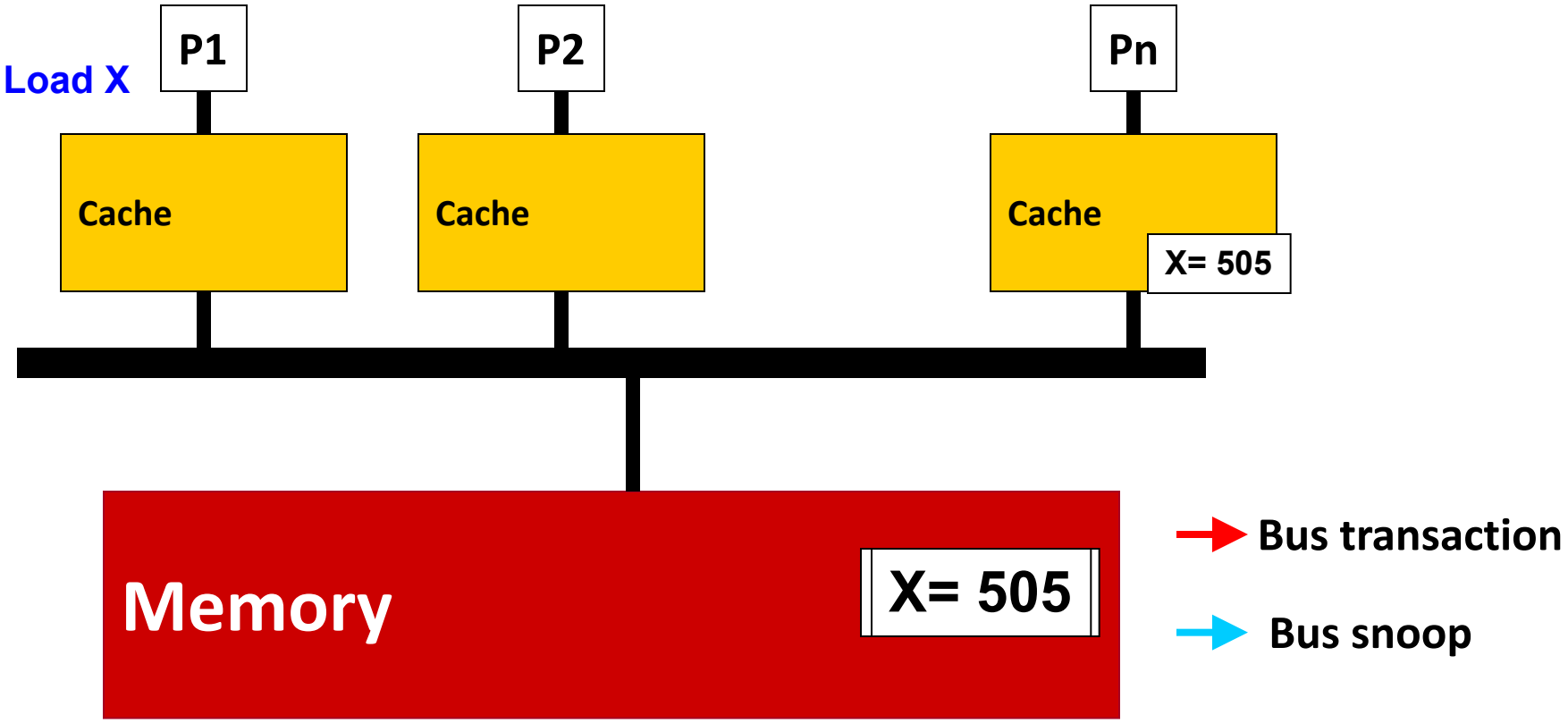
侦听式协议：Write-Through Cache + Invalidation-based



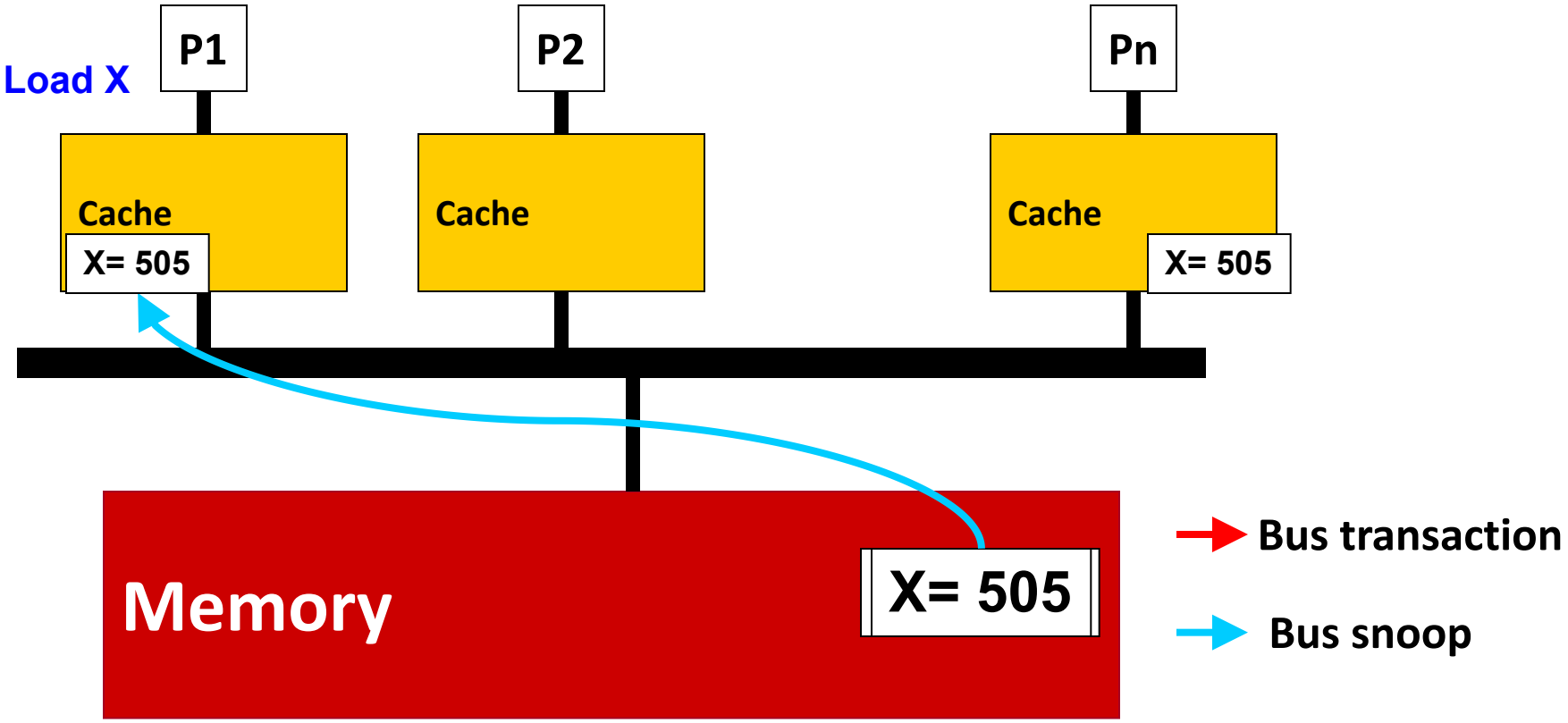
侦听式协议：Write-Through Cache + Invalidation-based



侦听式协议：Write-Through Cache + Invalidation-based



侦听式协议：Write-Through Cache + Invalidation-based



总线侦听的作用

- 当处理器写本地copy时，会广播到所有其他cache，所有cache将看到相同的更新
  → **write propagation**

- 当两个处理器几乎同时向各自的本地copy写入相同数据时，一个处理器将会赢得总线，并向所有其他缓存广播其更新信息，另一个处理器必须等待直到它能够获取总线
  → 这保证了所有其他缓存看到相同的顺序
  → **write serialization**

Update vs. Invalidate Tradeoffs

Update-based

+ 如果数据更新不频繁, 可以有效避免invalidate产生的额外开销(下次读要重新load)

- 如果数据在其他处理器介入读取之前又被重写(更新频繁)，那么更新就是无用的

Invalidate-based

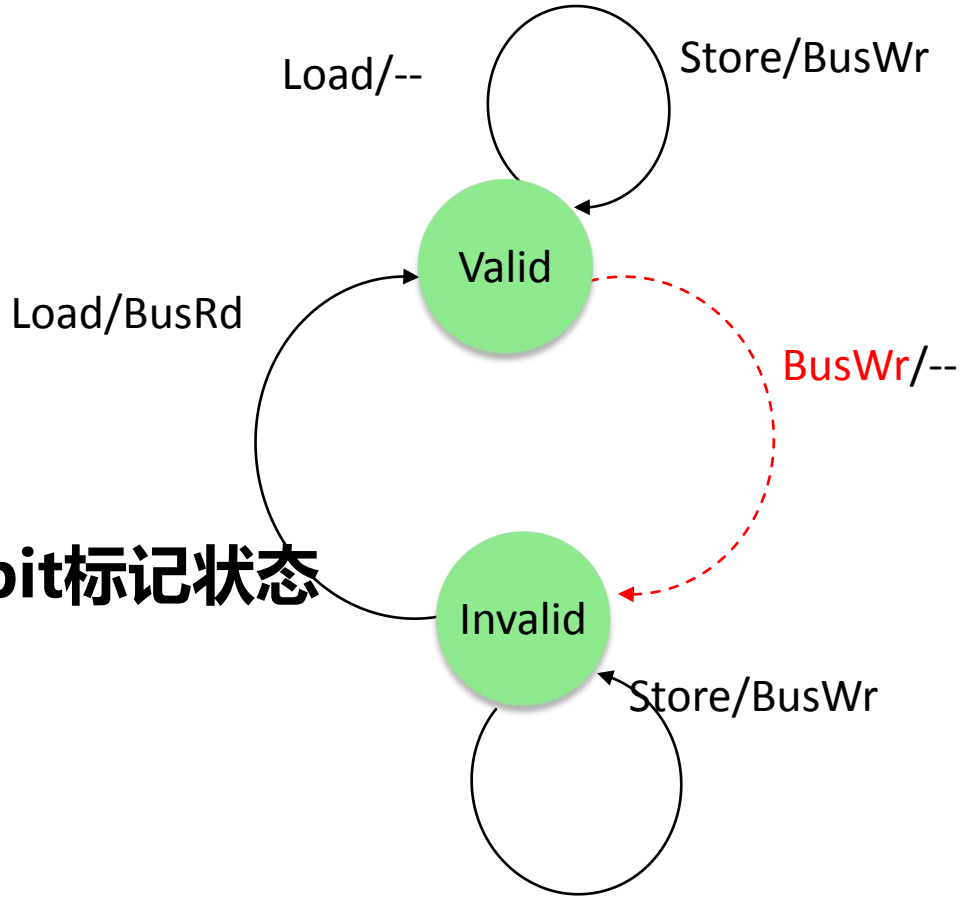
+ 使无效广播后, 处理器有唯一有效copy
+ 只有使无效后又读的处理器才会有本地copy

- 下次读取数据时间长(re-load)
- 如果写操作竞争激烈, 产生ping-pong效应 (使无效->读->又使无效)

简单的侦听式一致性协议 (VI 协议)

write-through, invalidation-based, no write-allocate cache

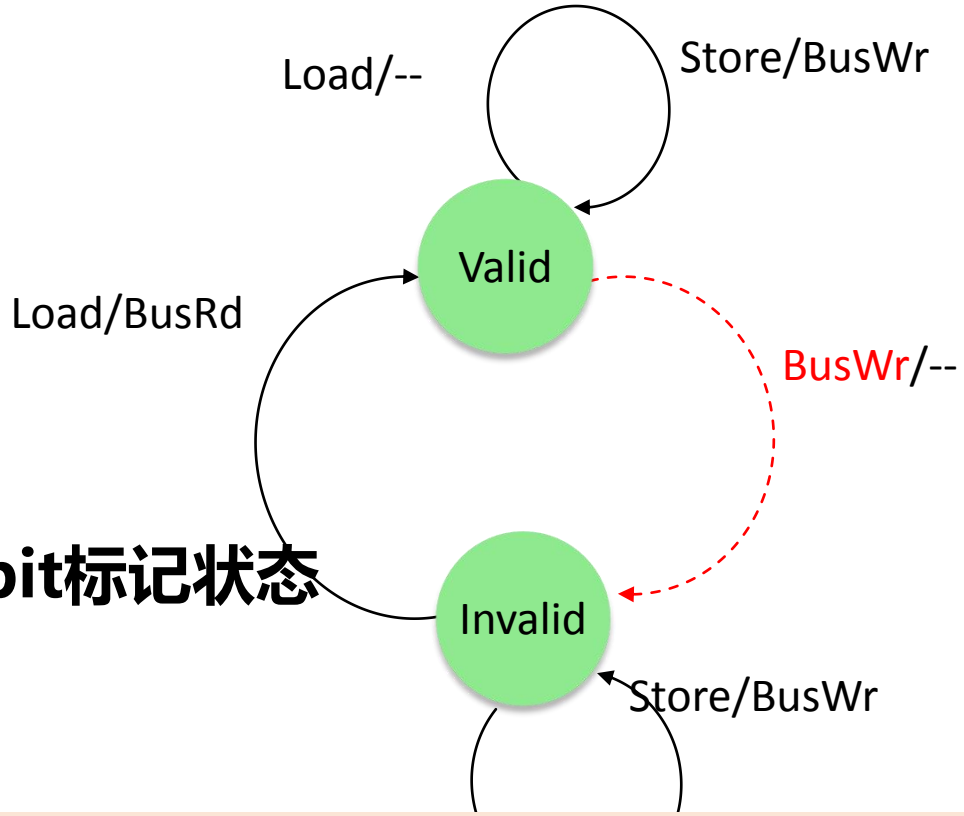
- **处理器请求**
  - Load, Store
- **总线消息**
  - BusRd, BusWr
- **每个cache block用1 bit标记状态**
  - Valid/Invalid



简单的侦听式一致性协议 (VI 协议)

write-through, invalidation-based, no write-allocate cache

- **处理器请求**
  - Load, Store
- **总线消息**
  - BusRd, BusWr
- **每个cache block用1 bit标记状态**
  - Valid/Invalid

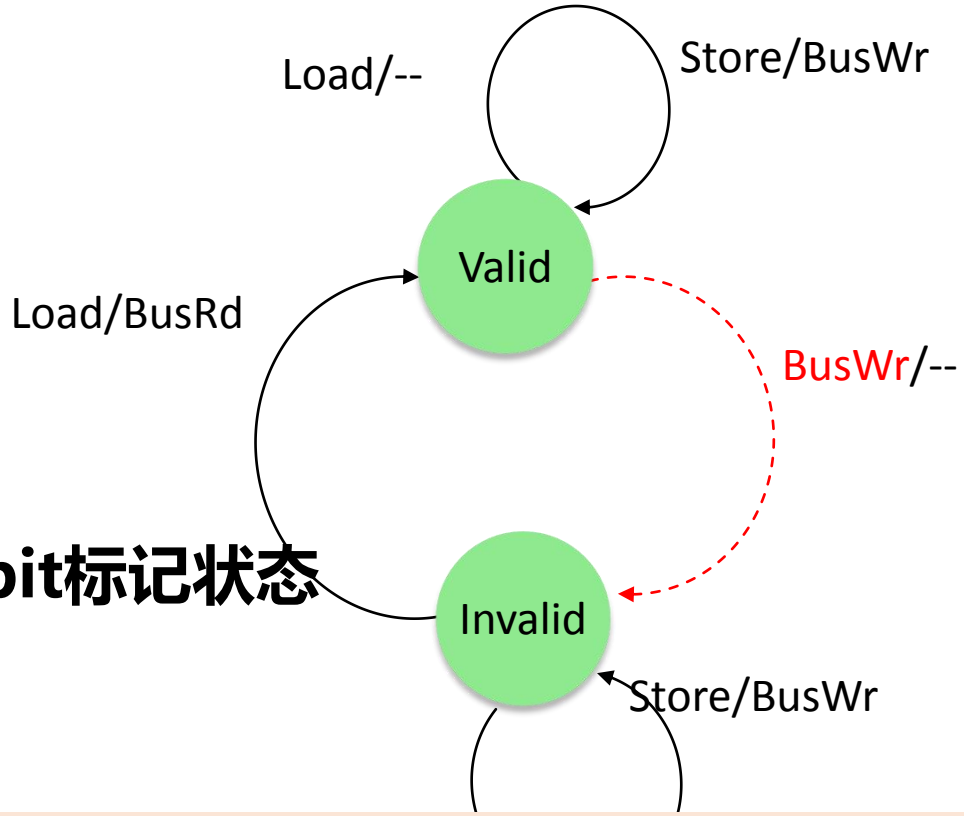


如果copy处于valid状态，本地读，仍保持valid；本地写，更新数据，保持valid状态，但同时要发起BusWr请求，去invalidate其他的copy

简单的侦听式一致性协议 (VI 协议)

write-through, invalidation-based, no write-allocate cache

- **处理器请求**
  - Load, Store
- **总线消息**
  - BusRd, BusWr
- **每个cache block用1 bit标记状态**
  - Valid/Invalid

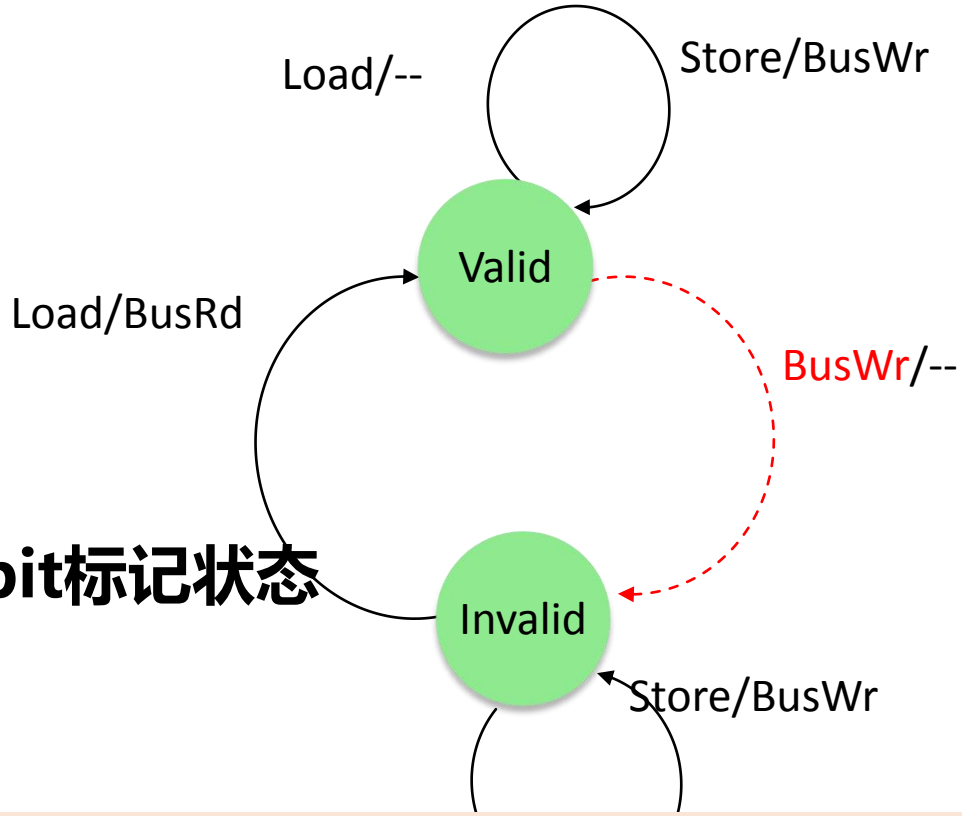


如果copy处于valid状态，收到来自bus的BusWr(红色)，说明别人在写，自己会变为invalid

简单的侦听式一致性协议 (VI 协议)

write-through, invalidation-based, no write-allocate cache

- **处理器请求**
  - Load, Store
- **总线消息**
  - BusRd, BusWr
- **每个cache block用1 bit标记状态**
  - Valid/Invalid



如果copy处于invalid状态，本地写，发生write miss，按照no write allocate规则，不进行cache填充，直接写到下一层(memory)，所以仍然保持invalid状态，同时发起BusWr

对于Write-Back Caches?

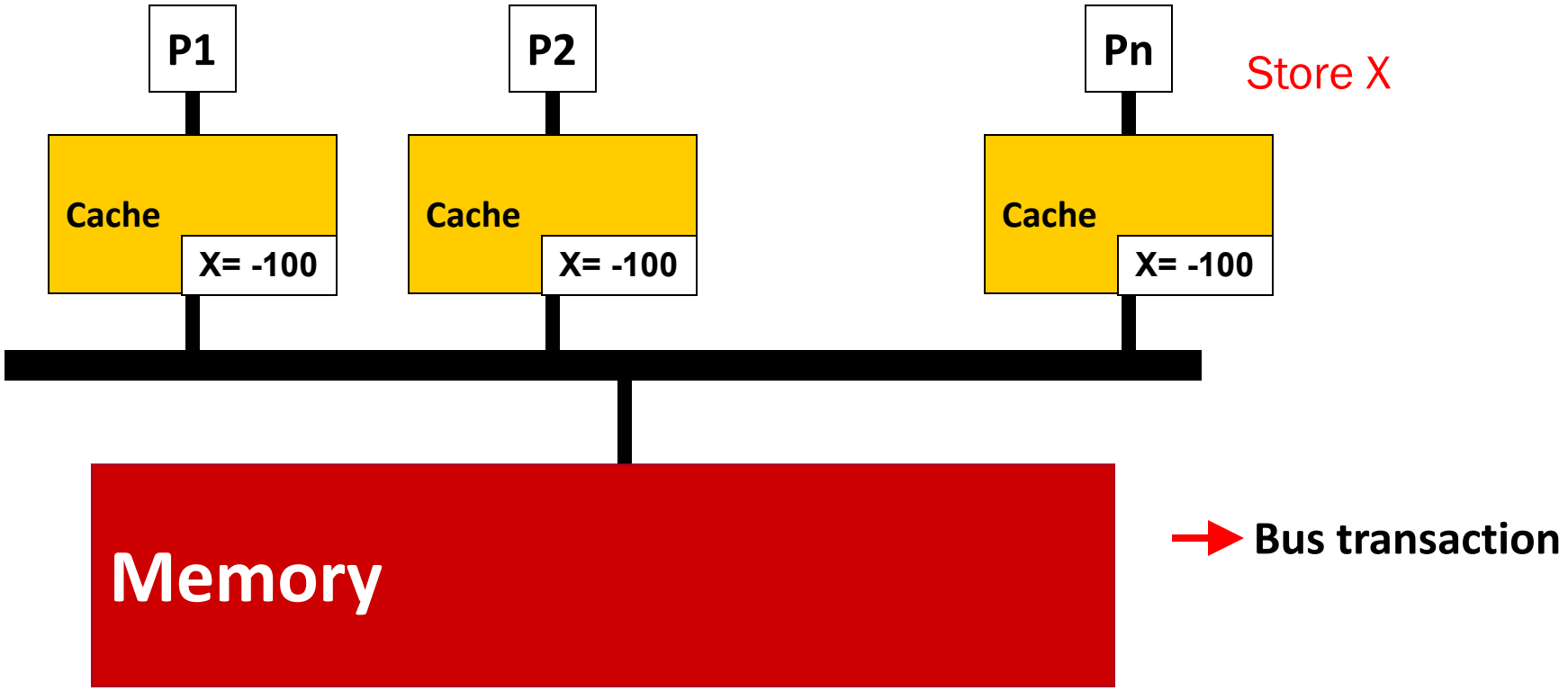
- **Write-back caches 仅在block被替换出去的时候才写数据到memory**
  - Cache和memory可能不一致
  - 减少总线写带宽 → 写操作不需要总线广播

对于Write-Back Caches?

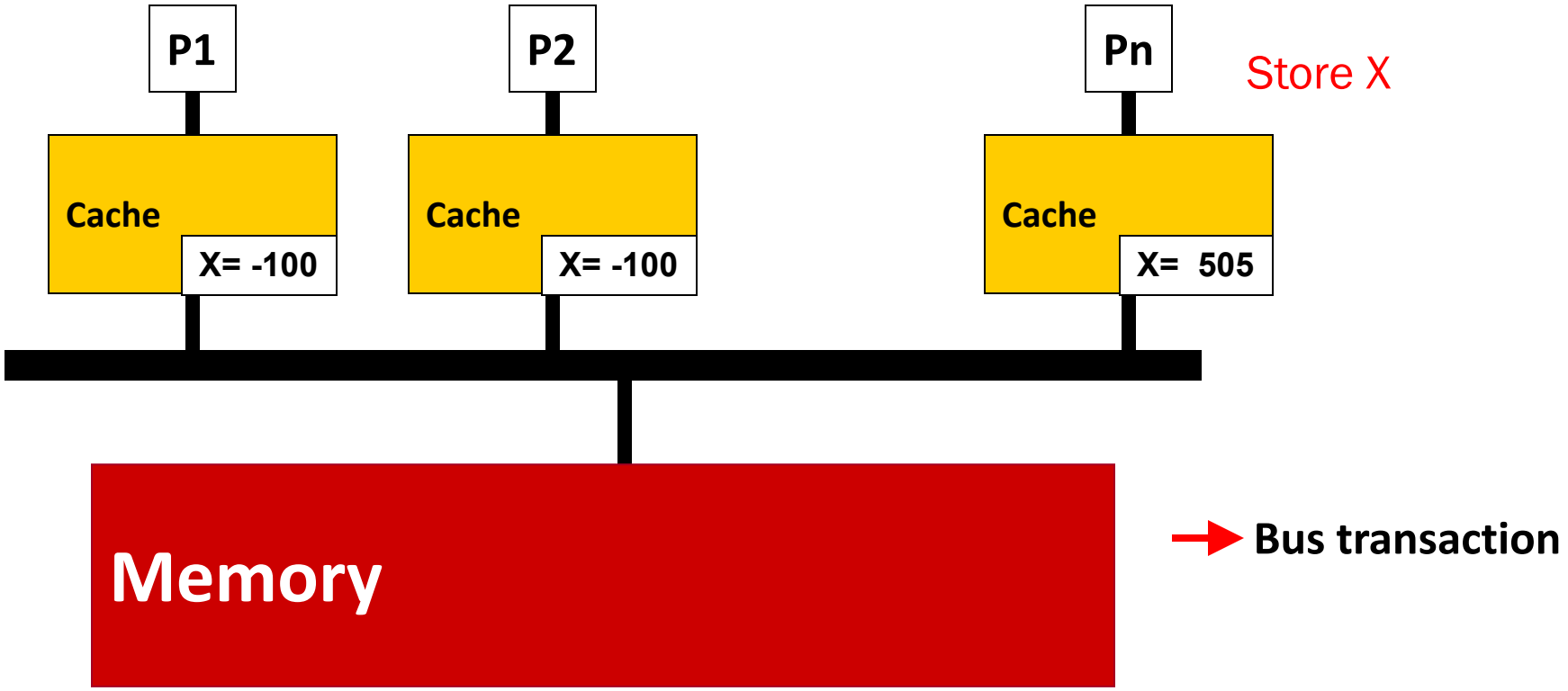
- **Write-back caches 仅在block被替换出去的时候才写数据到memory**
  - Cache和memory可能不一致
  - 减少总线写带宽 → 写操作不需要总线广播

- **Solution 1: update-based**
  - 强制让写操作在总线上广播，更新每一个copy
  - **与write-back思想不符**

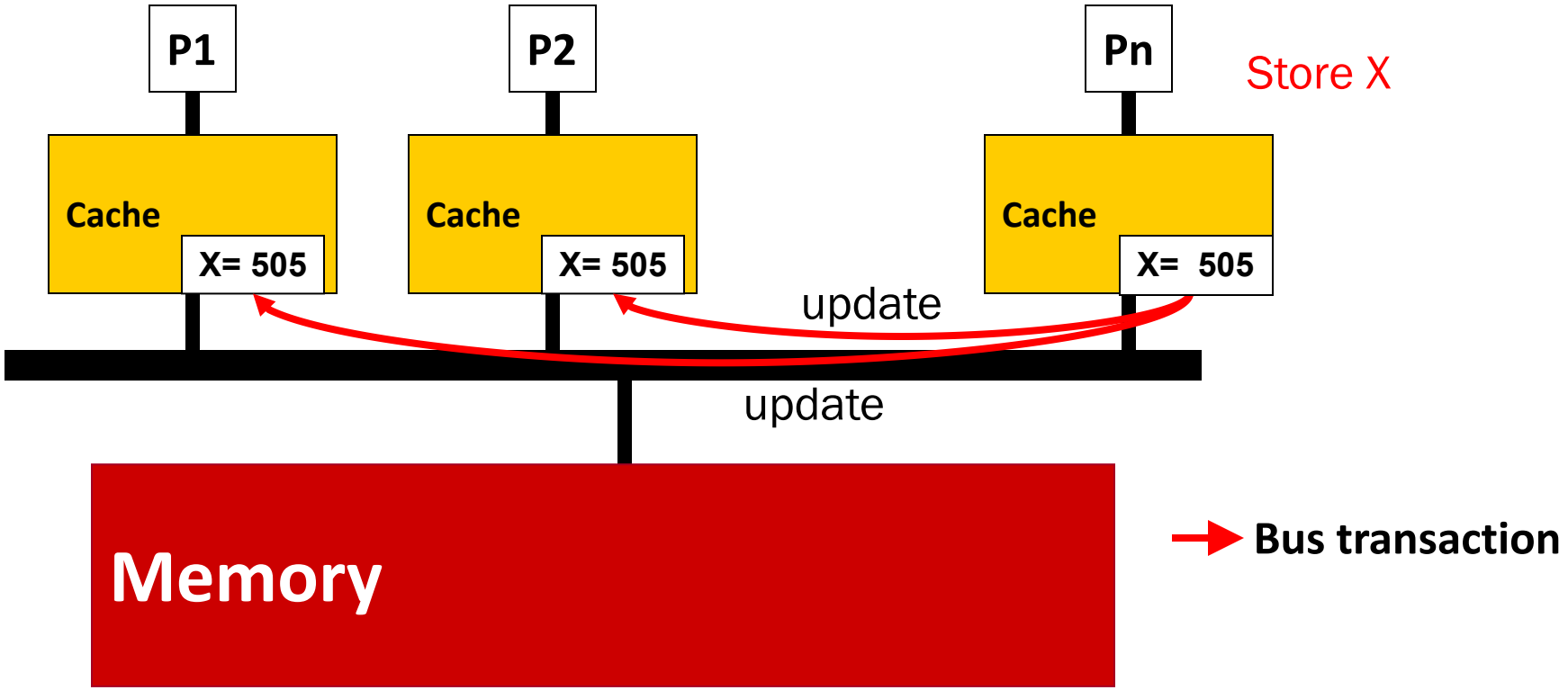
Write-back Cache：Update-based 协议



Write-back Cache：Update-based 协议



Write-back Cache：Update-based 协议



- 更新该数据在系统中所有的copy
- 和write-back理念不符!

对于Write-back Caches?

- **Write-back caches 仅在block被替换出去的时候才写数据到memory**
  - Cache和memory可能不一致
  - 减少总线写带宽 → 写操作不需要总线广播

- **Solution 1: update-based**
  - 强制让写操作在总线上广播，更新每一个copy
  - 与write-back思想不符

- **Solution 2: invalidation-based**
  - 为了符合write-back思想，需要重新设计
  - 尽量让写在本地完成，减少总线事物

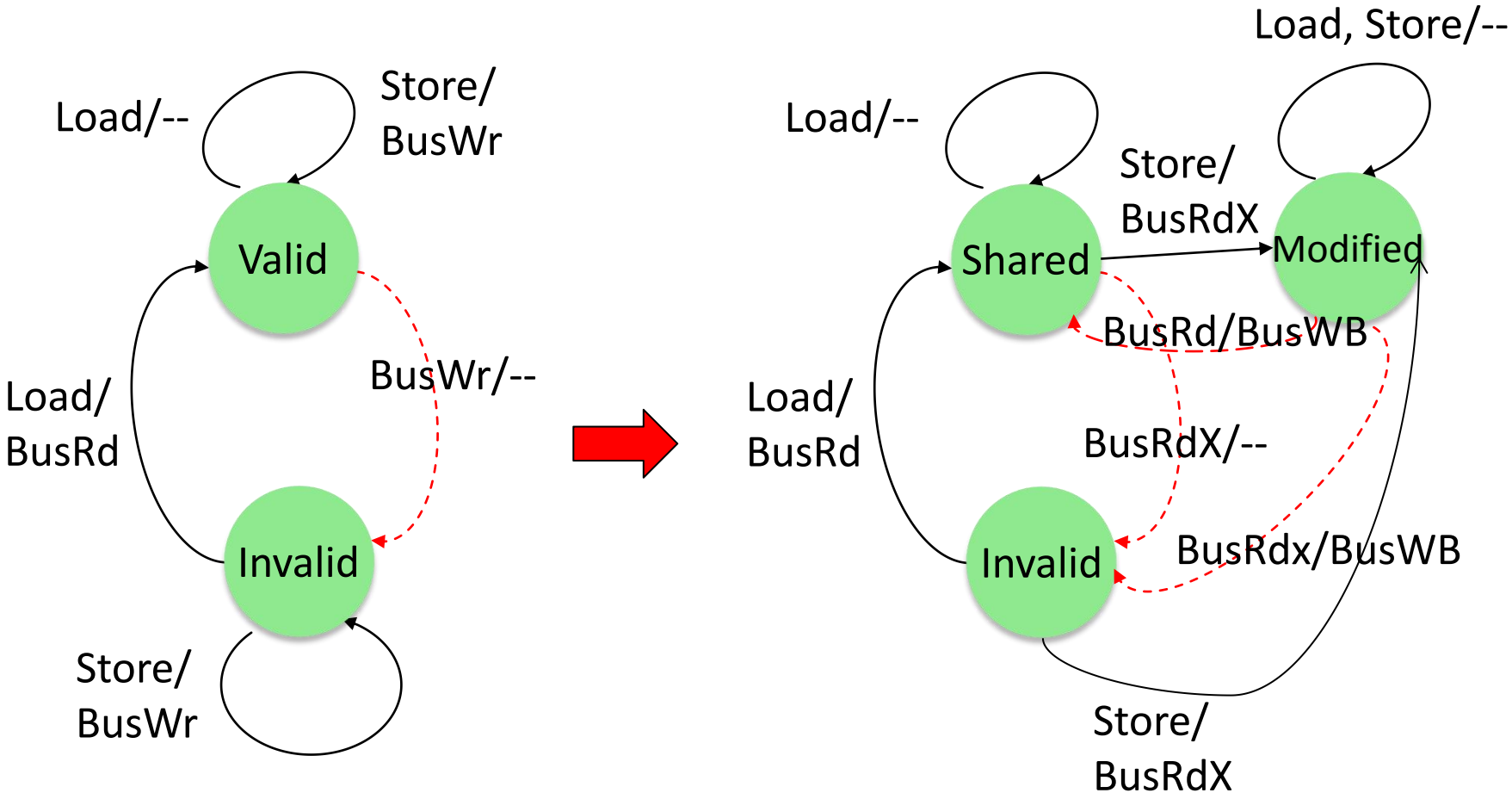
Write-back Cache：MSI 协议

- **每个cache block有3种状态**
  - ***Invalid*** ：本地没有copy
  - ***Shared*** ：本地有一个只读copy，该copy有最新的数据
  - ***Modified*** ：本地有全局唯一的valid copy(其他cache无copy)，该copy处于dirty状态(可能和内存不一致)，本地可写(不需要通知他人)

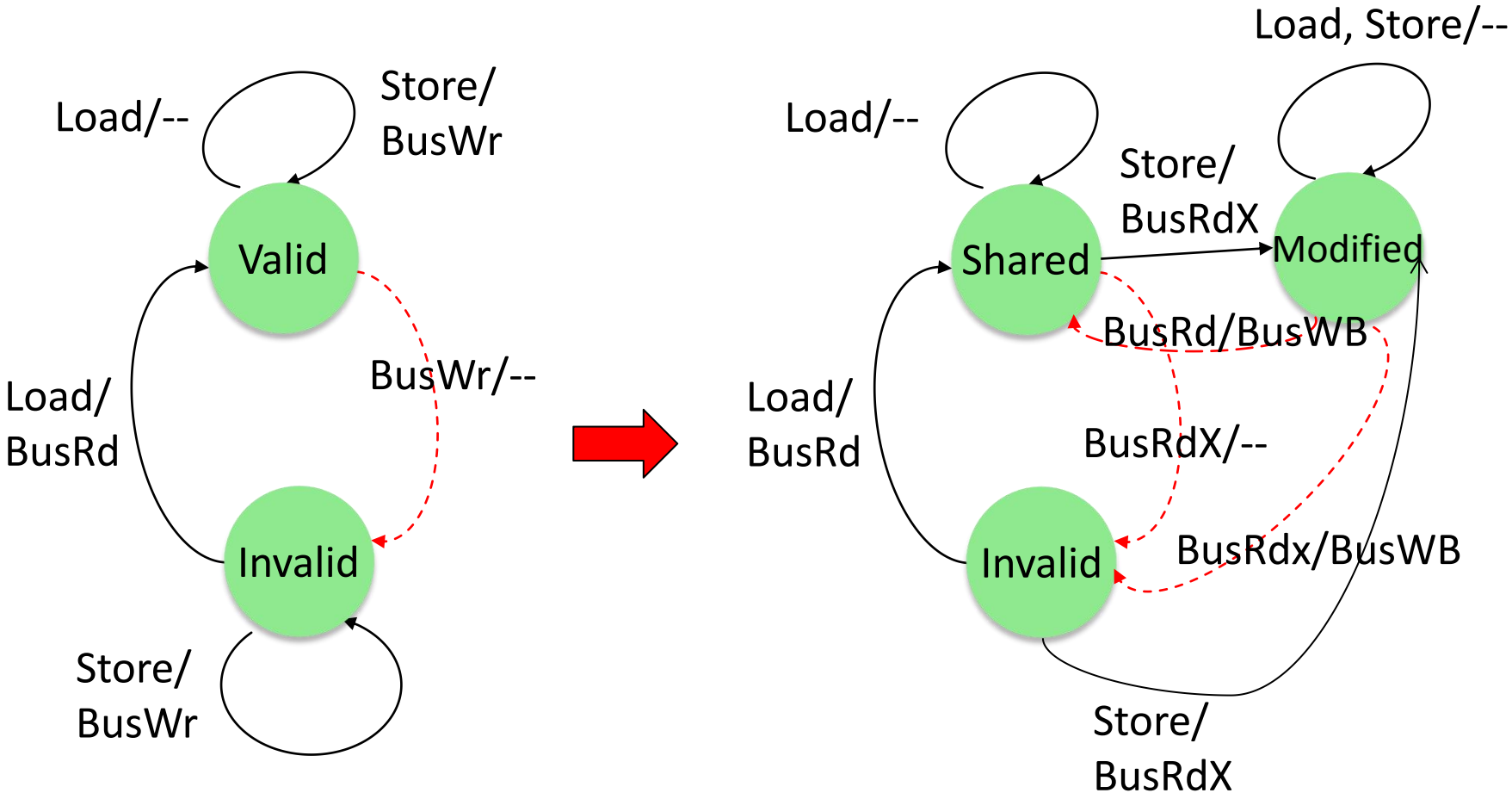
- **处理器的动作**
  - load, store, Evict(cache被换出)

- **总线事物**
  - BusRd (load), BusRdX (store), BusWB (write back), …

MSI 协议状态转换

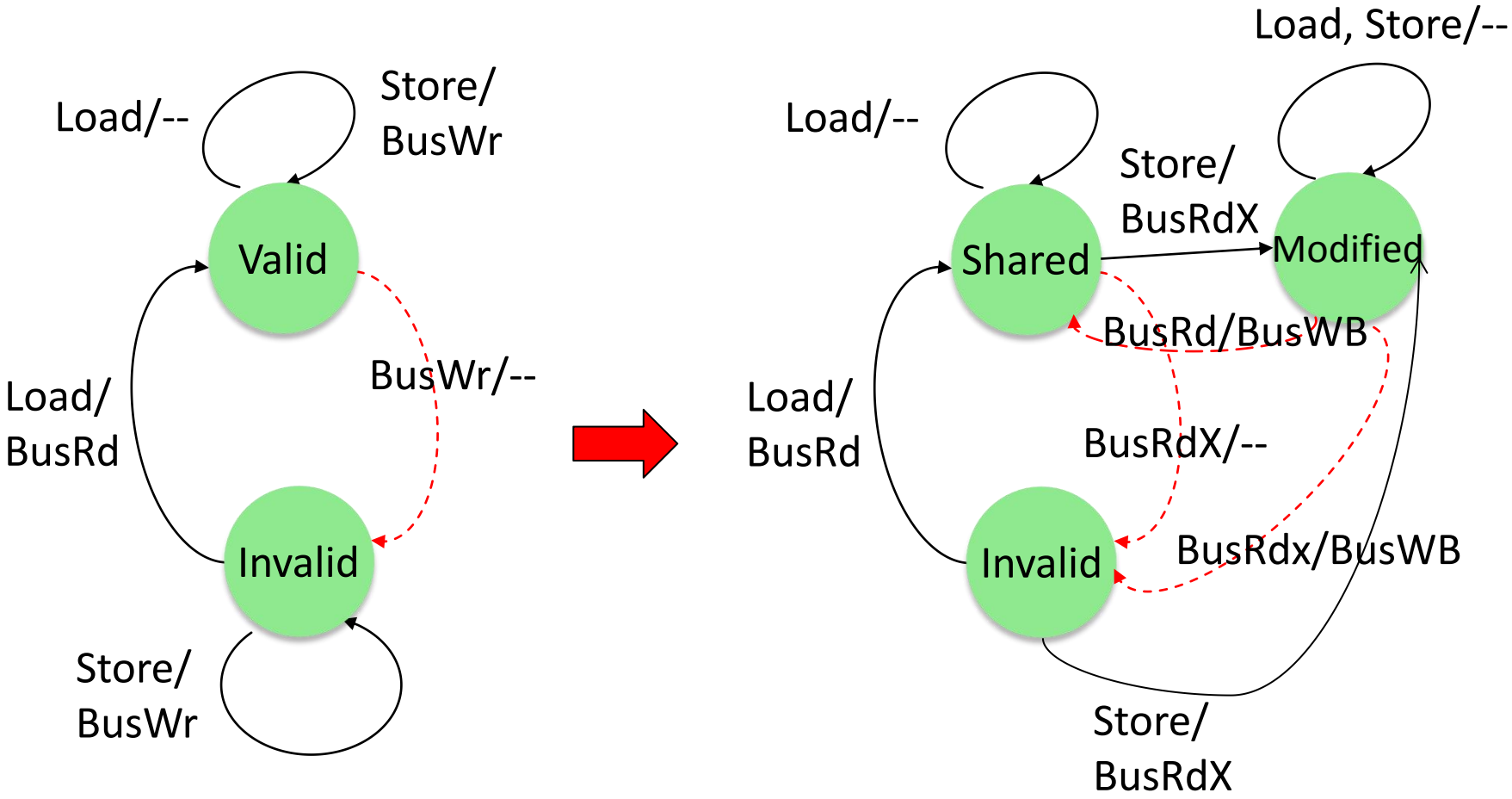

Load/--
Store/
BusWr
Valid
Load/
BusRd
BusWr/--
Invalid
Store/
BusWr
Load, Store/--
Load/--
Store/
BusRdX
Shared
Modified
BusRd/BusWB
Load/
BusRd
BusRdX/--
Invalid
BusRdx/BusWB
Store/
BusRdX

MSI 协议状态转换



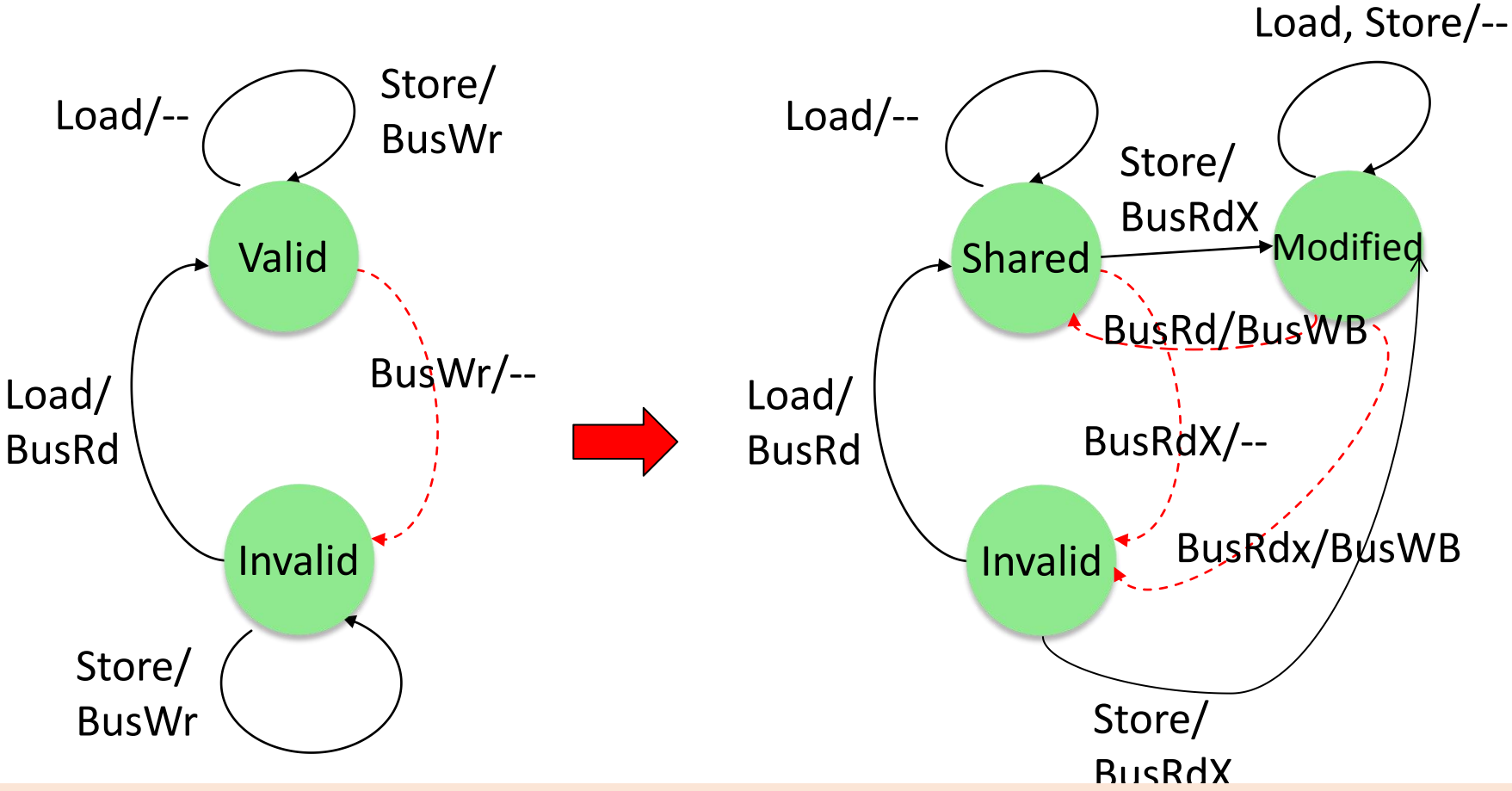
本地处于shared状态：本地读操作，保持shared状态; 本地写操作，先向总线发起BusRdX请求，将自己变为modified状态；如果收到总线BurRdX(别人写)，那么变为invalid状态

MSI 协议状态转换



本地处于invalid状态：本地读，先向总线发送BusRd，将自己变为shared；本地写（发生write miss），自己变为modified，并向总线发出BusRdx

MSI 协议状态转换

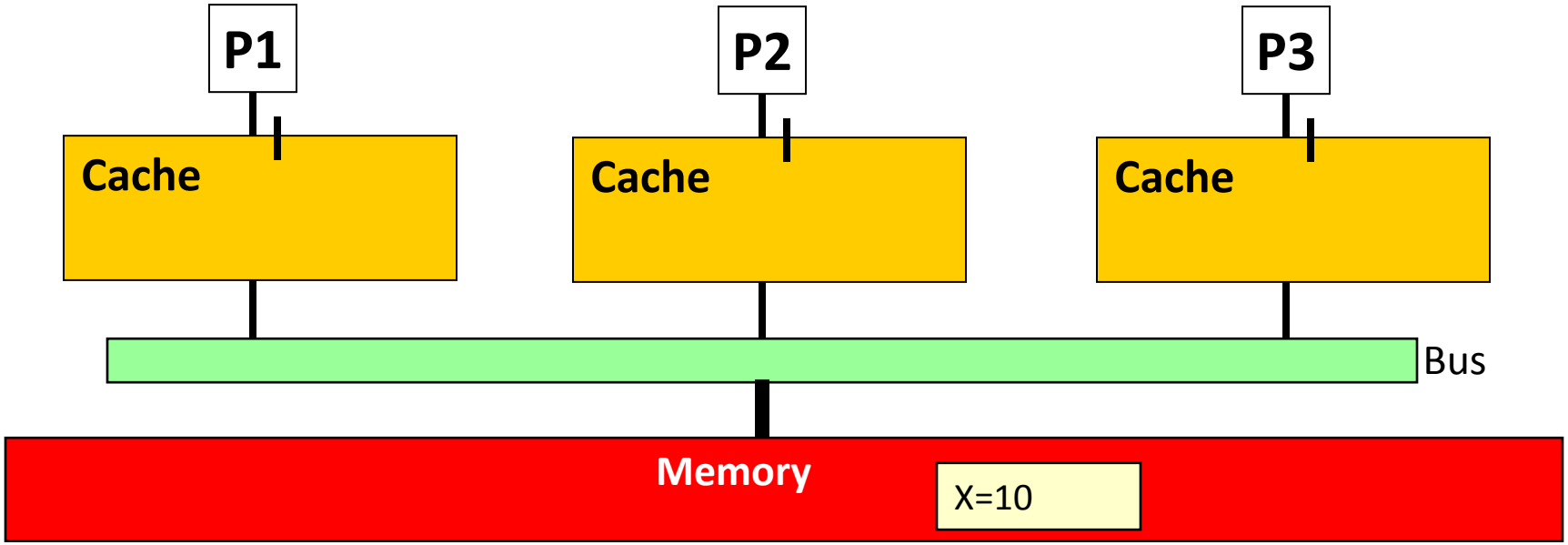


本地处于modified状态：本地读或者写，保持modified不变；如果收到总线BusRD(表示别人读)，自己变为shared，同时发起BusWB(自己的数据是dirty的，需要写回)，将最新值广播出去并写回内存；如果收到BusRdx，表明别人发生了write miss，自己变为invalid，同时发起BusWB

MSI 协议状态转换

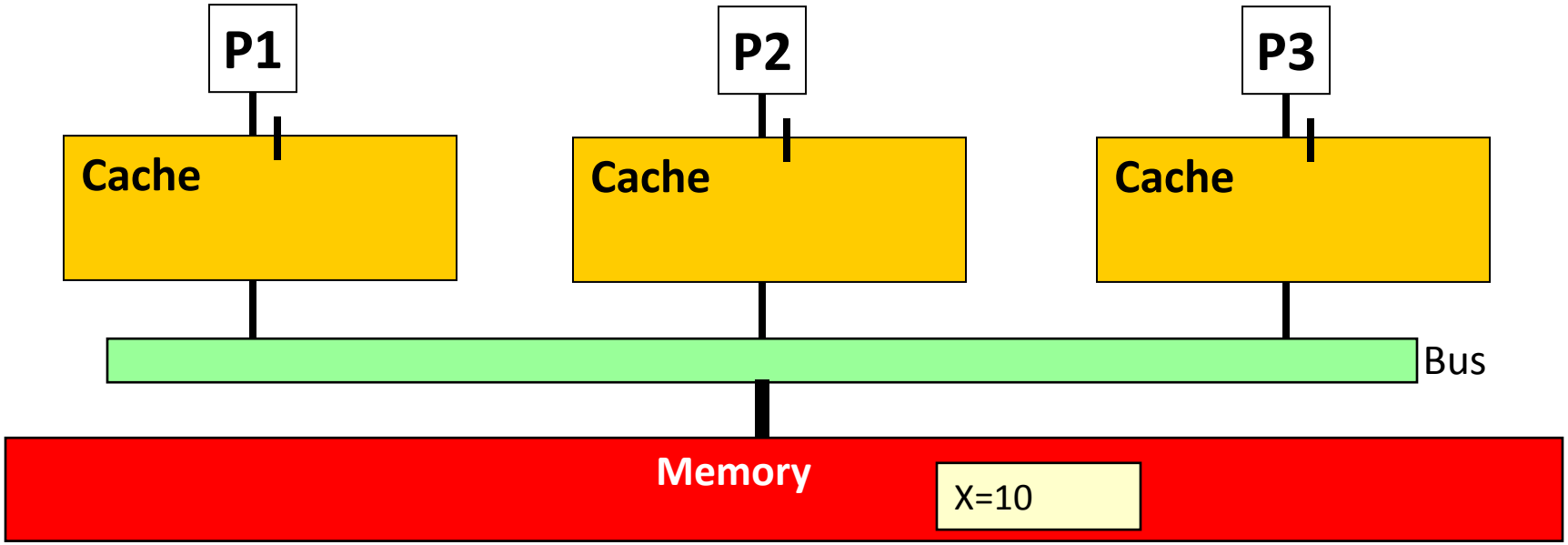
State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Load Miss ➔ S	Store Miss ➔ M	---	---
Shared (S)	Hit	➔ M	➔ S	➔ I
Modified (M)	Hit	Hit	➔ S	➔ I

MSI Example



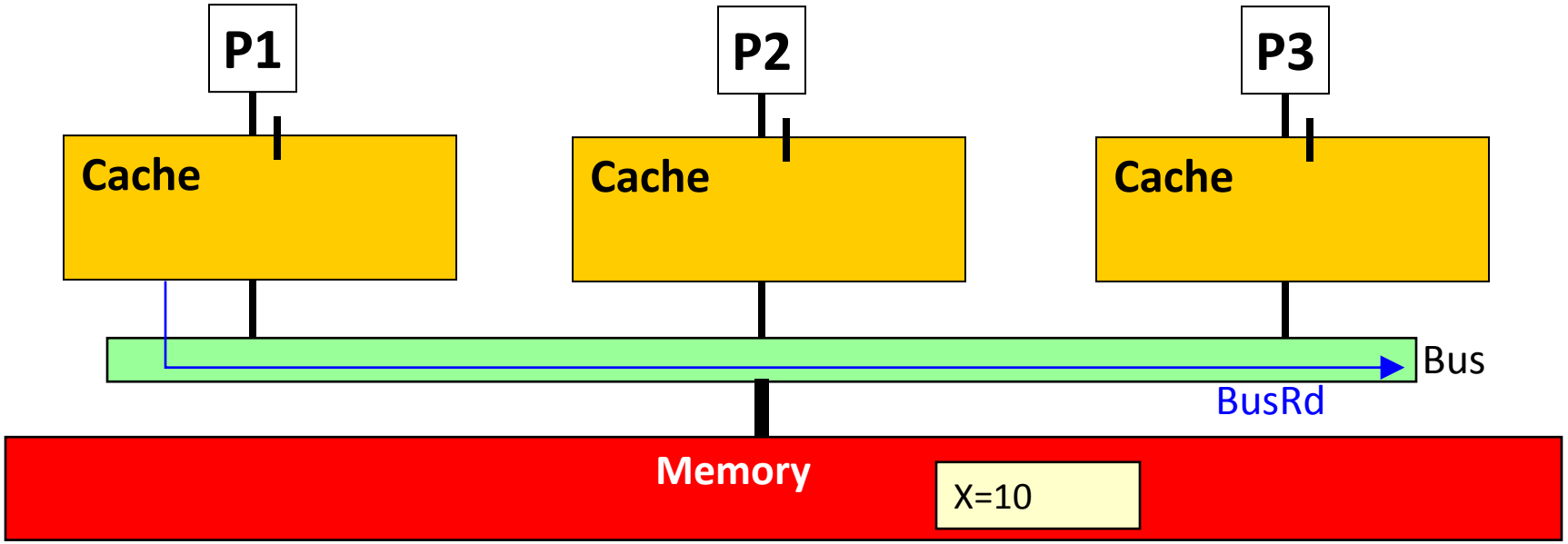
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier

MSI Example



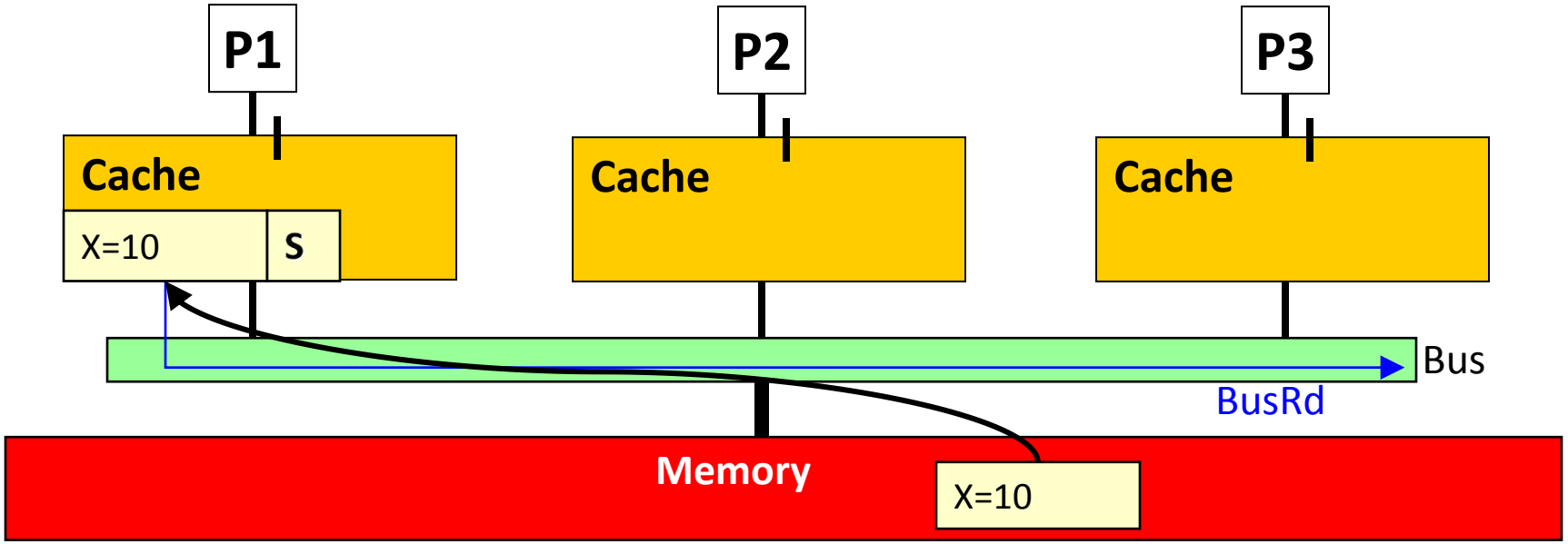
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X					

MSI Example



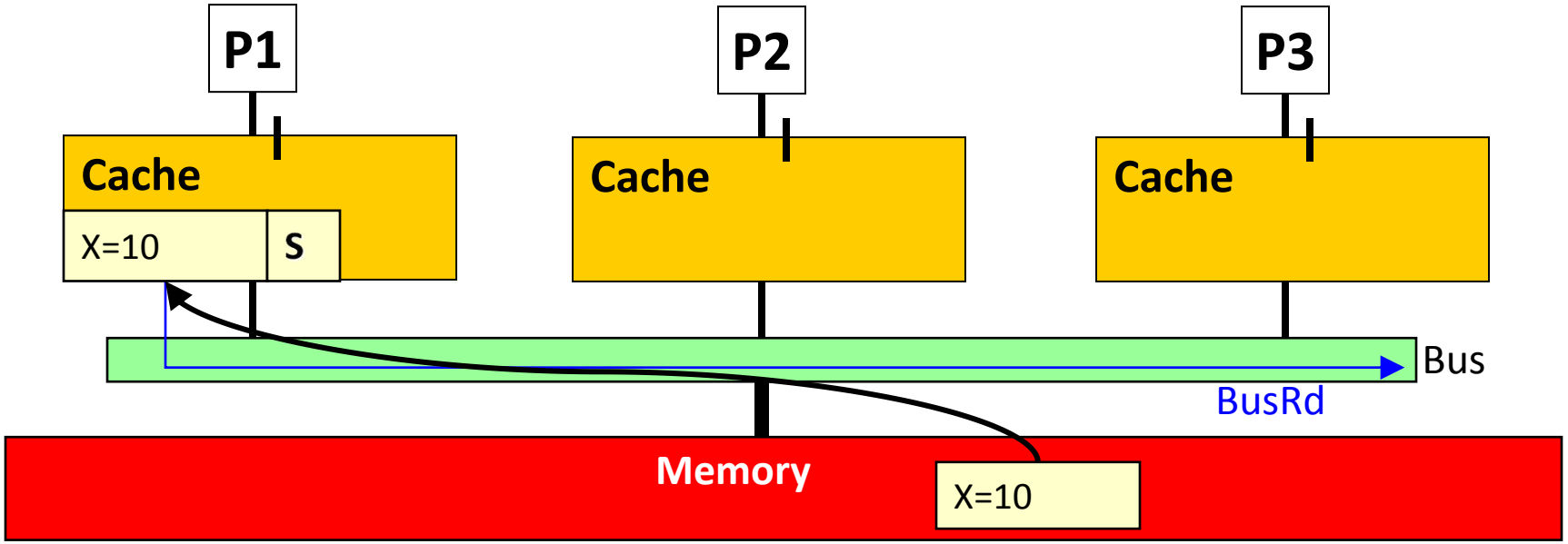
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X					

MSI Example



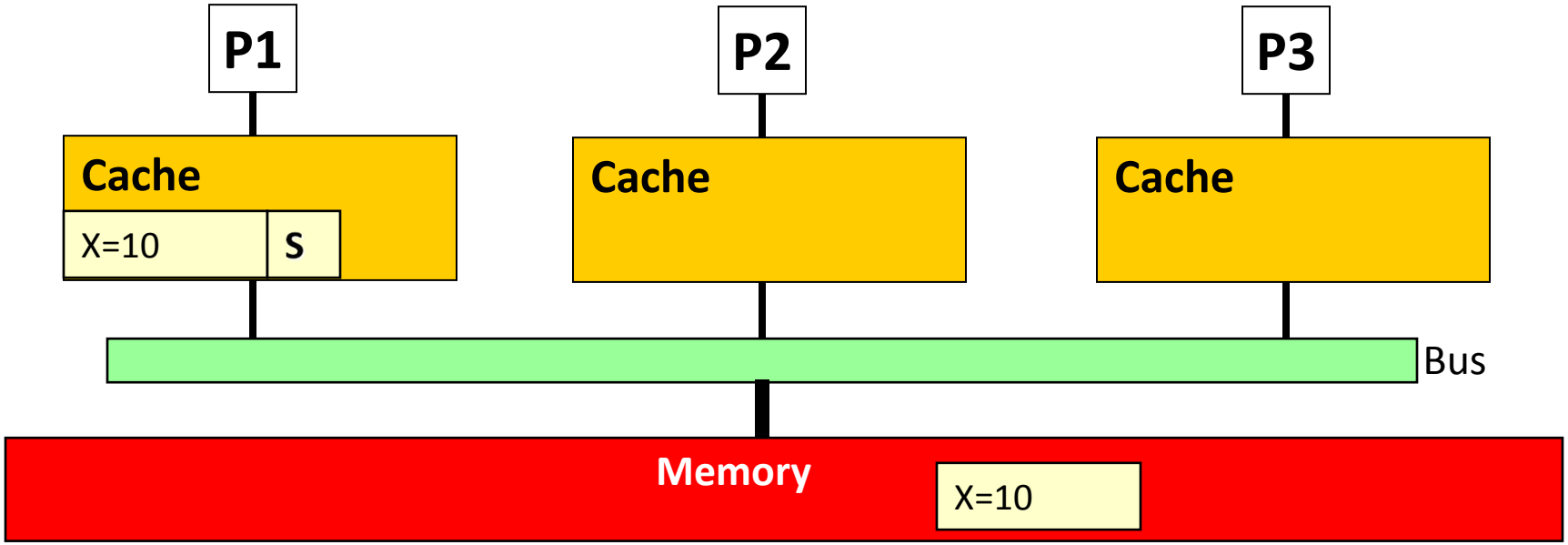
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X					

MSI Example



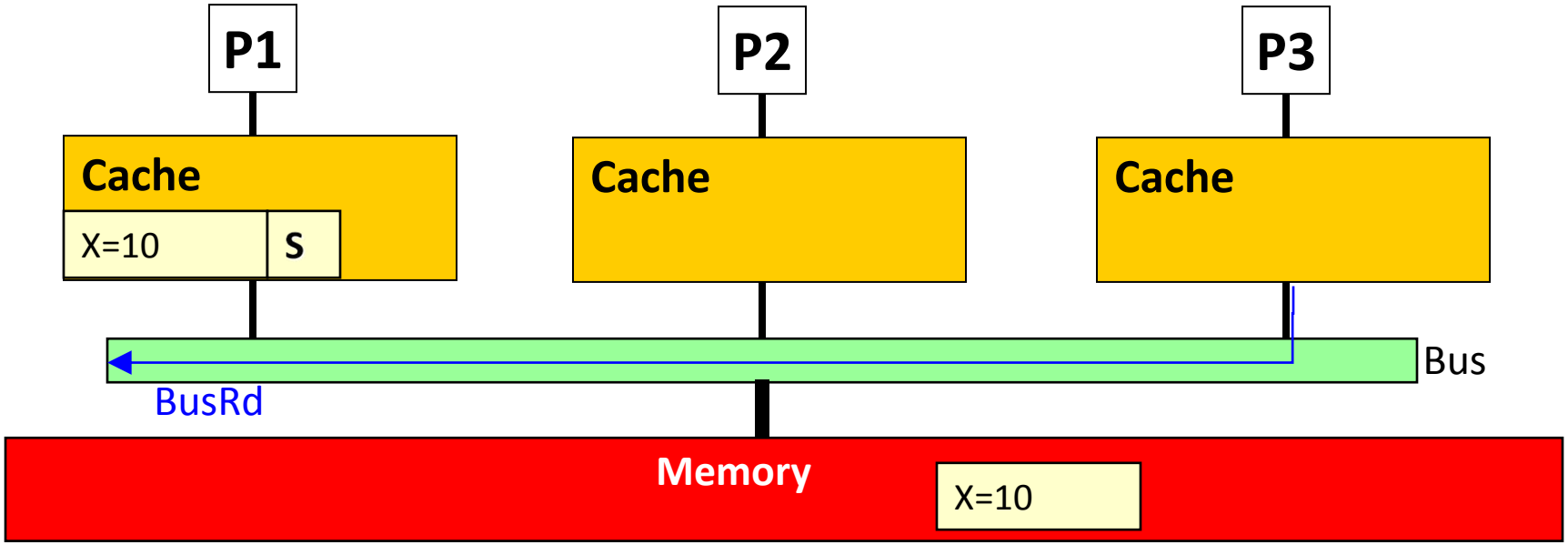
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory

MSI Example



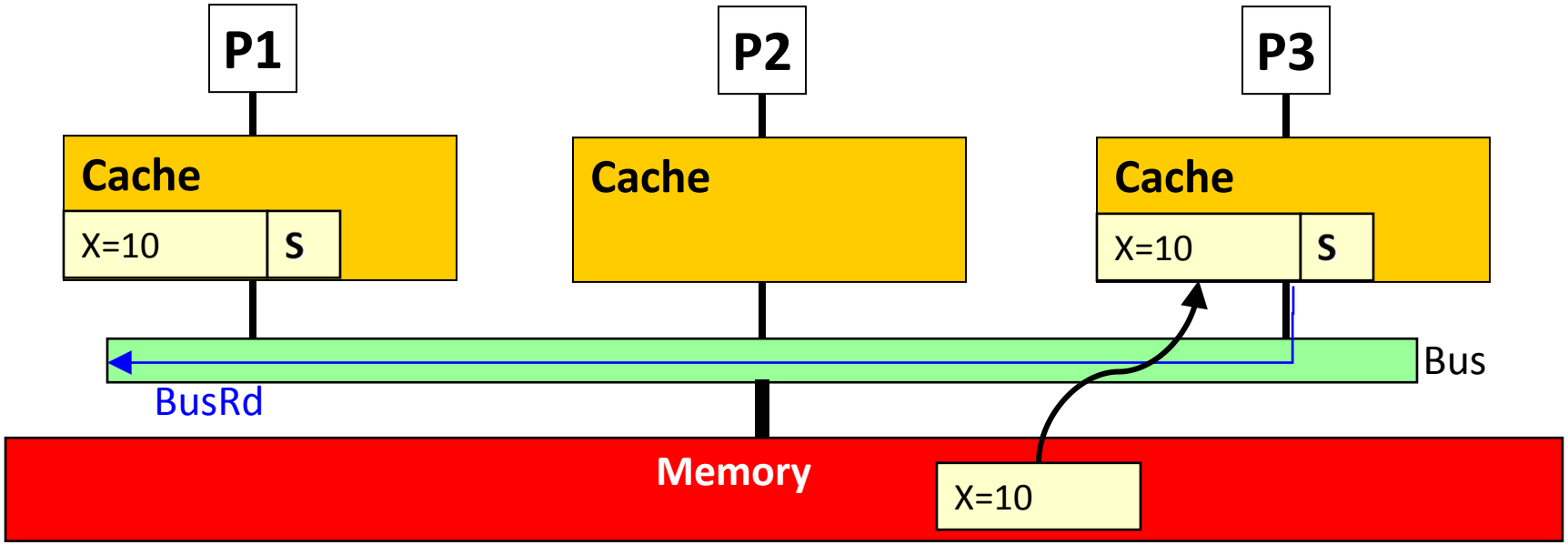
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X					

MSI Example



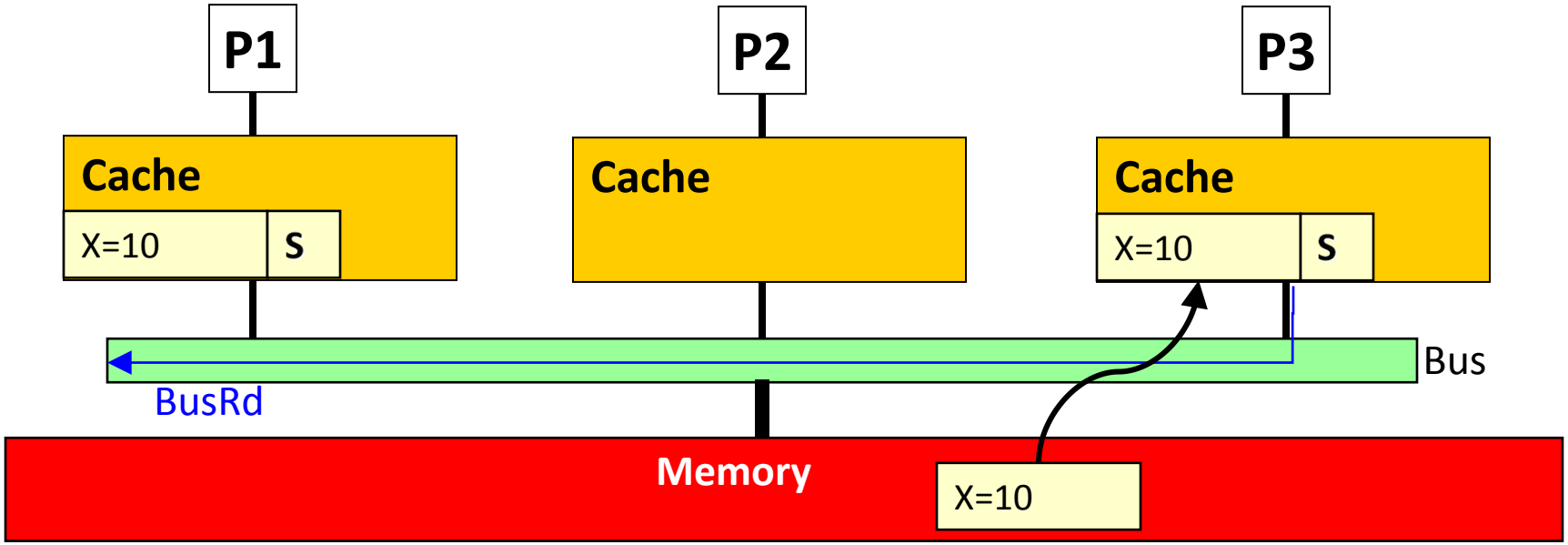
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X					

MSI Example



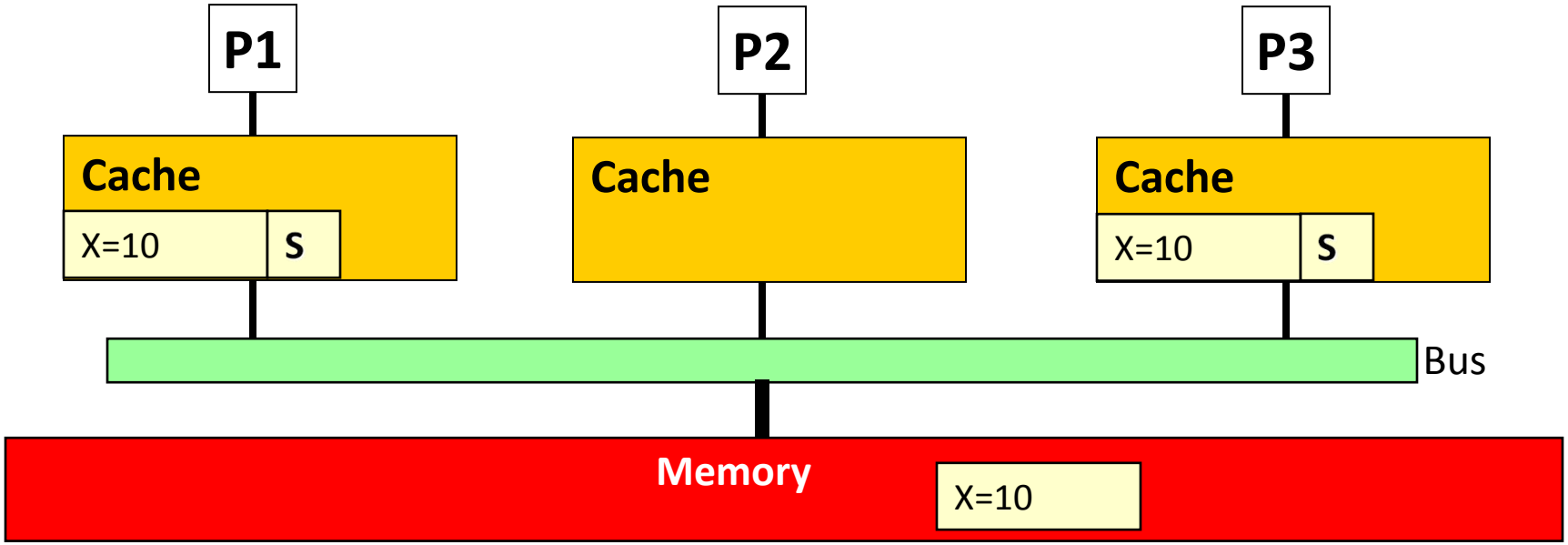
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X					

MSI Example



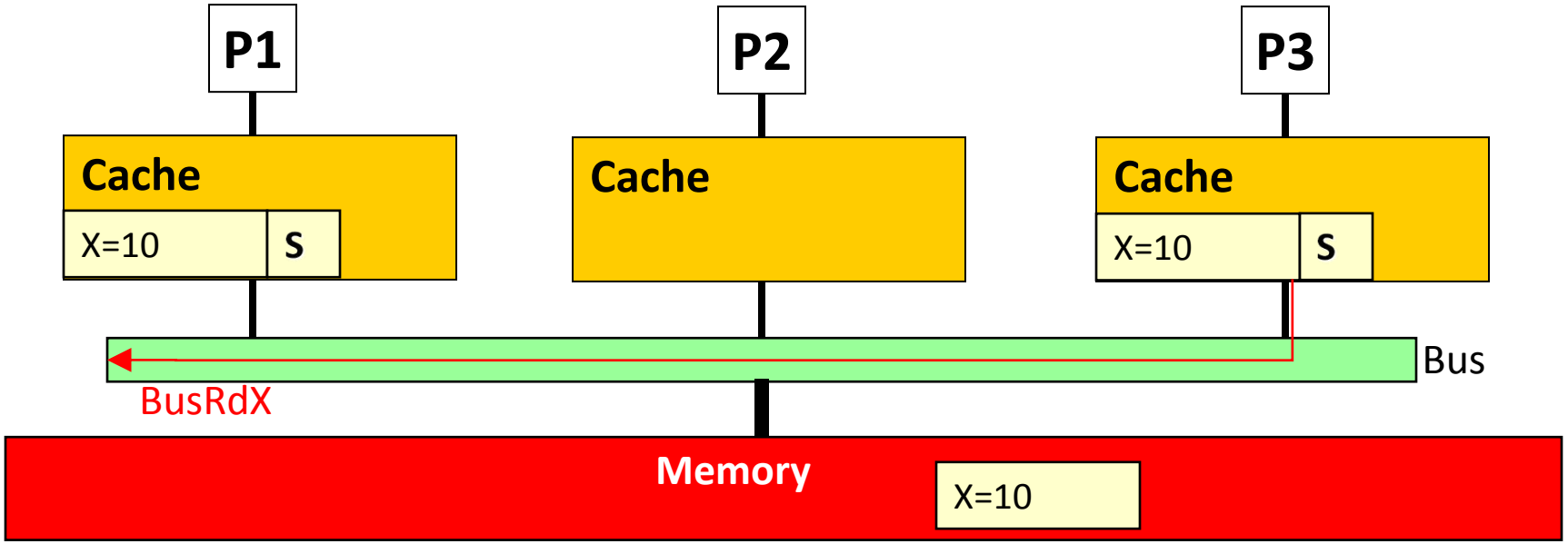
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory

MSI Example



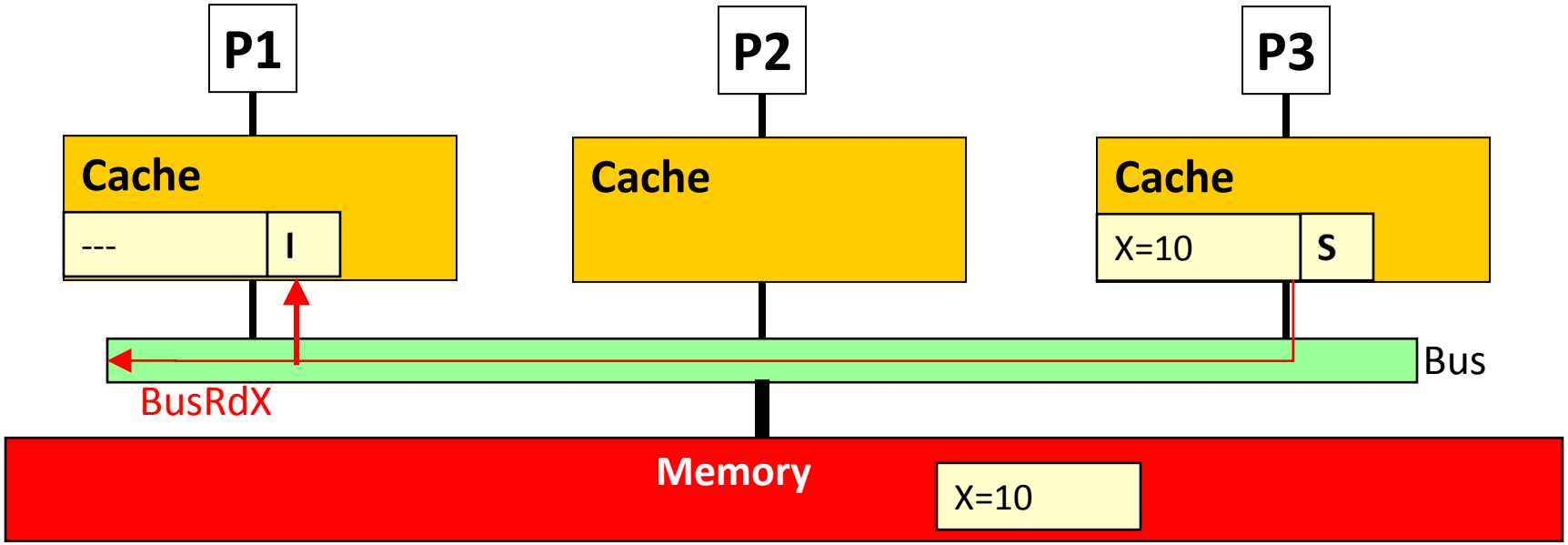
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X					

MSI Example



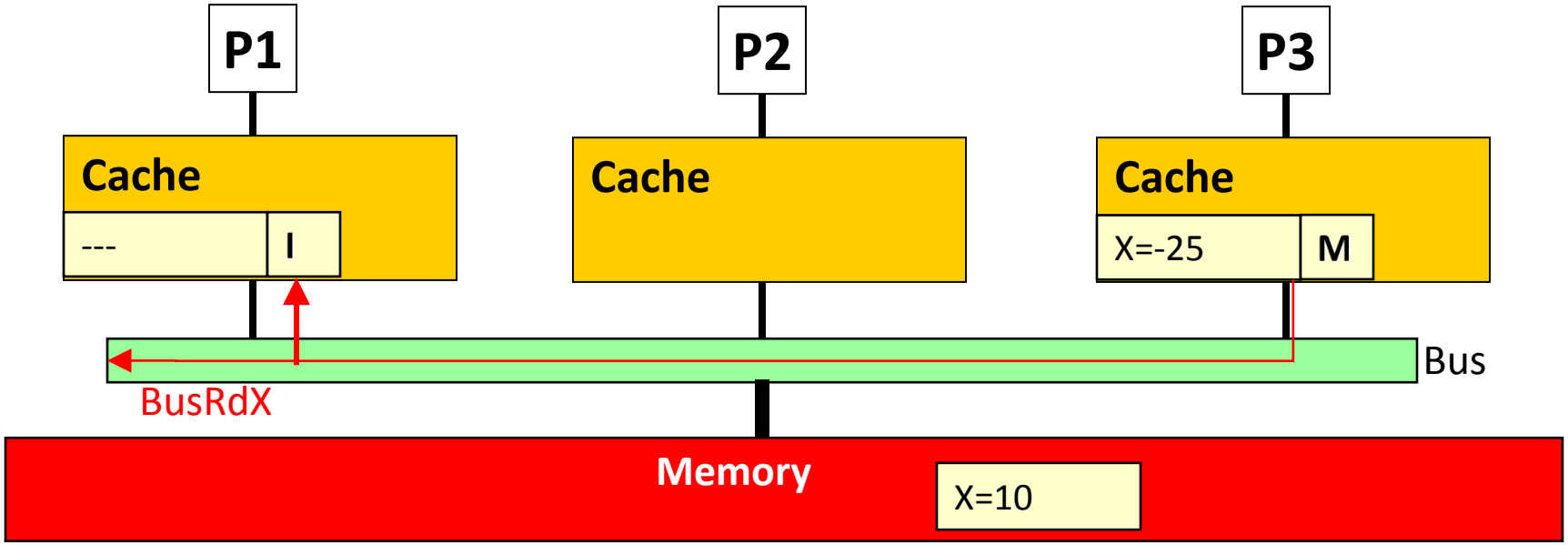
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X					

MSI Example



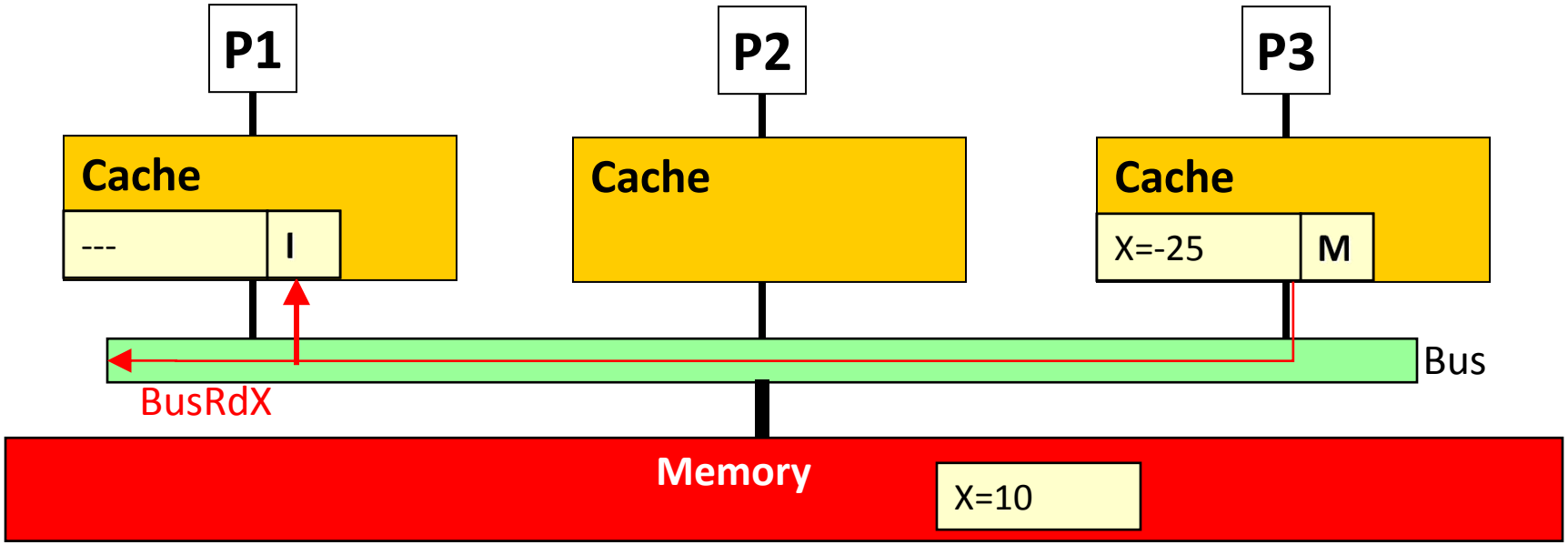
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X					

MSI Example



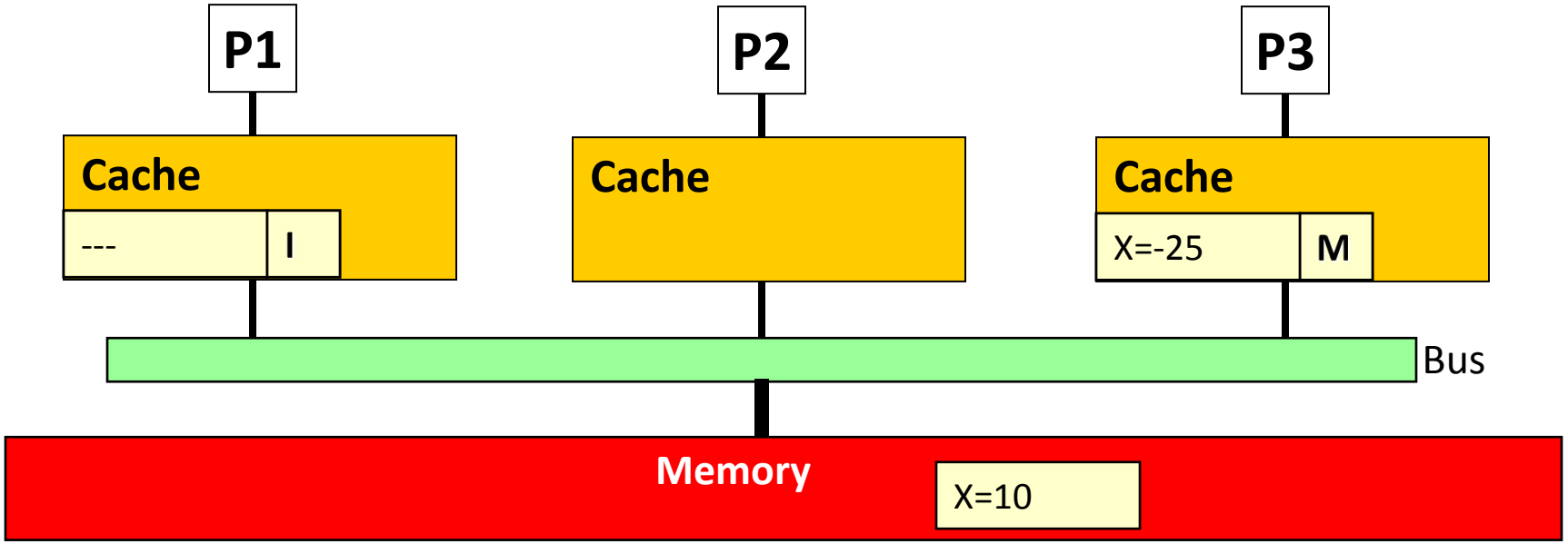
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X					

MSI Example



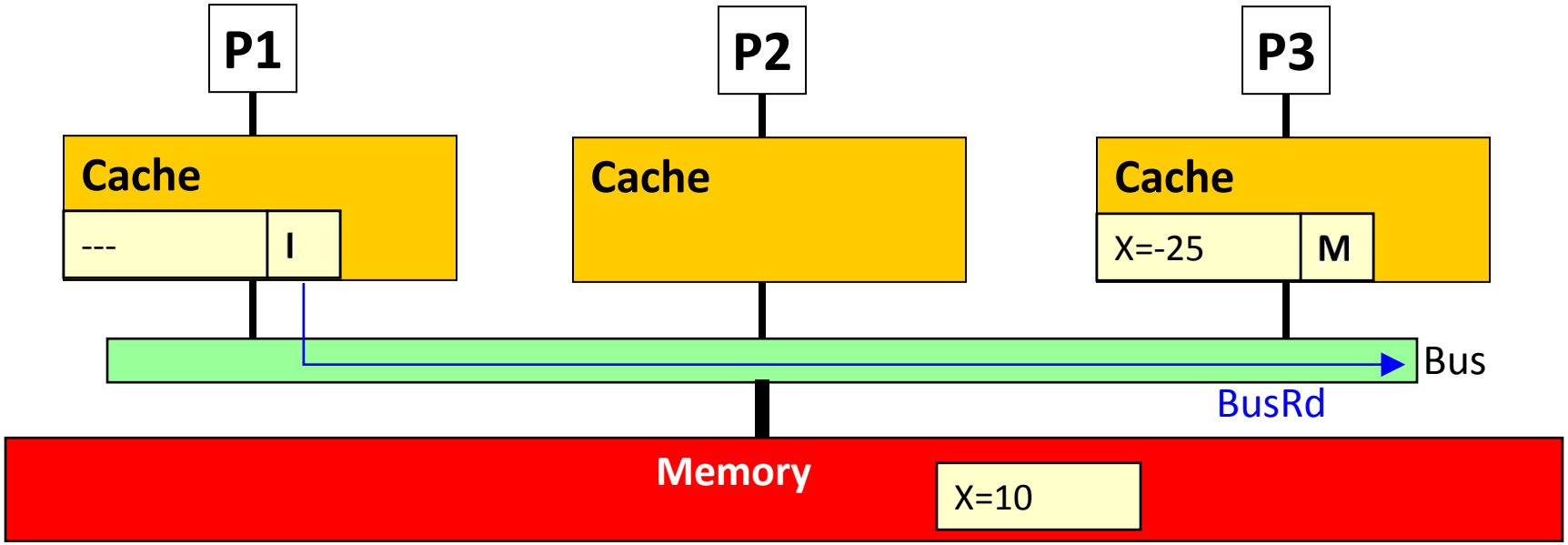
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---

MSI Example



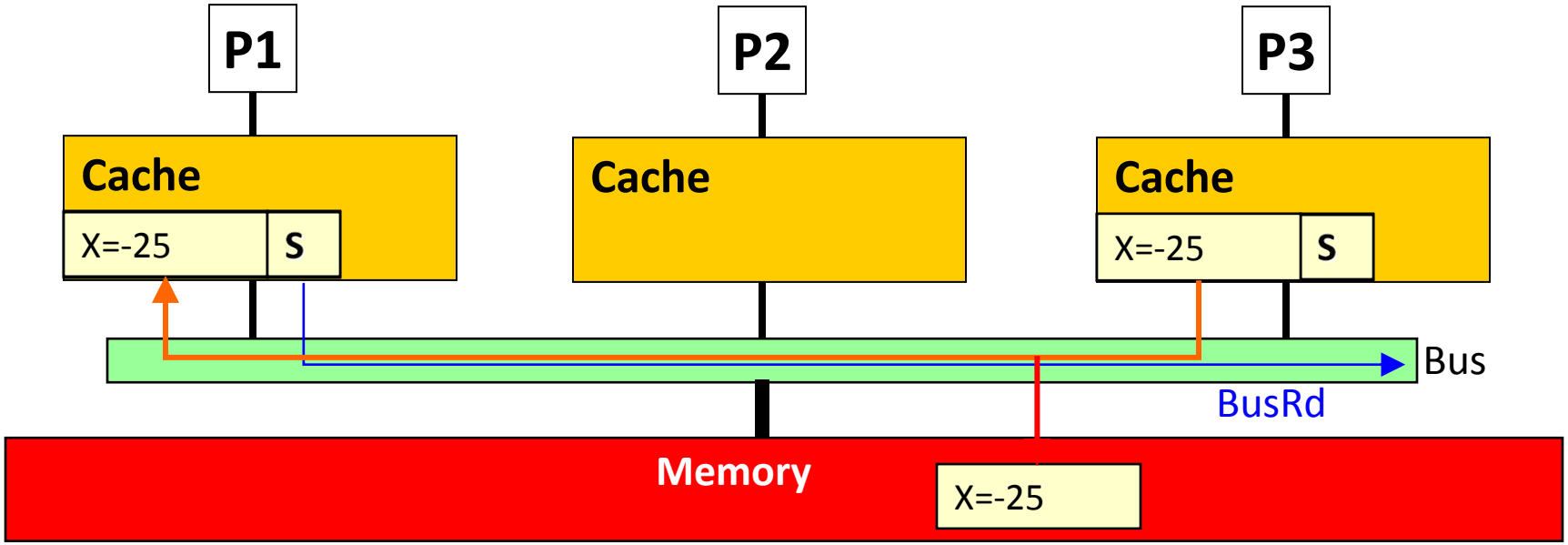
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X					

MSI Example



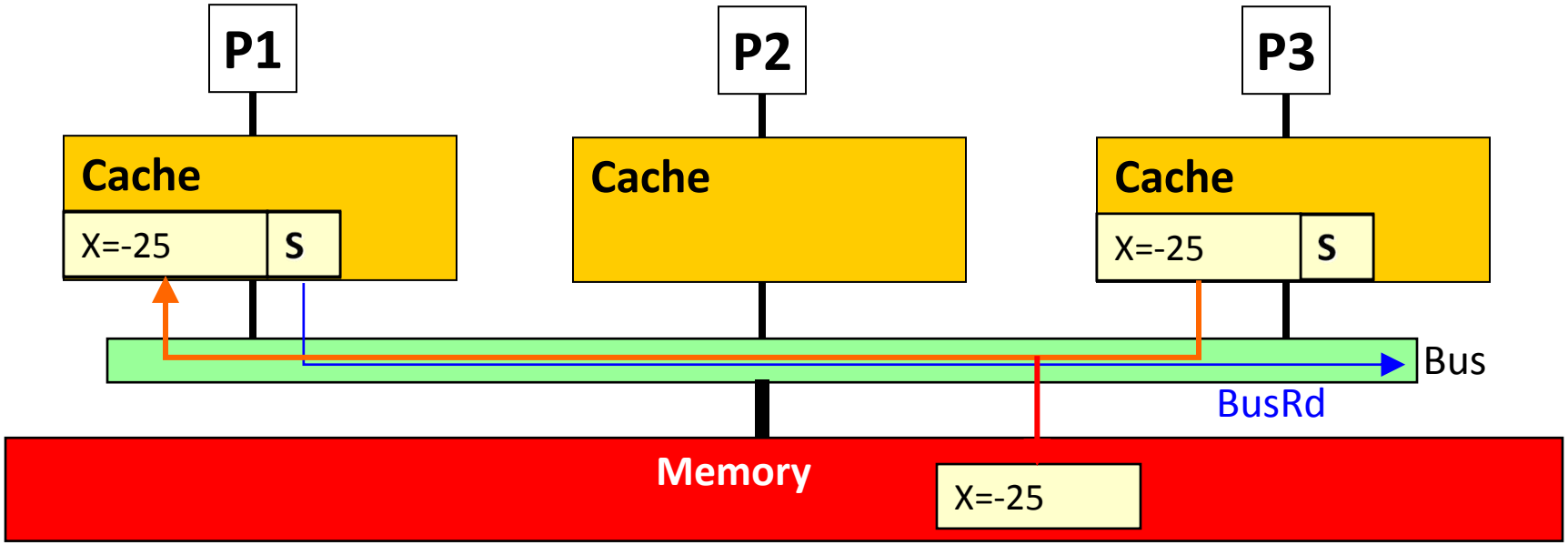
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X					

MSI Example



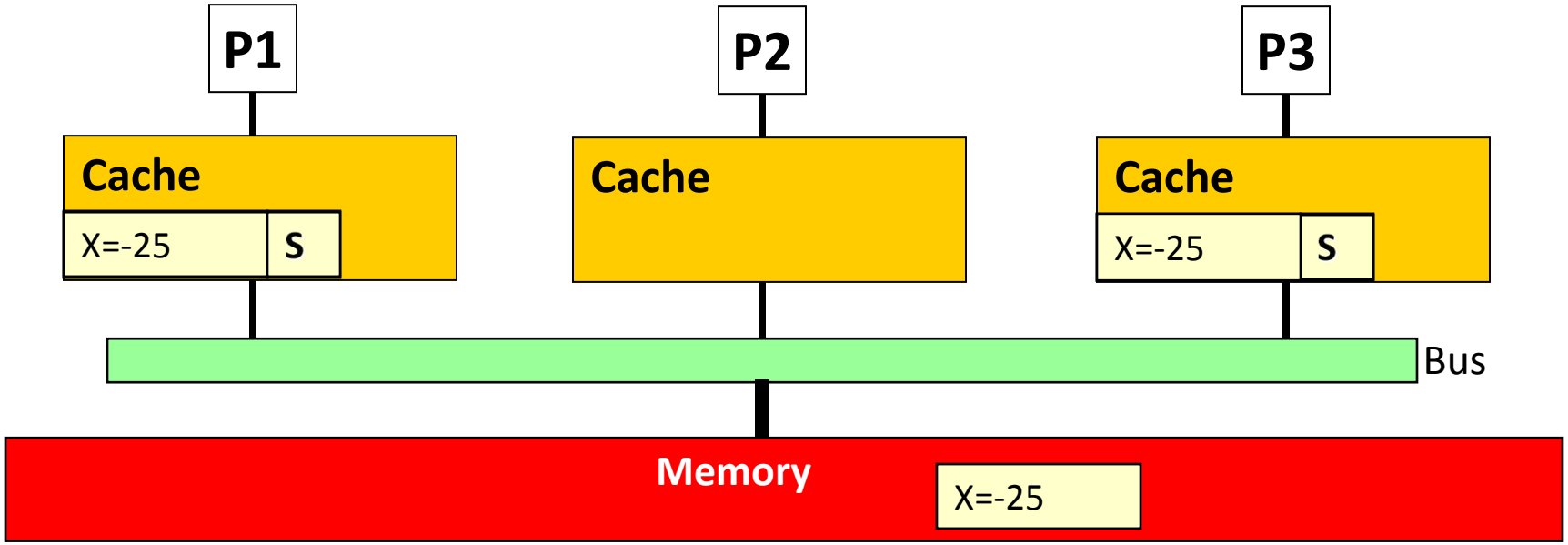
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X					

MSI Example



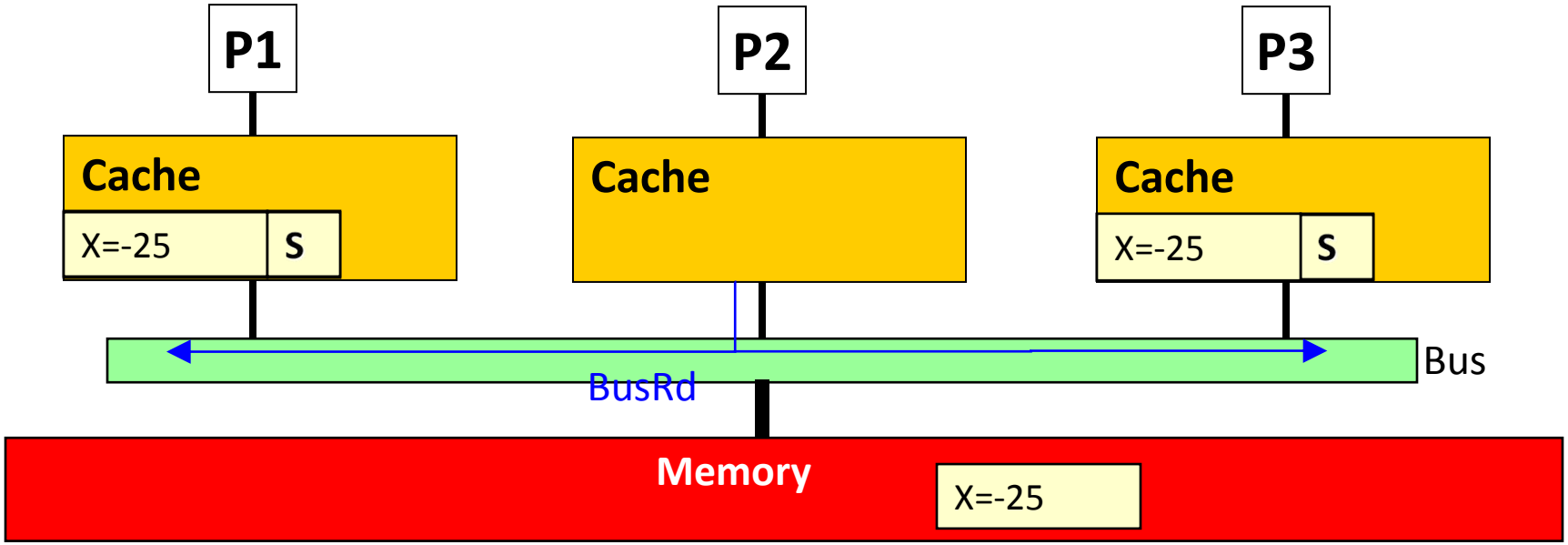
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X	S	---	S	BusRd	P3 Cache

MSI Example



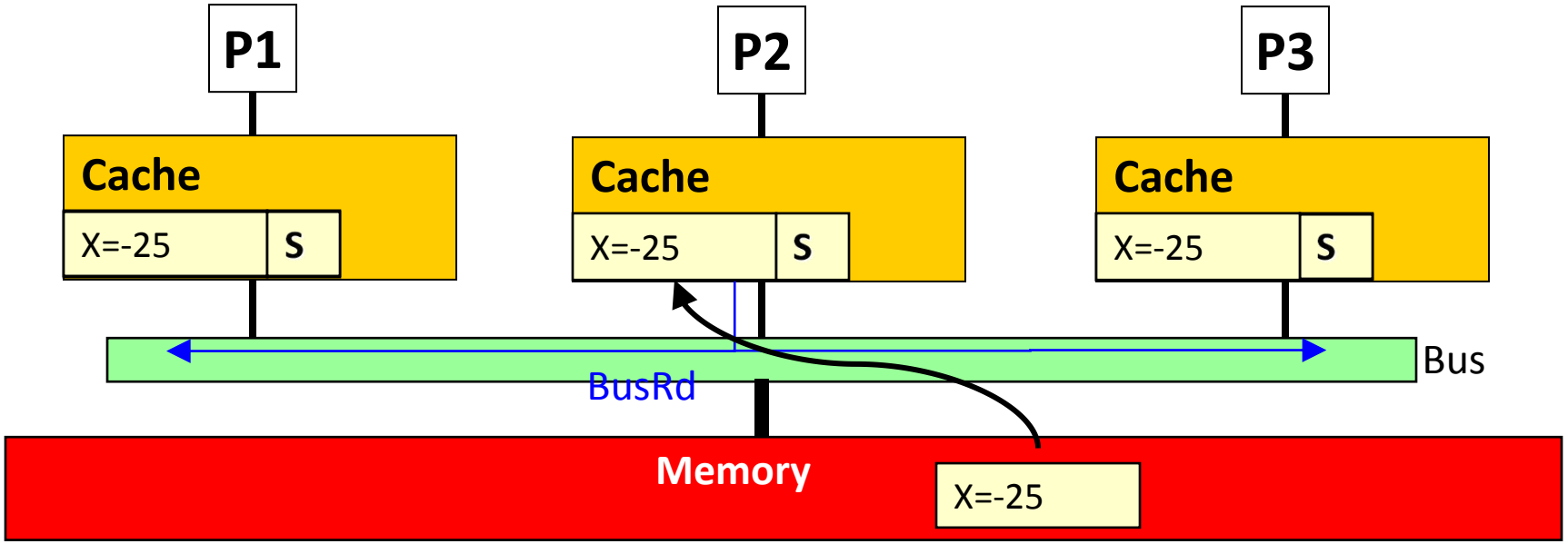
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X	S	---	S	BusRd	P3 Cache
P2 reads X					

MSI Example



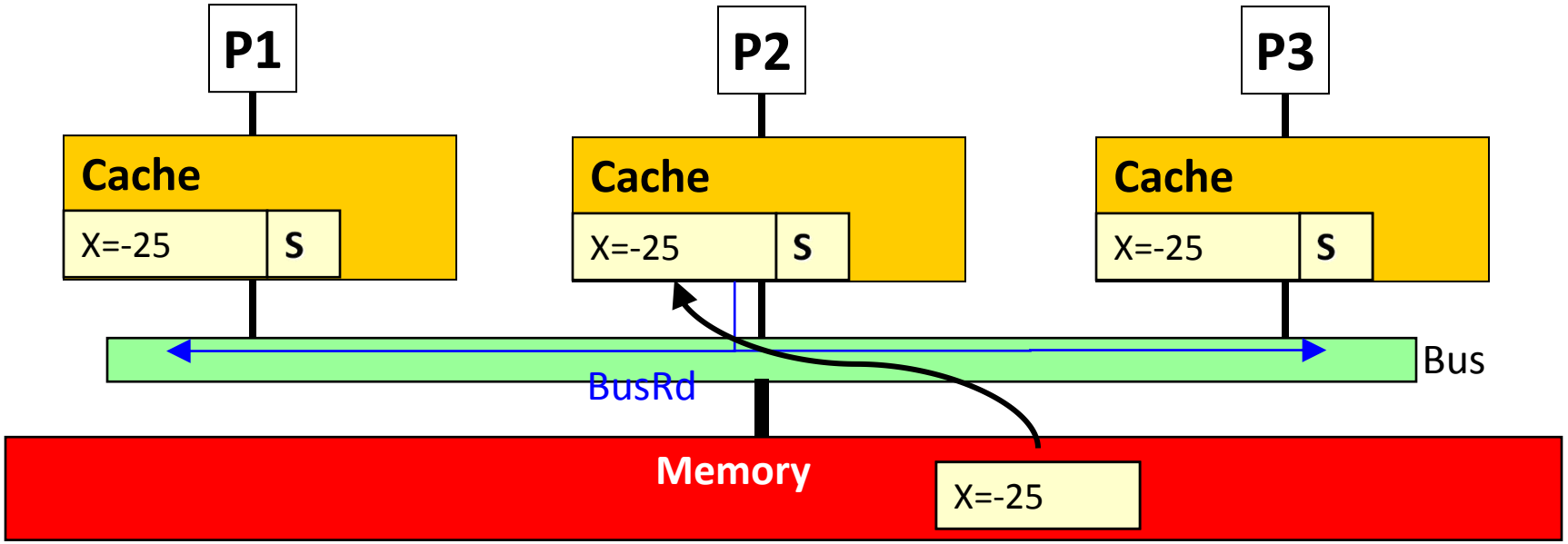
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X	S	---	S	BusRd	P3 Cache
P2 reads X					

MSI Example



Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X	S	---	S	BusRd	P3 Cache
P2 reads X					

MSI Example

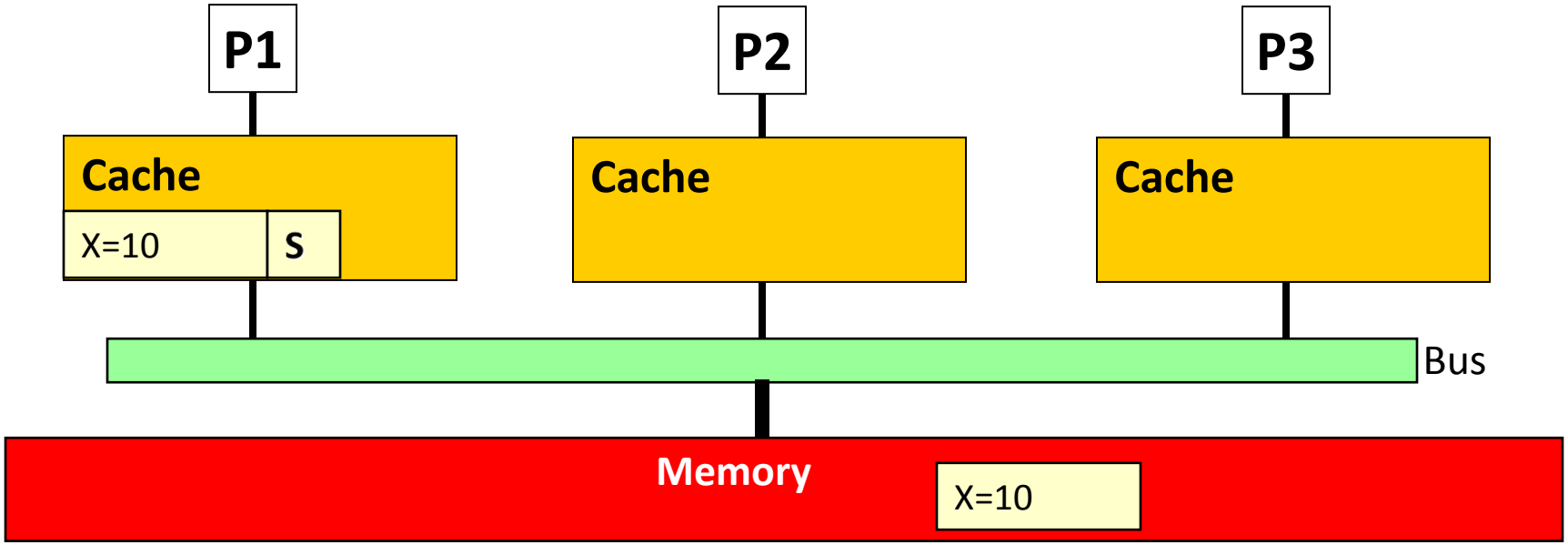


Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P3 reads X	S	---	S	BusRd	Memory
P3 writes X	I	---	M	BusRdX	---
P1 reads X	S	---	S	BusRd	P3 Cache
P2 reads X	S	S	S	BusRd	Memory

MSI 协议存在的问题

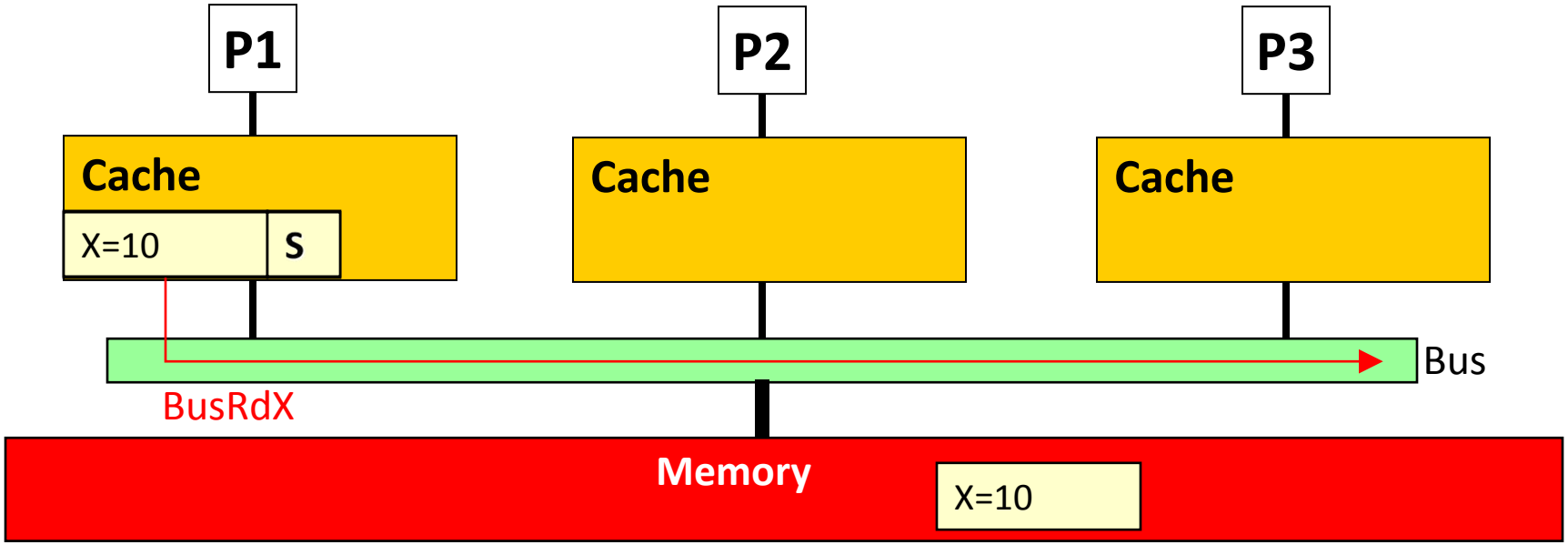
- **如果本地有全局唯一的copy，处于shared状态**
  - 此时**发生本地write**，会在总线上广播BusRdX，自己升级为modified，然后再写
  - 因为全局只有一个copy，其实没必要广播消息

MSI 协议存在的问题



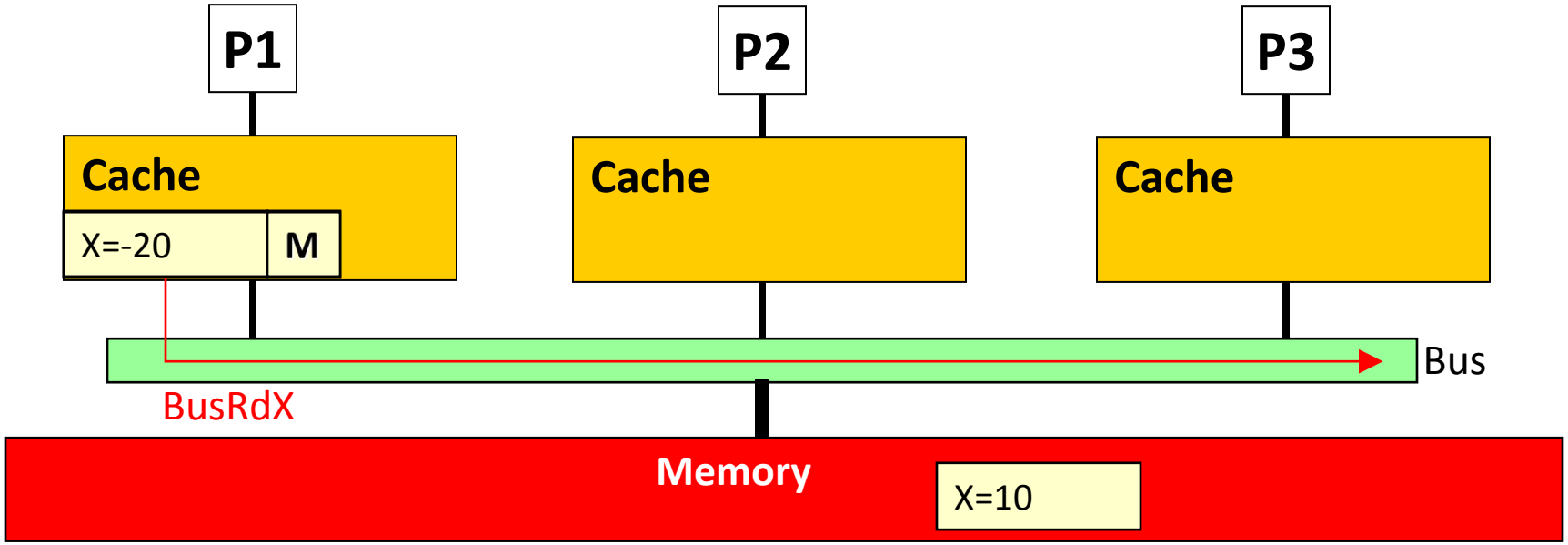
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory

MSI 协议存在的问题



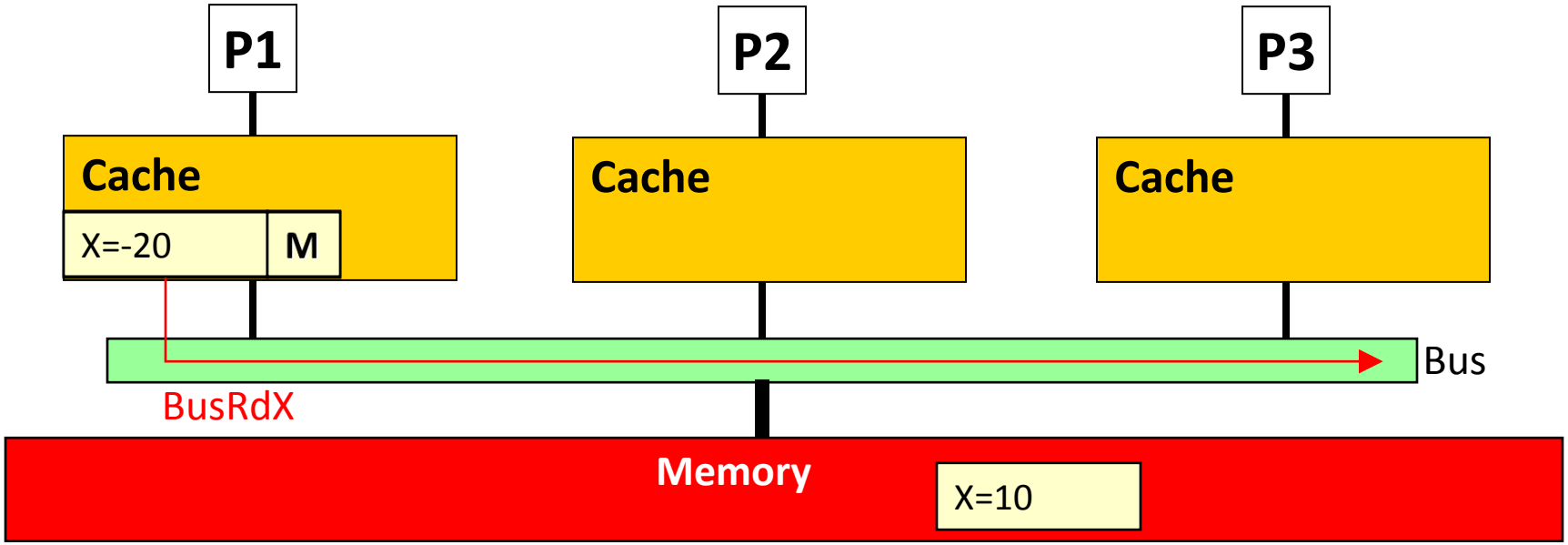
Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P1 write X					

MSI 协议存在的问题



Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P1 write X					

MSI 协议存在的问题



Processor action	State in P1	State in P2	State in P3	Bus transaction	Data supplier
P1 reads X	S	---	---	BusRd	Memory
P1 write X	M	---	---	BusRdX	---

广播BusRdX没必要，因为全局只有1个copy ！

解决方案: MESI

- **Idea:** 增加一个状态(Exclusive)，表明该copy是全局唯一的，并且值是最新的(clean)
  - ***Exclusive* 状态**: 可以直接变为M状态，不需要广播消息

- **如何变为E状态**：全局仅有一个clean的copy时
  - ✓ 如果read一个block的时候其他cache中都没有，那么该block直接为E状态
  - ✓ 系统原本有多个S状态的copy，但是因为cache替换，全局仅剩一个S状态的copy时，该copy也应该变为E状态

MESI Protocol 状态转换

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → **S or E**	Miss → M	---	---
Shared (S)	Hit	→ M	→ S	→ I
Exclusive (E)	Hit	Hit → M	→ S	→ I
Modified (M)	Hit	Hit	→ S	→ I

*注意表格没有体现cache替换引起的状态变化

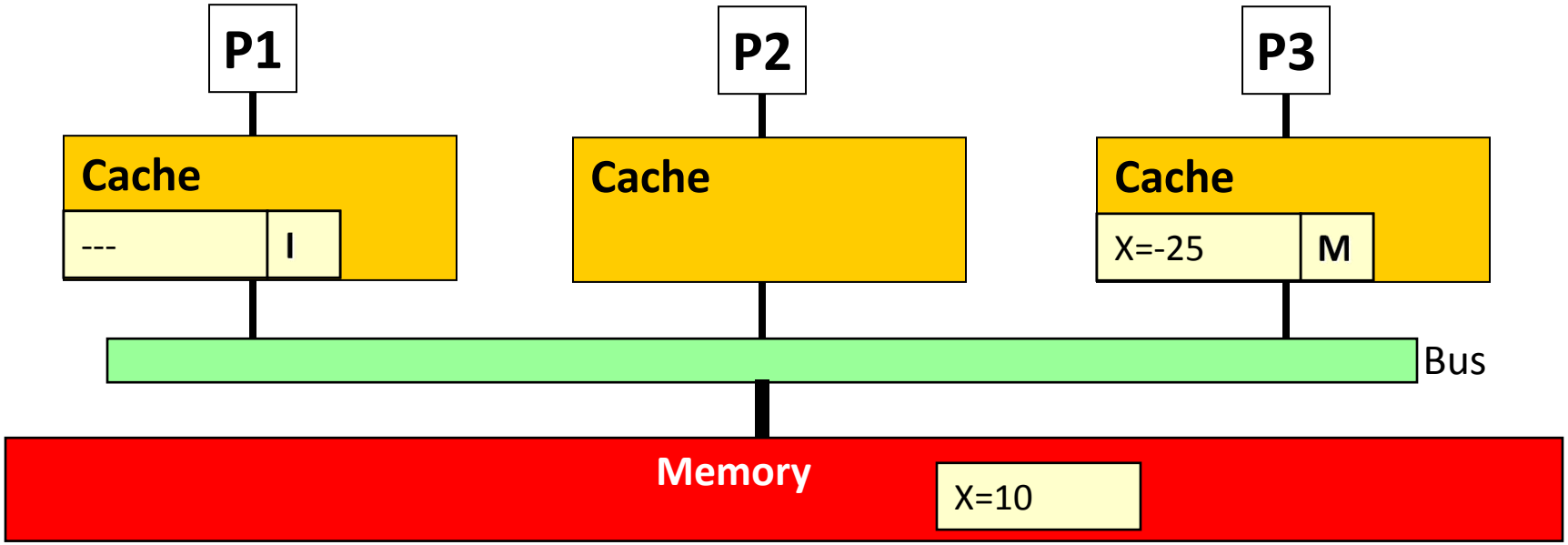
MESI 存在的问题

- **Shared状态要求数据总是最新的(clean)**
  - i.e., 所有shared copy都有最新的数据，包括内存

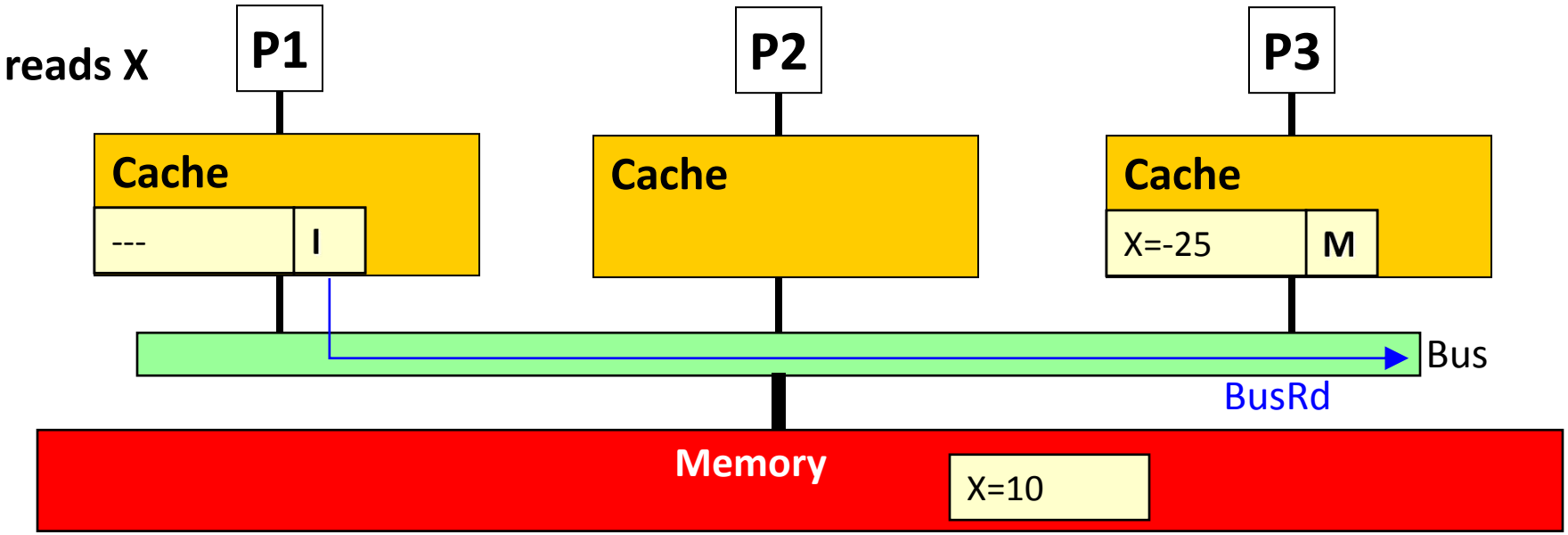
- **Problem:** 一个block处于M状态(表明数据dirty)，如果其他cache读该block，那么需要将data先写回内存，否则大家都变为shared状态时，数据都是dirty的

- **为何这会成为一个问题?**
  - 如果每次从M->S都将数据写回内存，其实很多都是没必要的 → 因为很快别的处理器可能又更新了数据

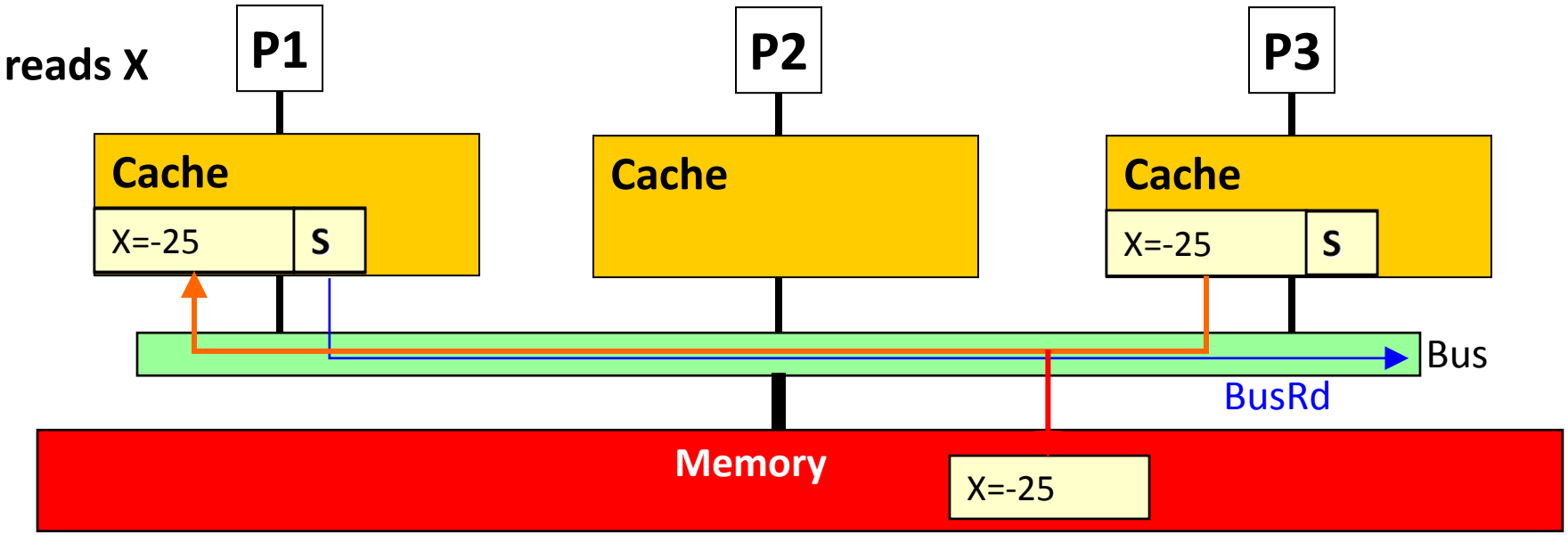
MESI 存在的问题



MESI 存在的问题



MESI 存在的问题



- P3需要将X先写回内存，否则大家都变为shared状态时，数据都是dirty的(MESI要求S状态的数据必须是clean的)
- 如果X经常更新，写内存次数会增加，但其实很多时候更新内存是没必要的(一会儿X又被更新了)

MESI 改进 -> MOESI Protocol

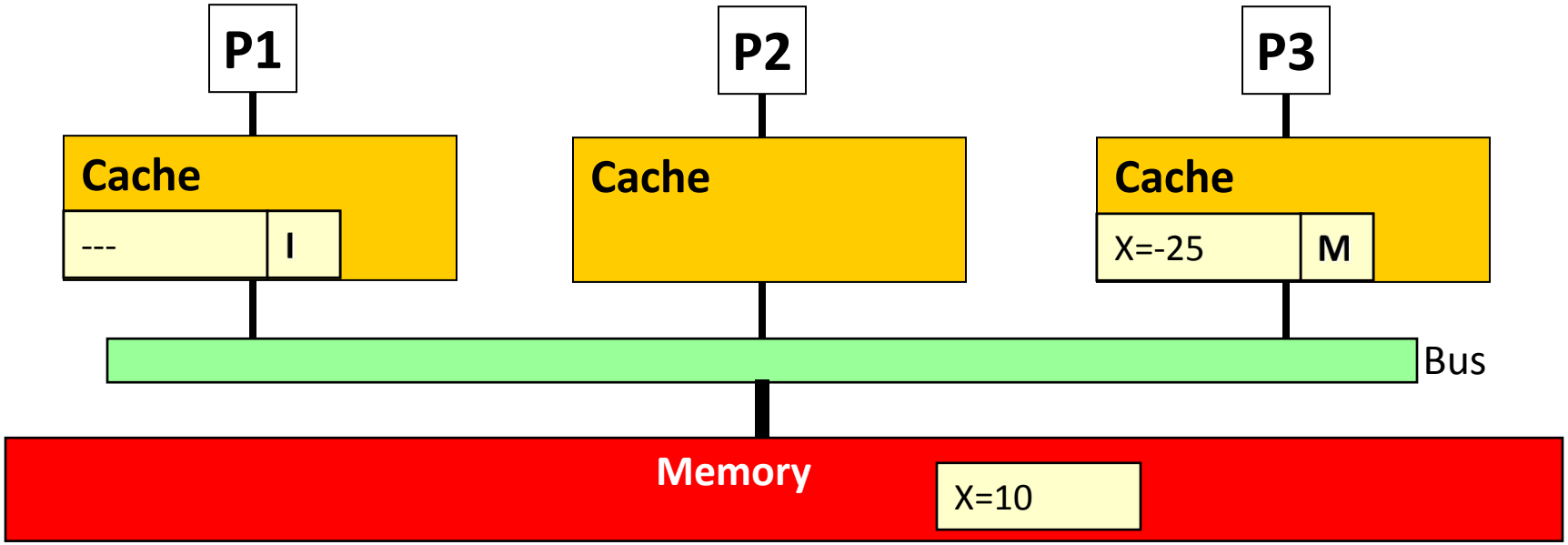
思路：

- **发生M→S转换时，只将最新数据传给请求者，新数据不立即写回内存**
- **包含dirty数据的cache块被换出时再写回内存**
  - ✓ 符合write-back原则

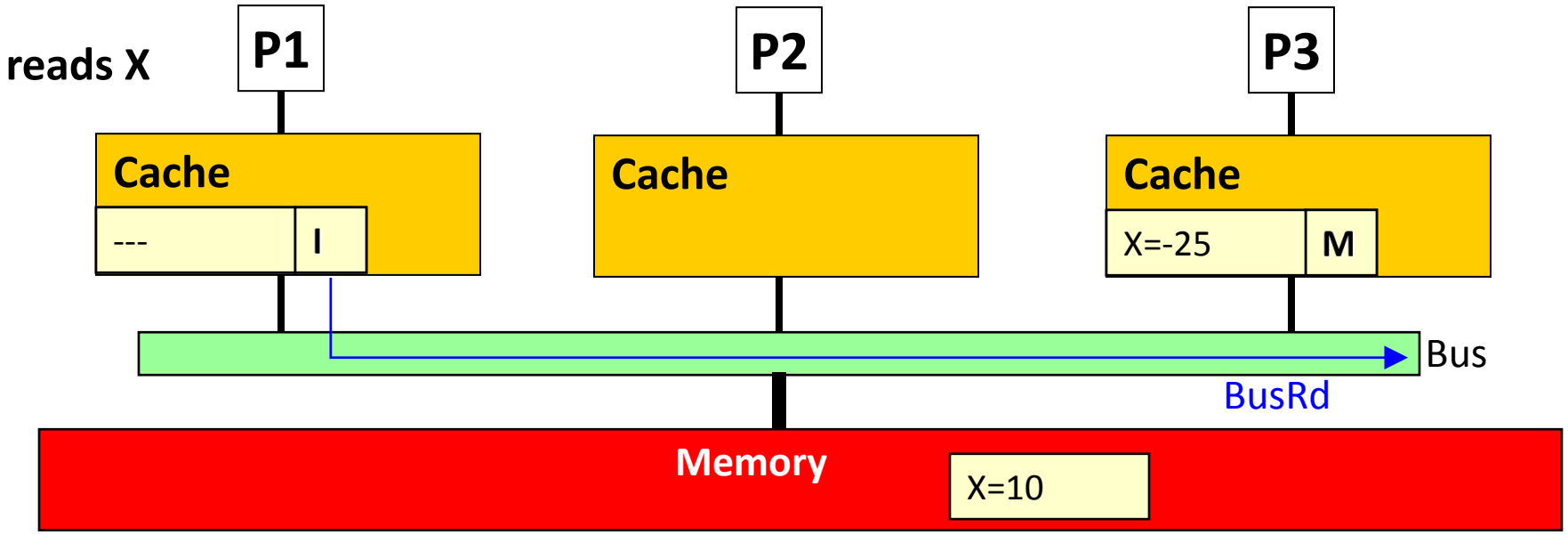
挑战：

- **M->S转换后，出现多个shared copy都包含dirty数据，谁负责写回？**
  - ✓ shared copy的数据是否为dirty无法判断(除非增加标记)
  - ✓ 如果每个shared copy被换出时都写一遍内存->开销大

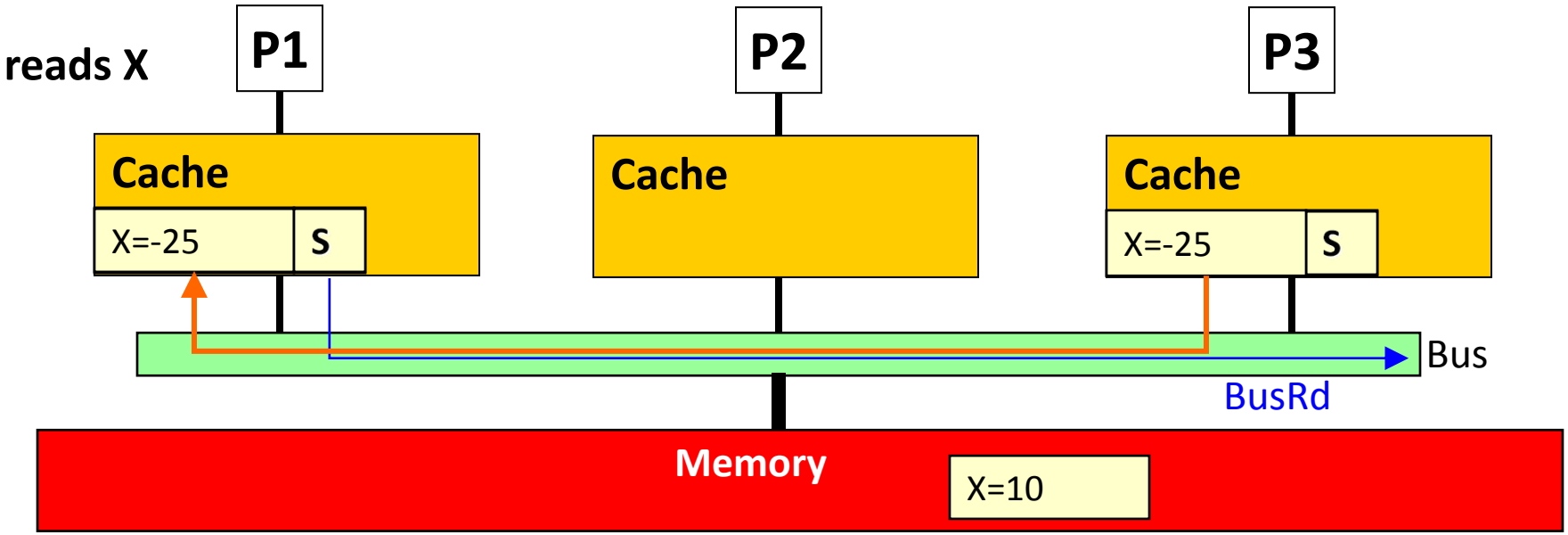
示例



示例



示例



- P3只给P1发送数据，不写回内存
- P1和P3都是shared copy，都包含dirty的数据
- P1和P3谁负责写回？

MESI 改进 -> MOESI Protocol

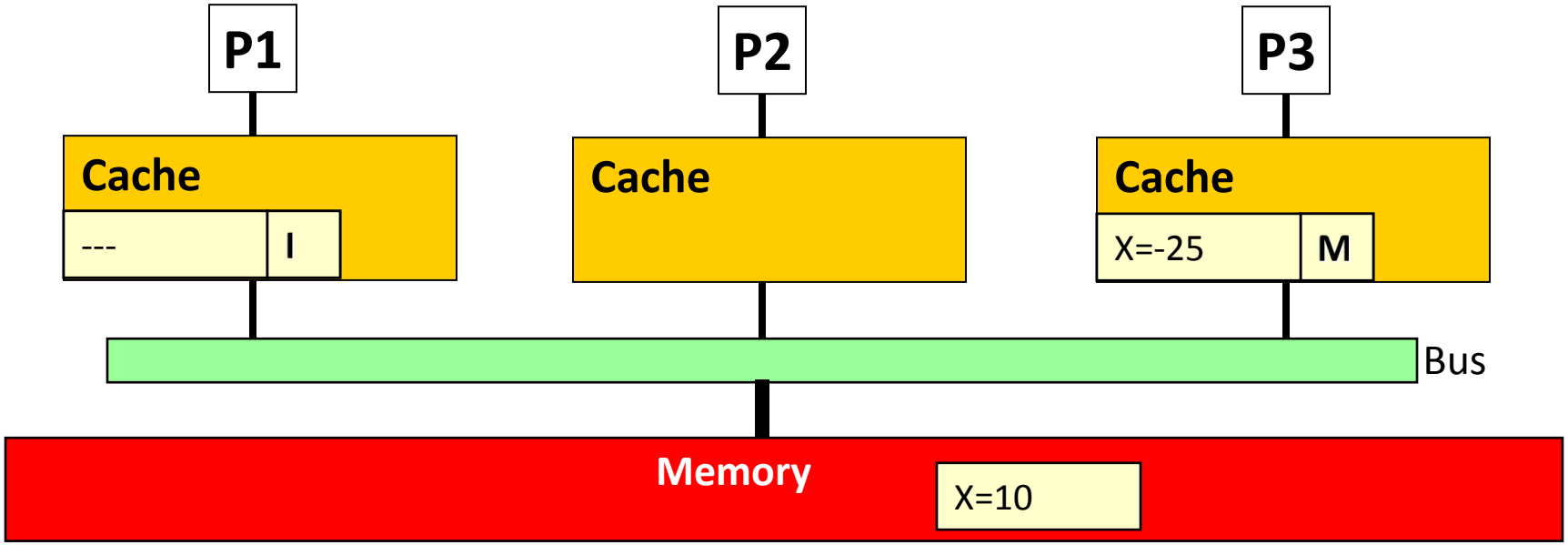
思路：

- **发生M→S转换时，只将最新数据传给请求者，新数据不立即写回内存**
- **包含dirty数据的cache块被换出时再写回内存**
  - ✓ 符合write-back原则

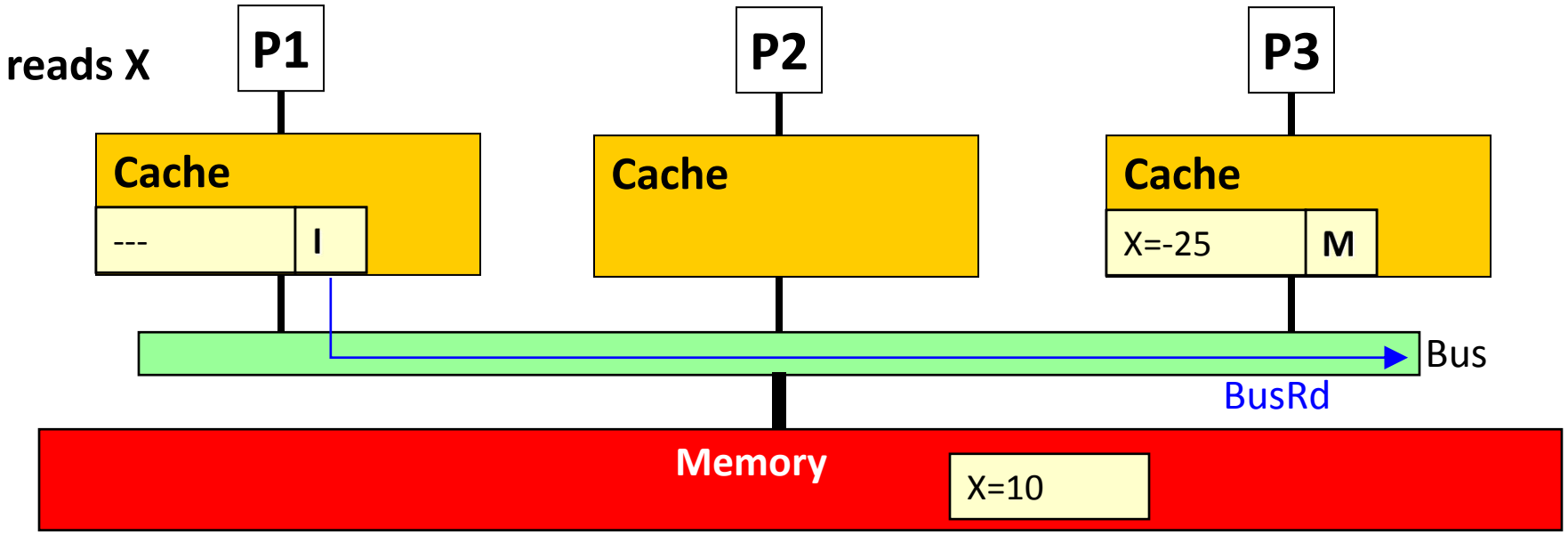
解决方案：MOESI 协议

- **发生M→S转换时，将其中一个shared copy标记为owner (O)状态，它在被换出的时候写回内存**
  - ✓ O状态的cache copy一定是dirty的
  - ✓ 不用每个shared copy都写回内存

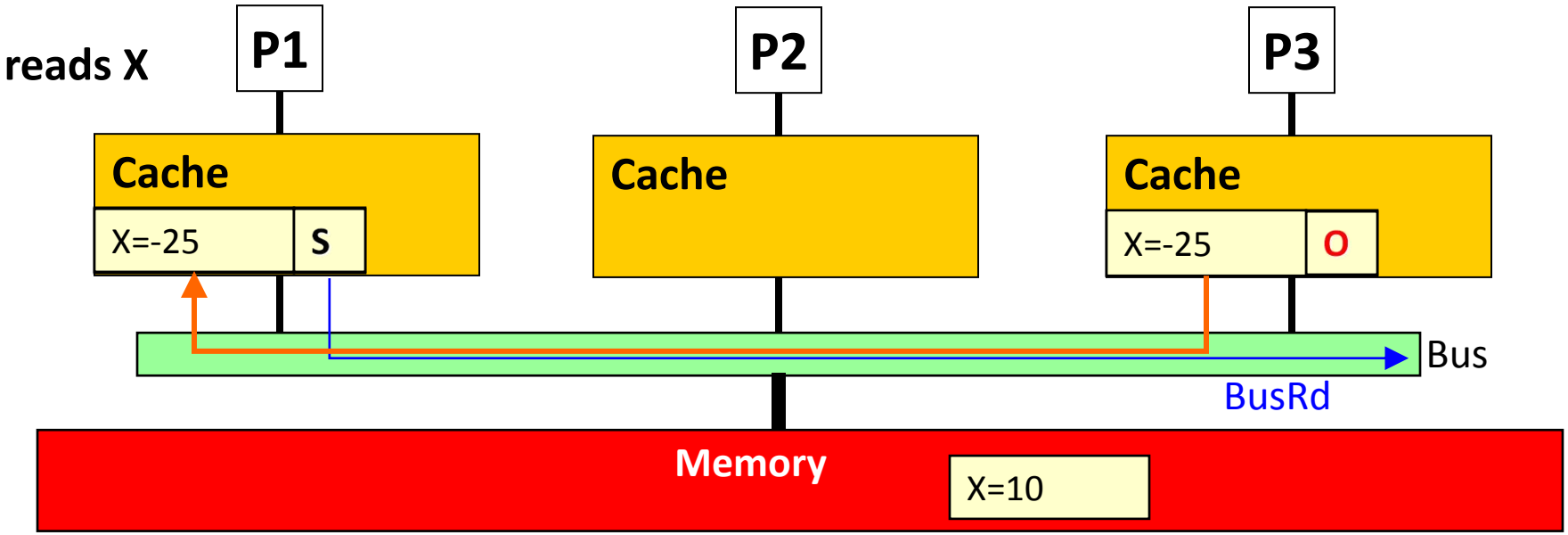
示例



示例

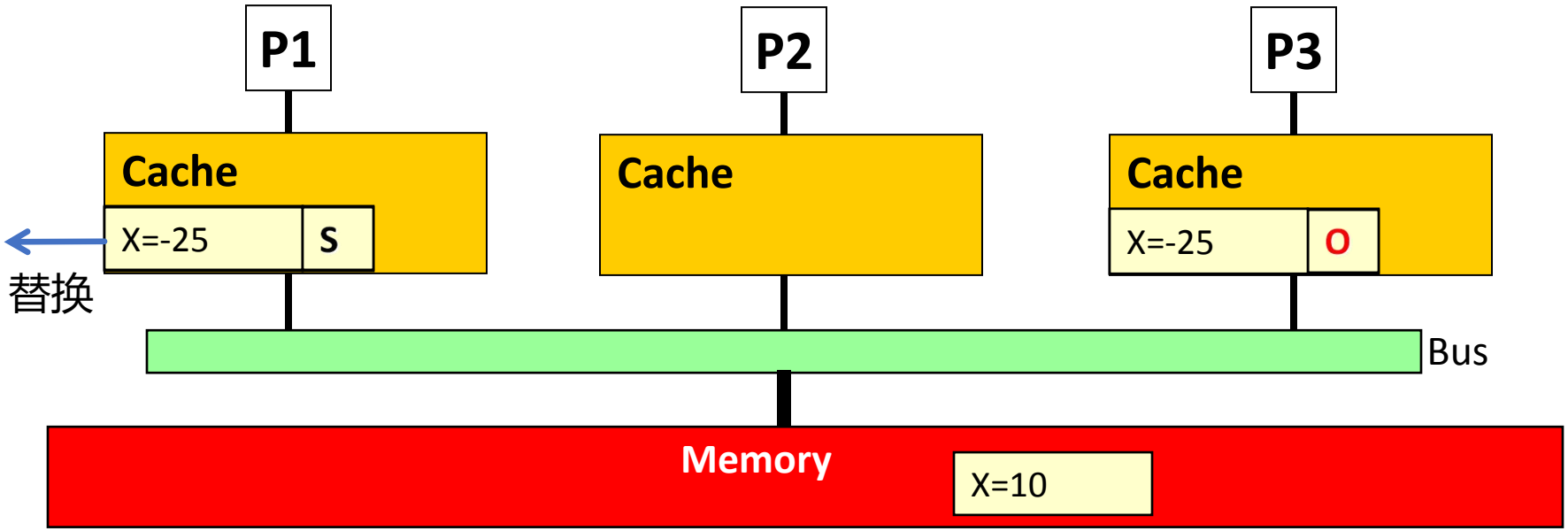


示例



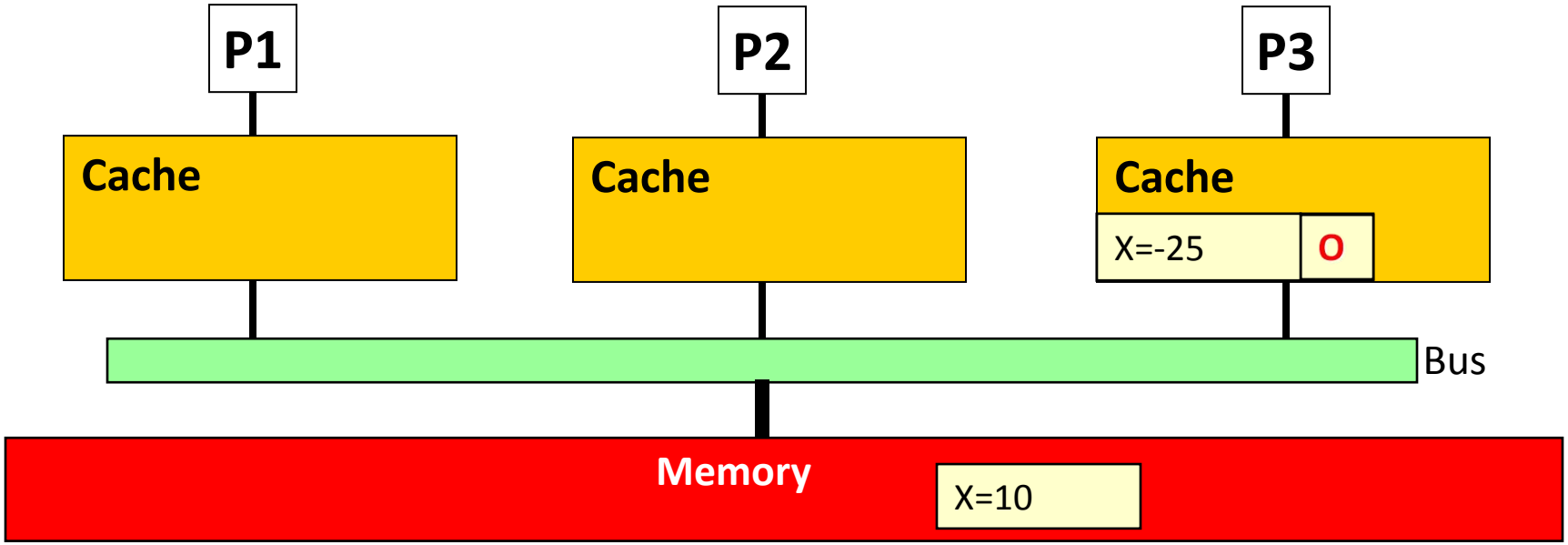
- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

示例



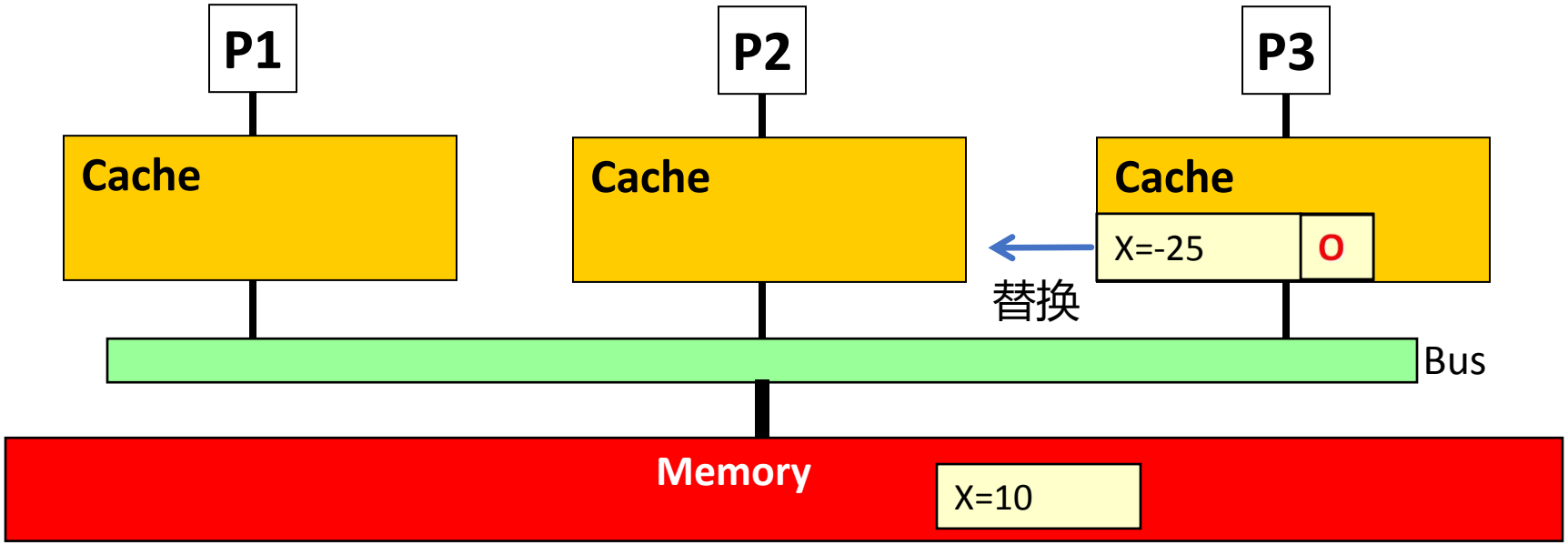
- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

示例



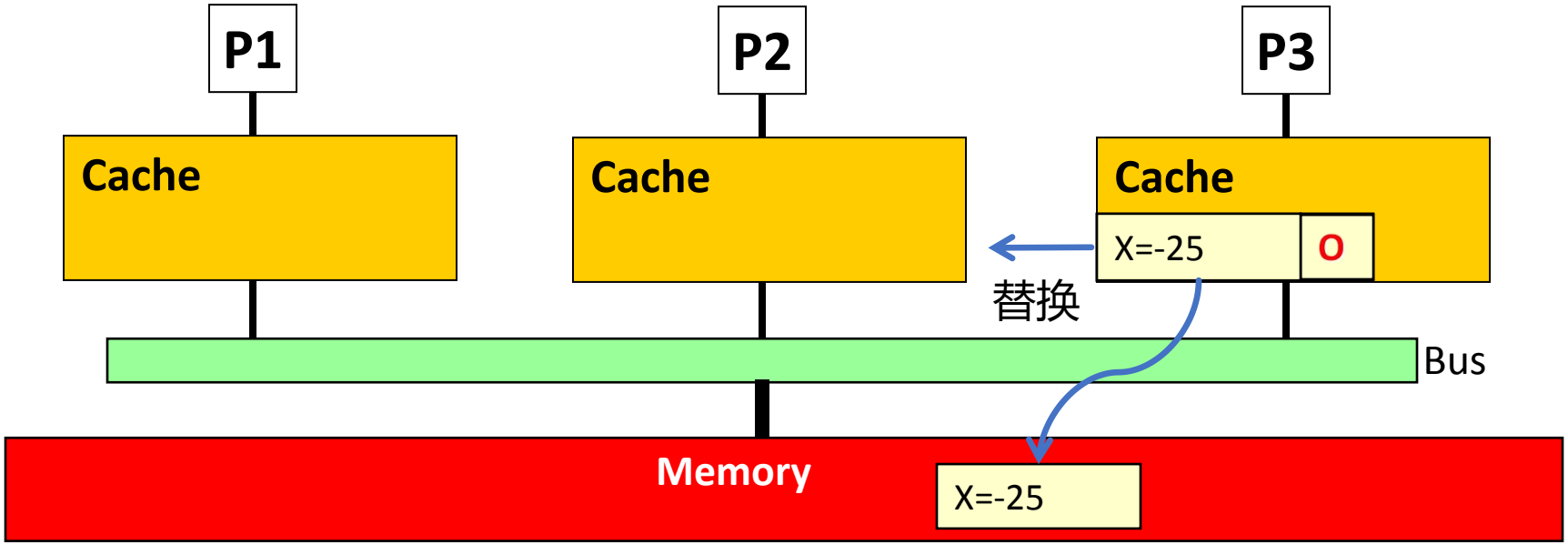
- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

示例



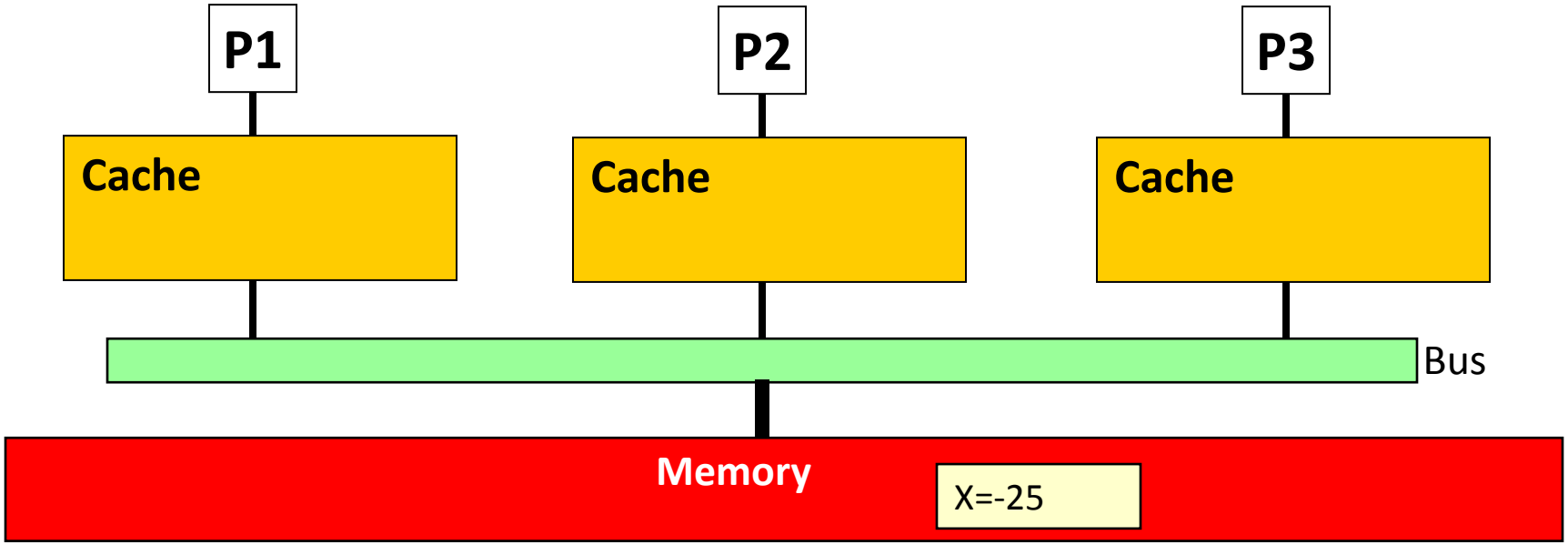
- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

示例



- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

示例



- P3只给P1发送数据，不写回内存
- P1变为shared copy，P3变为O状态
- P1负责写回内存

MOESI Protocol 状态转换表

State	*This Processor*		*Other Processor*	
	Load	Store	Load Miss	Store Miss
Invalid (I)	Miss → S or E	Miss → M	---	---
Shared (S)	Hit	→ M	→ S	→ I
Owned (O)	Hit	→ M	→ O	→ I
Exclusive (E)	Hit	Hit → M	→ S	→ I
Modified (M)	Hit	Hit	→ O	→ I

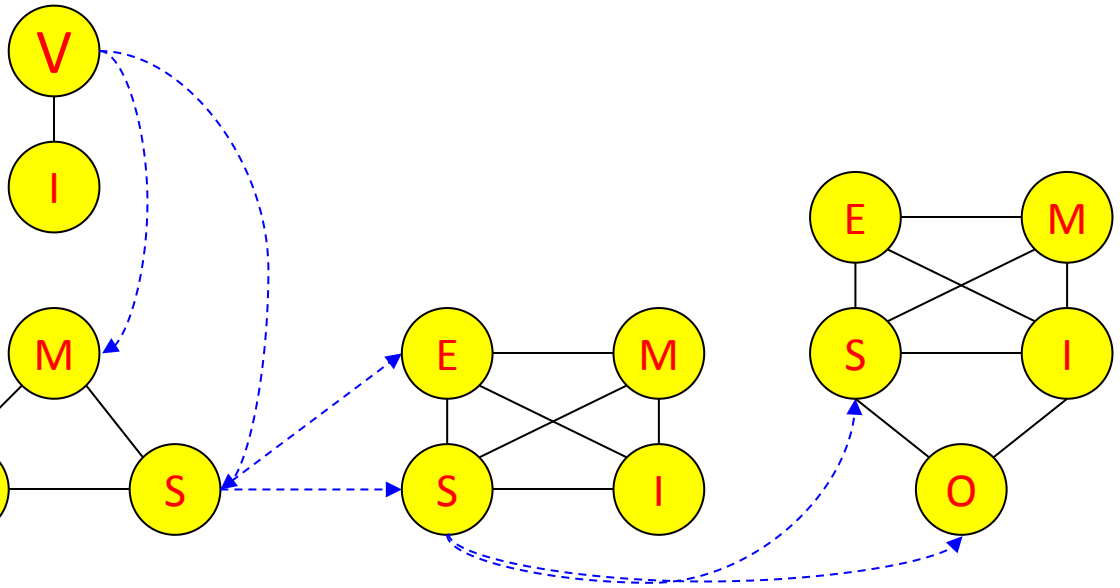
Snoopy Cache Coherence 总结

Correctness **Performance**

Write through

Update

Invalidate



Write back

总结

- **Symmetric multiprocessor (SMP):**
  - 共享单一内存，每个处理器内存访问延迟相同
    → 中心化共享内存, 统一内存访问
  - 通常采用总线互联

- **侦听式协议非常适合总线式SMP**
  - 基于总线实现请求序列化
  - 当某个处理器发生miss, 必须探测每个cache和处理器来观察他们的反应

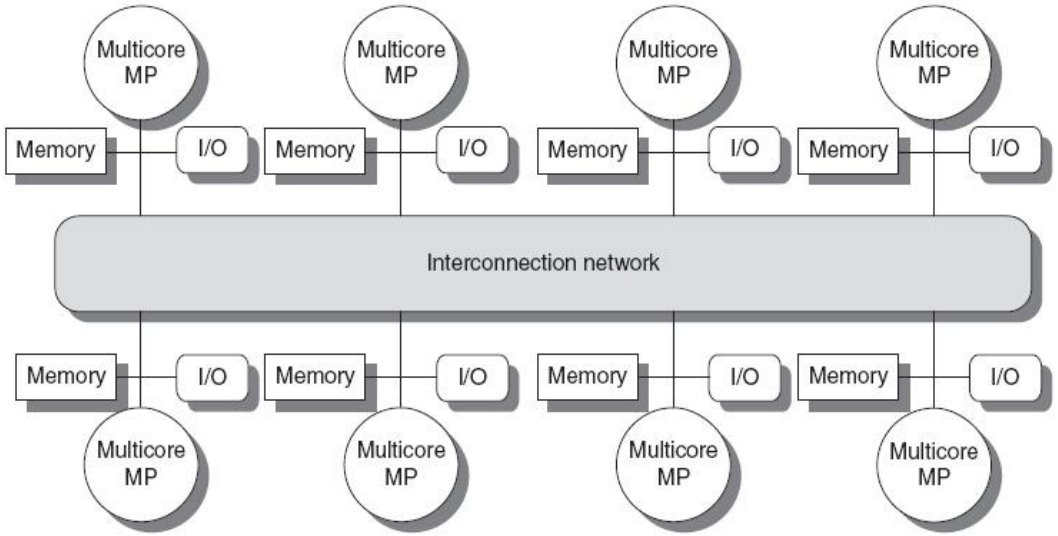
- **可扩展性受限:**
  - 总线带宽

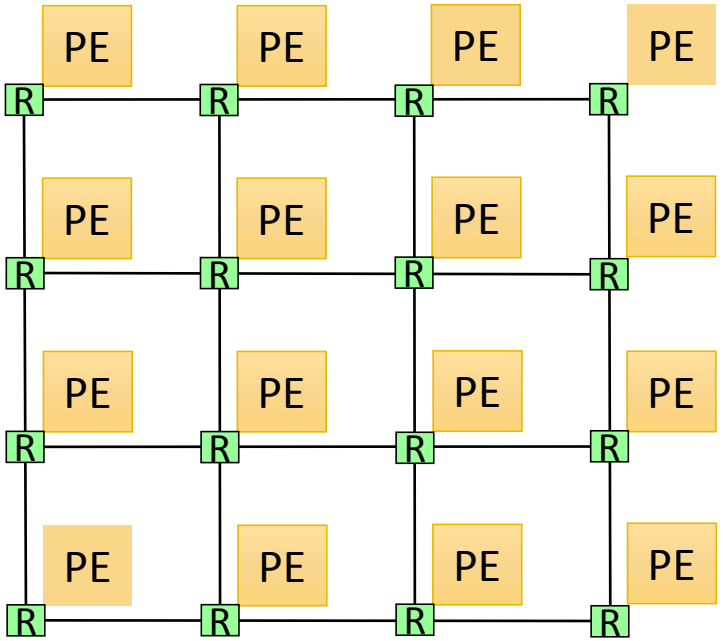
分布式共享内存
(Distributed Shared Memory)

分布式共享内存 (DSM)

Non-uniform memory access (NUMA)

- 内存分布在处理之间, 逻辑上共享
- 所有处理器可以直接访问所有内存
- 可采用可扩展性好的点-对-点互联网络
  → 无中心化节点
  → 可同时进行通信





分布式共享内存 (DSM)

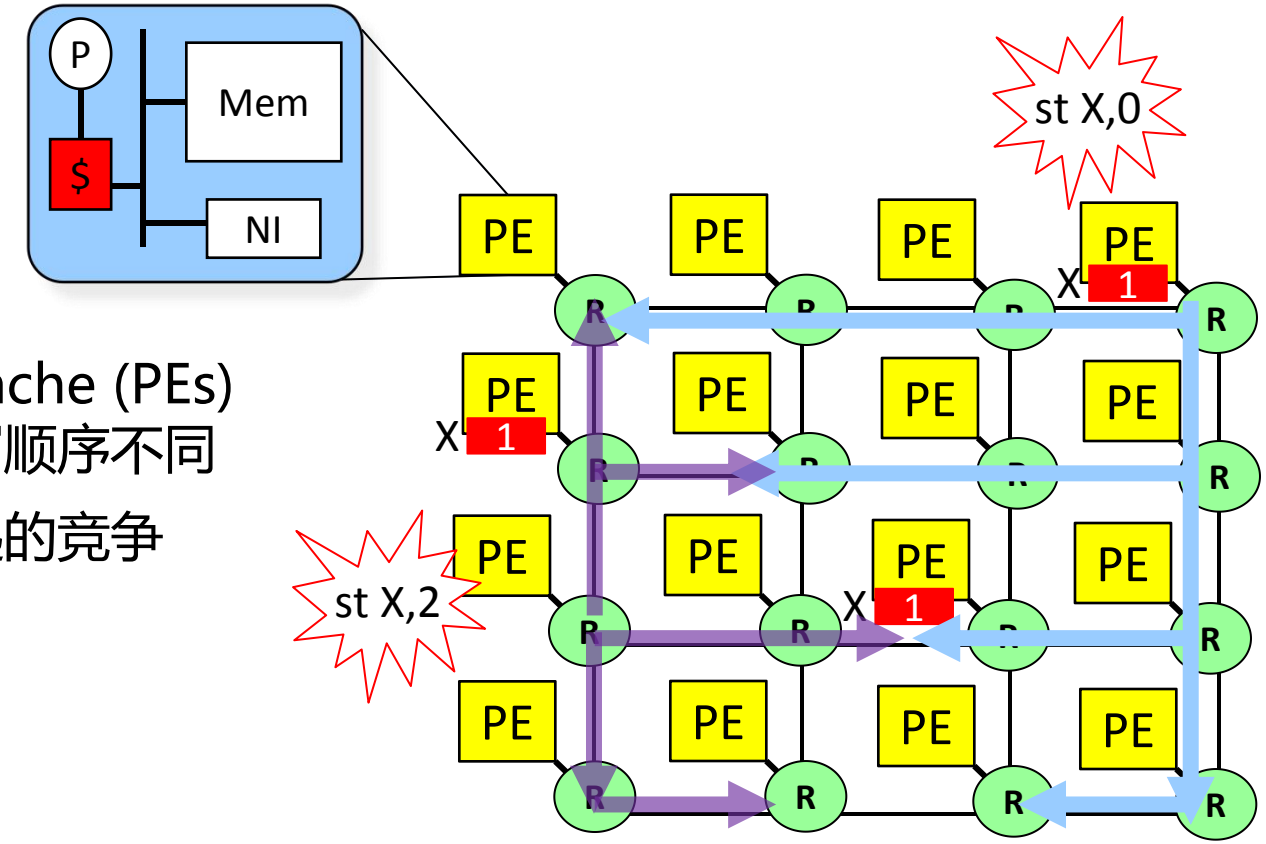
- **Non-uniform memory access (NUMA)**
  - 内存分布在处理之间, 逻辑上共享
  - 所有处理器可以直接访问所有内存
  - 可采用可扩展性好的点-对-点互联网络
    → 无中心化节点
    → 可同时进行通信

侦听式cache一致性协议可用于DSM么?

DSM中的竞争

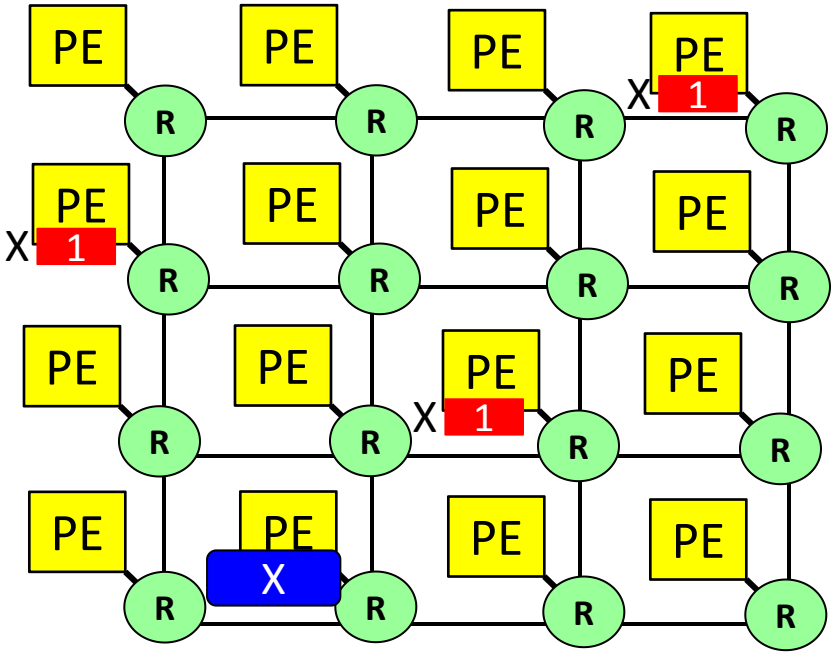
□ 考虑采用mesh网络互联的DSM

- 不同的cache (PEs)看到的写顺序不同
- 网络引起的竞争



可扩展的 Cache Coherence

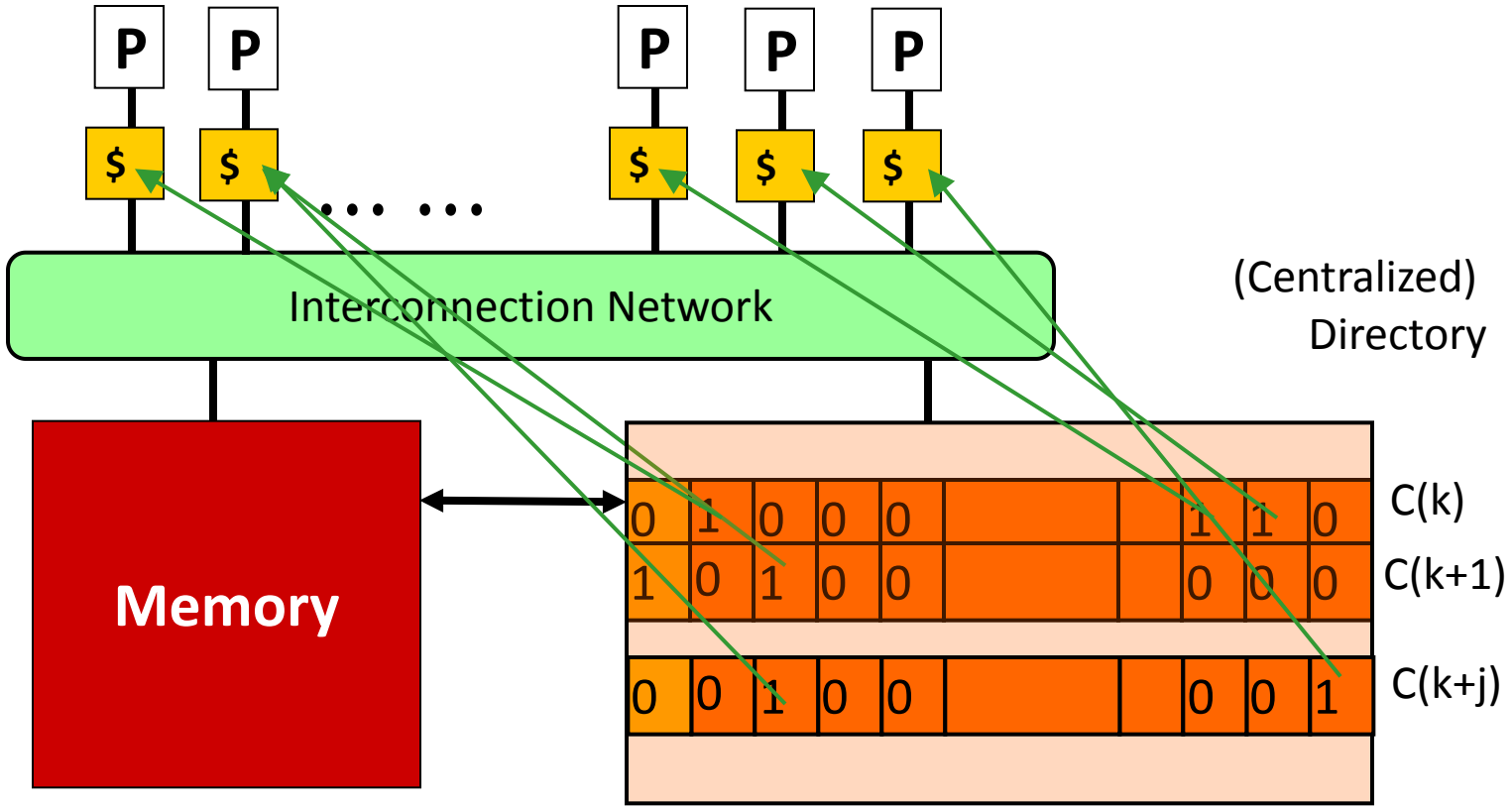
- 需要一种机制将**写操作序列化**(排序)
- **思路:** 不依赖互联网络解决序列化，寻找一个单一的协调者
  → *directory*



Directory-based Cache Coherence

- **Directory:**
  - 单一序列化节点，每个内存block有一个entry
    - entry记录该block的所有cached copies
  - 处理器通过directory请求blocks
  - Directory负责协调invalidation等操作，并仅与有copy的处理器进行通信
  - 所有通信以点对点方式完成

Directory-based Cache Coherence

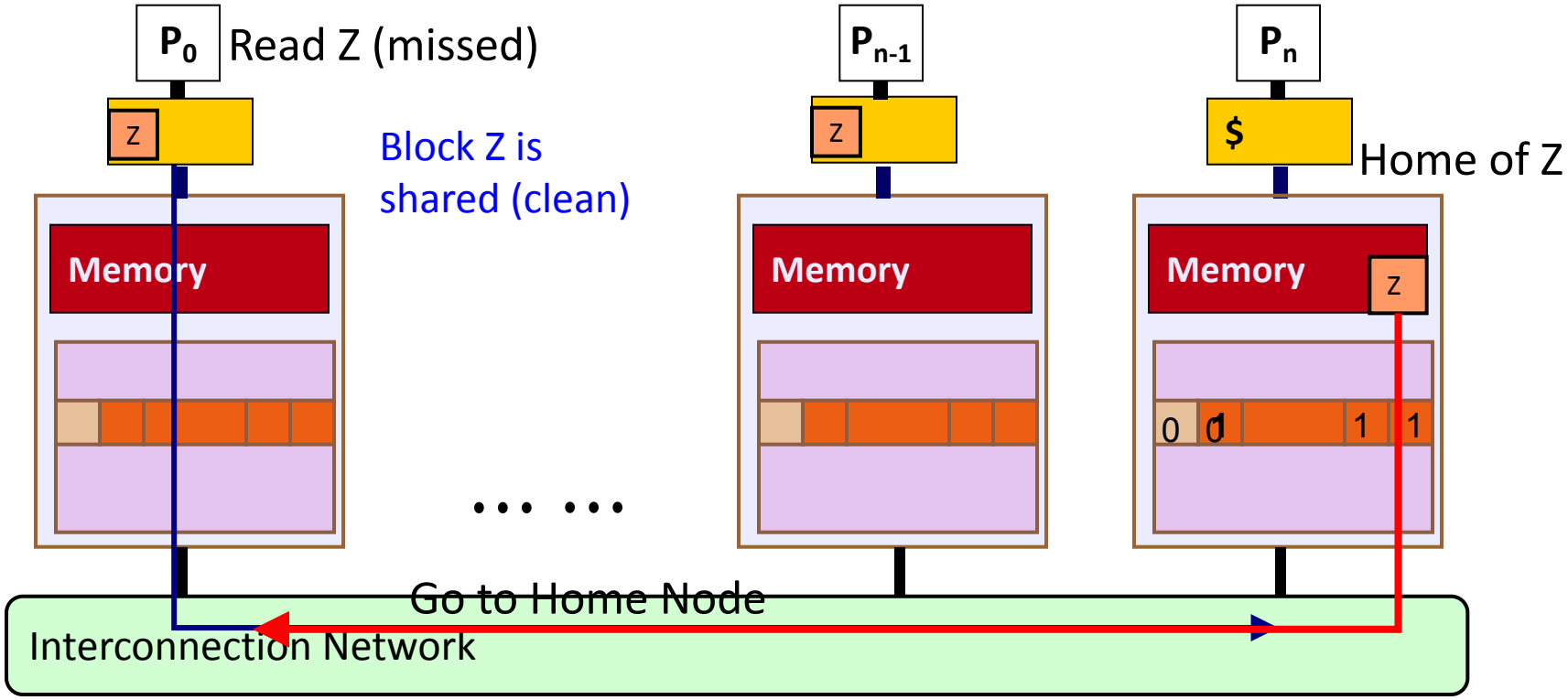


1 *modified bit (每个block有一个)*

1 *presence bit (每个block, 每个processor有1个)*

Directory Coherence Protocol: Read Miss

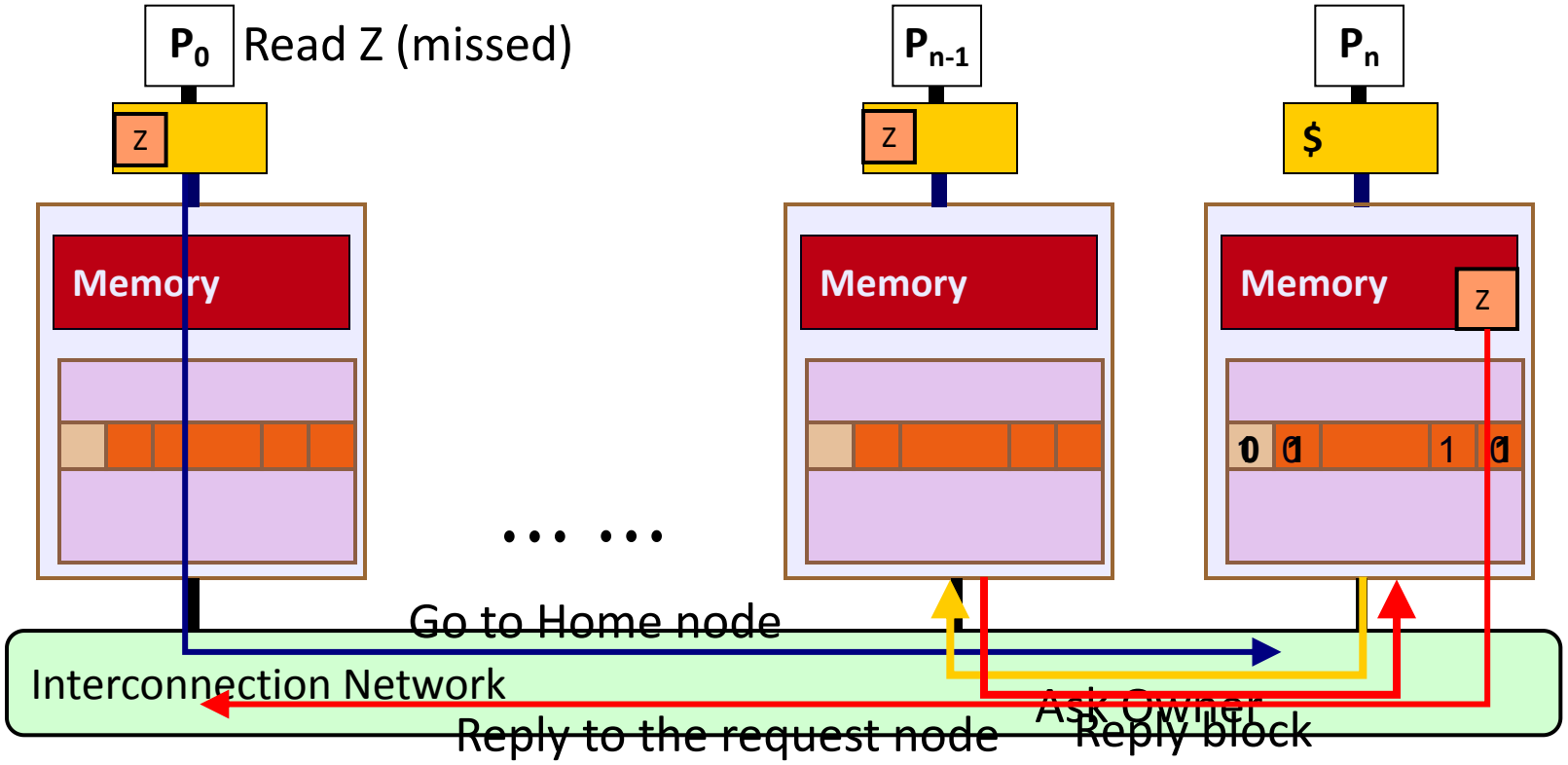
- 每个block有一个“home”节点, 存放其directory entry
  - Home节点可以根据block的地址计算出来(block就在该节点的memory中)



Directory Coherence Protocol: Read Miss

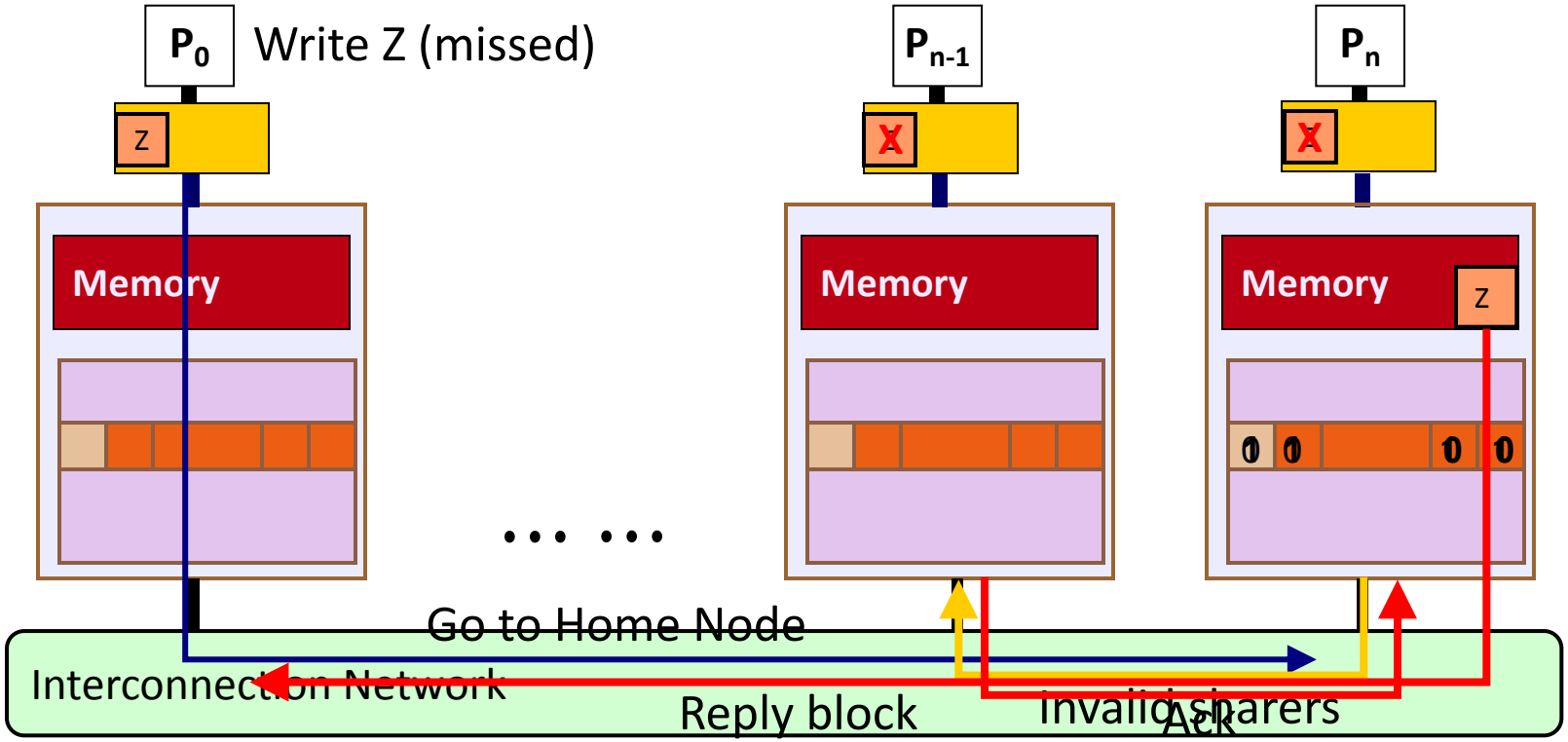
- Block Z is dirty (“Modified” in P_{n-1})

Block Z 变为 clean, 由3个节点共享 (M→S)



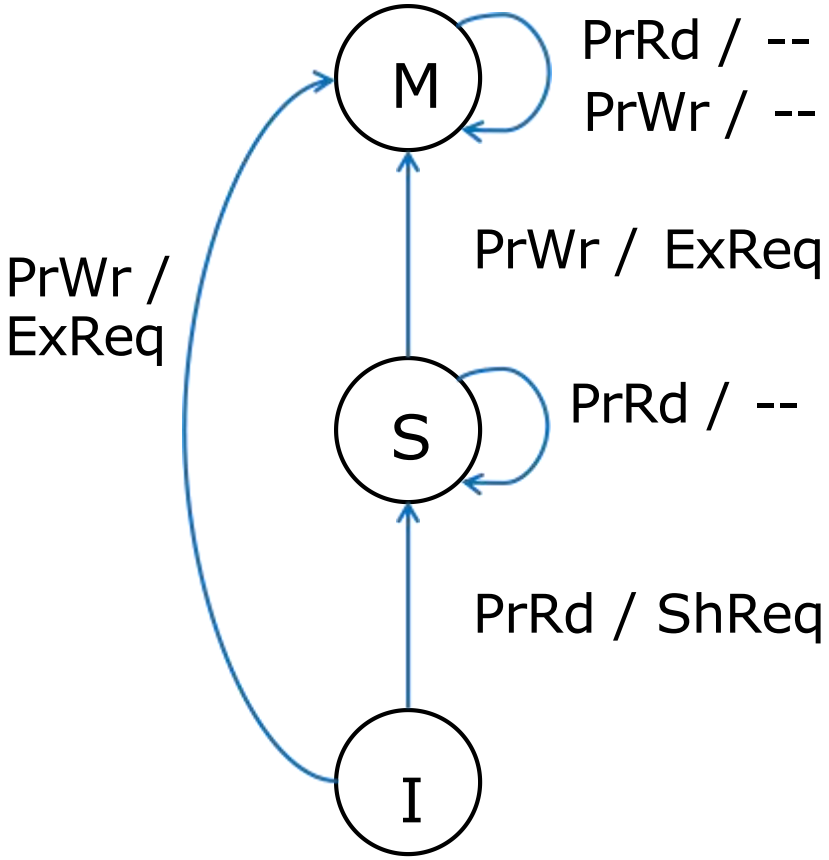
Directory Coherence Protocol: Write Miss

P_0 现在可以写入 block Z (I→M)



MSI Protocol: Caches 状态变化

处理器访问引发的状态转换:



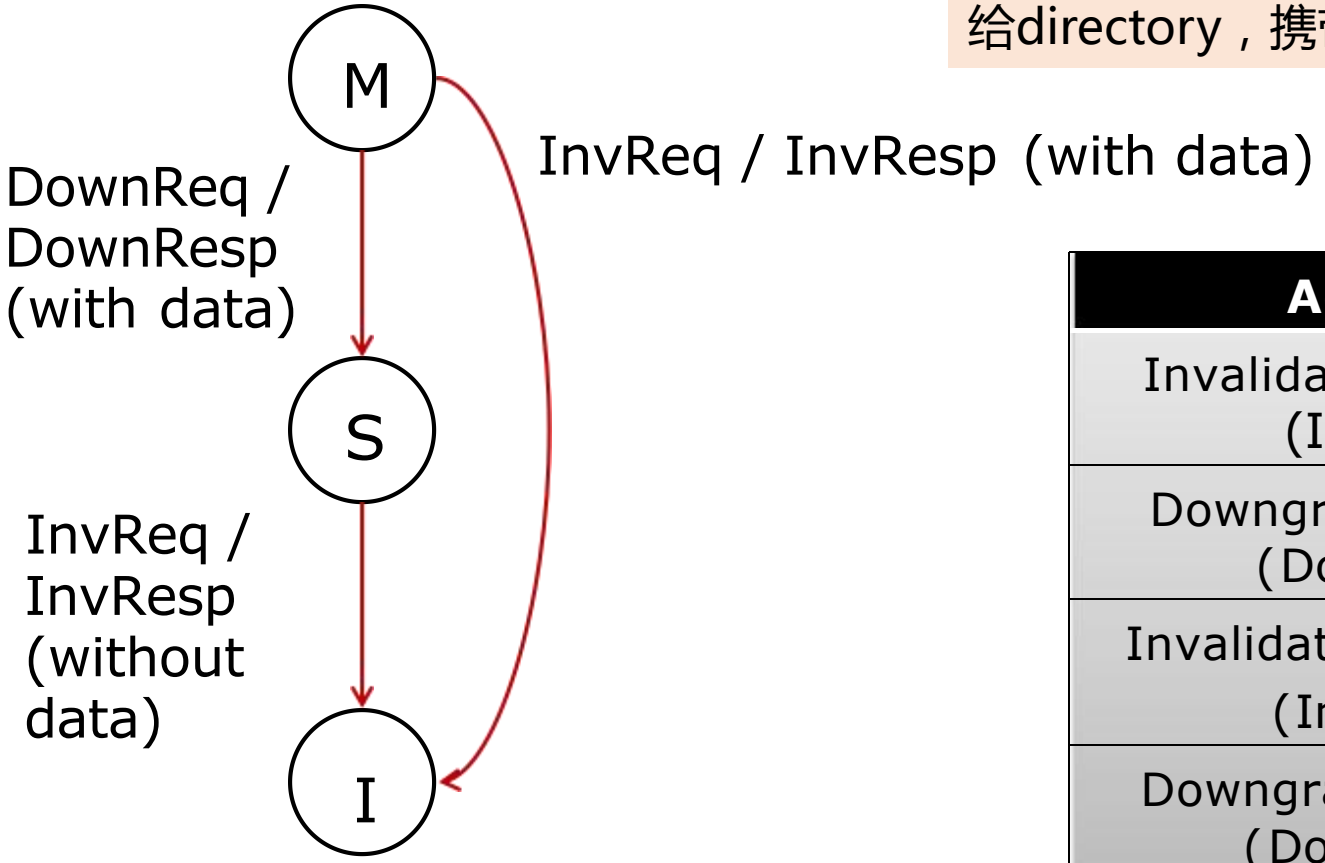
- cache处于S状态，处理器读，保持S状态；处理器写，申请变为M状态(发出ExReq到directory)
- cache处于I状态，处理器读，申请变为S状态(发出ShReq)；处理器写，申请变为M状态(发出ExReq)
- cache处于M状态，处理器读/写，保持M状态

Actions
Processor Read (PrRd)
Processor Write (PrWr)
Shared Request (ShReq)
Exclusive Request (ExReq)

MSI Protocol: Caches 状态变化

处理器收到来自directory的请求：例如M->I，表示directory发送了invreq请求，处于M状态的处理器收到后，会变为I状态，同时发送回复消息给directory，携带着数据

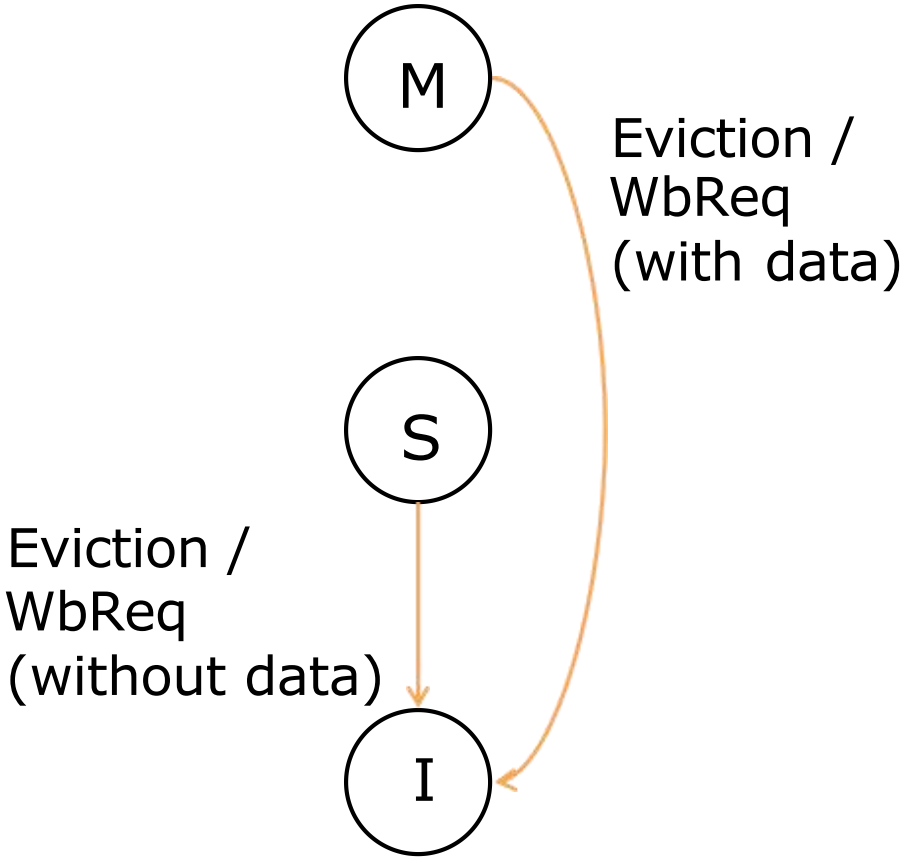
Directory请求引发的状态转换:



Actions
Invalidation Request (InvReq)
Downgrade Request (DownReq)
Invalidation Response (InvResp)
Downgrade Response (DownResp)

MSI Protocol: Caches 状态变化

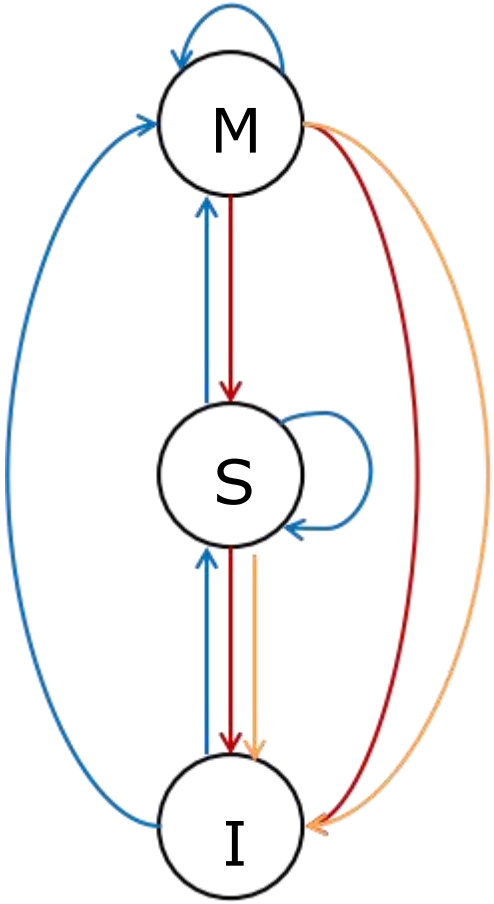
cache替换引发的状态变化:



- 处于M状态的block被替换，要写回内存，同时变为Invalid状态
- 处于S状态的block被替换，也变为invalid，但不用写回，因为shared状态一定是clean的

Actions
Writeback Request (WbReq)

MSI Protocol: Caches 状态变化



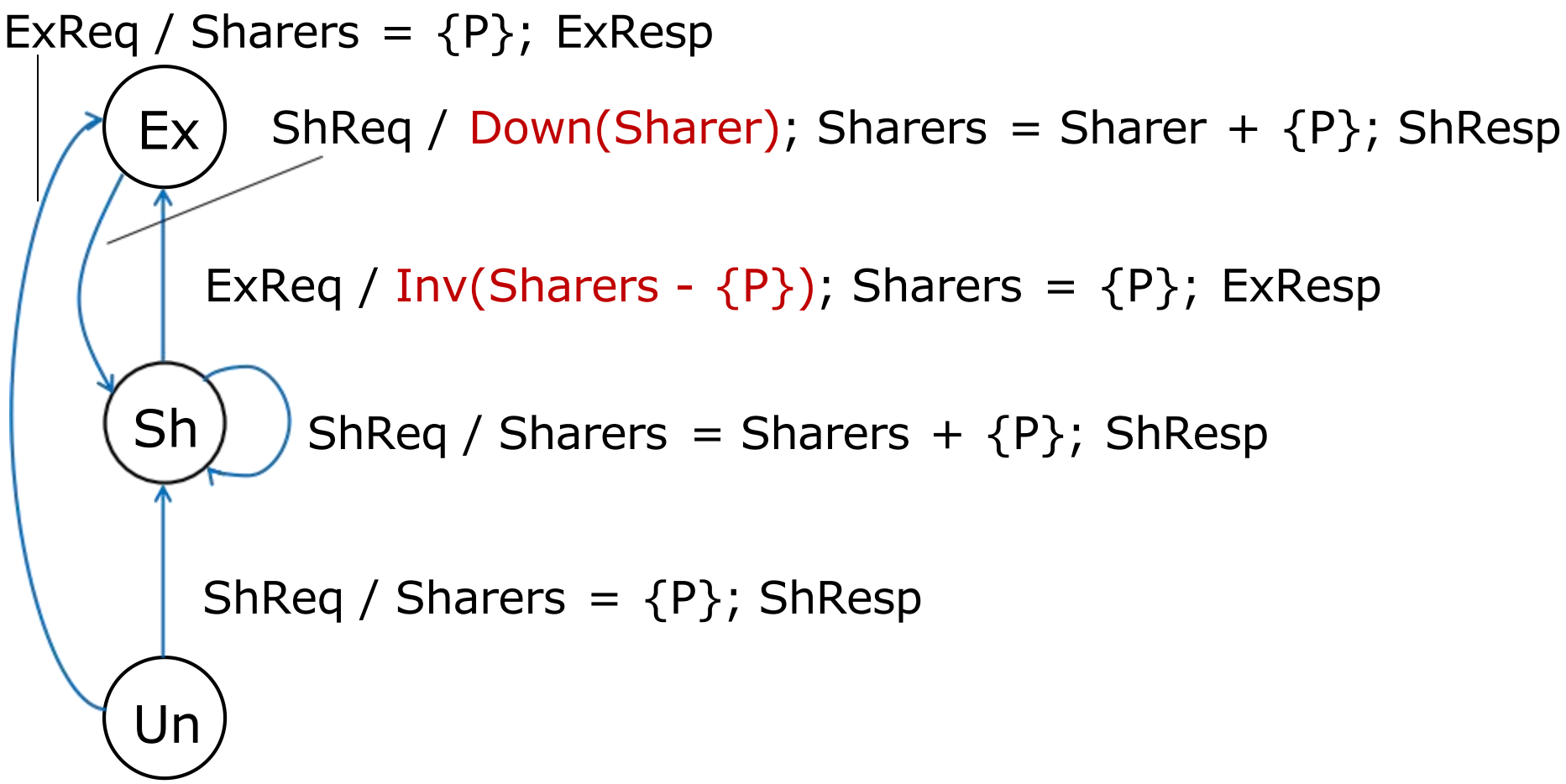
→ 处理器访问引发的状态转换

→ directory请求引发的状态转换

→ cache替换引发的状态转换

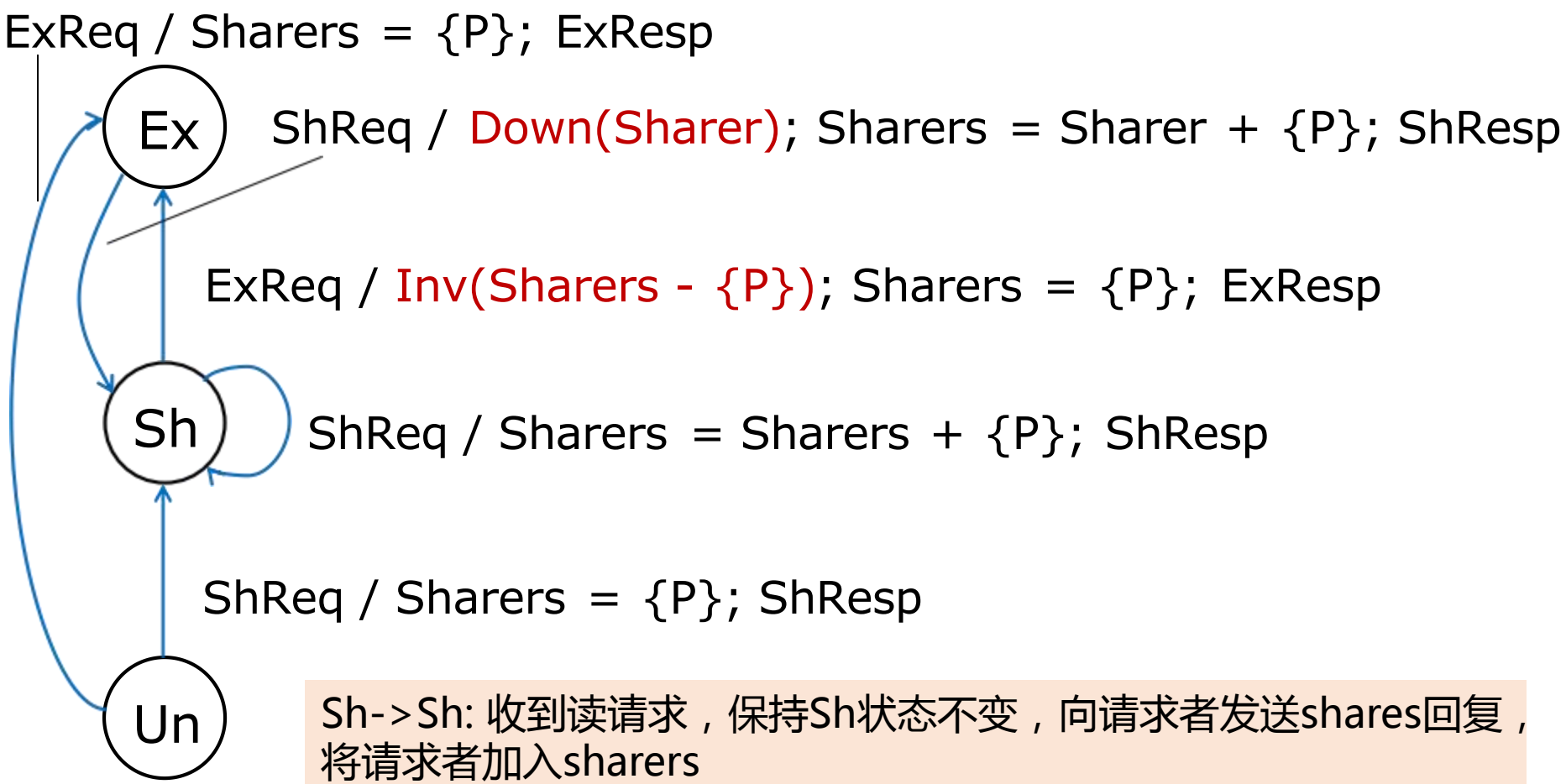
MSI Protocol: Directory 状态变化

数据请求引发的状态变化:



MSI Protocol: Directory 状态变化

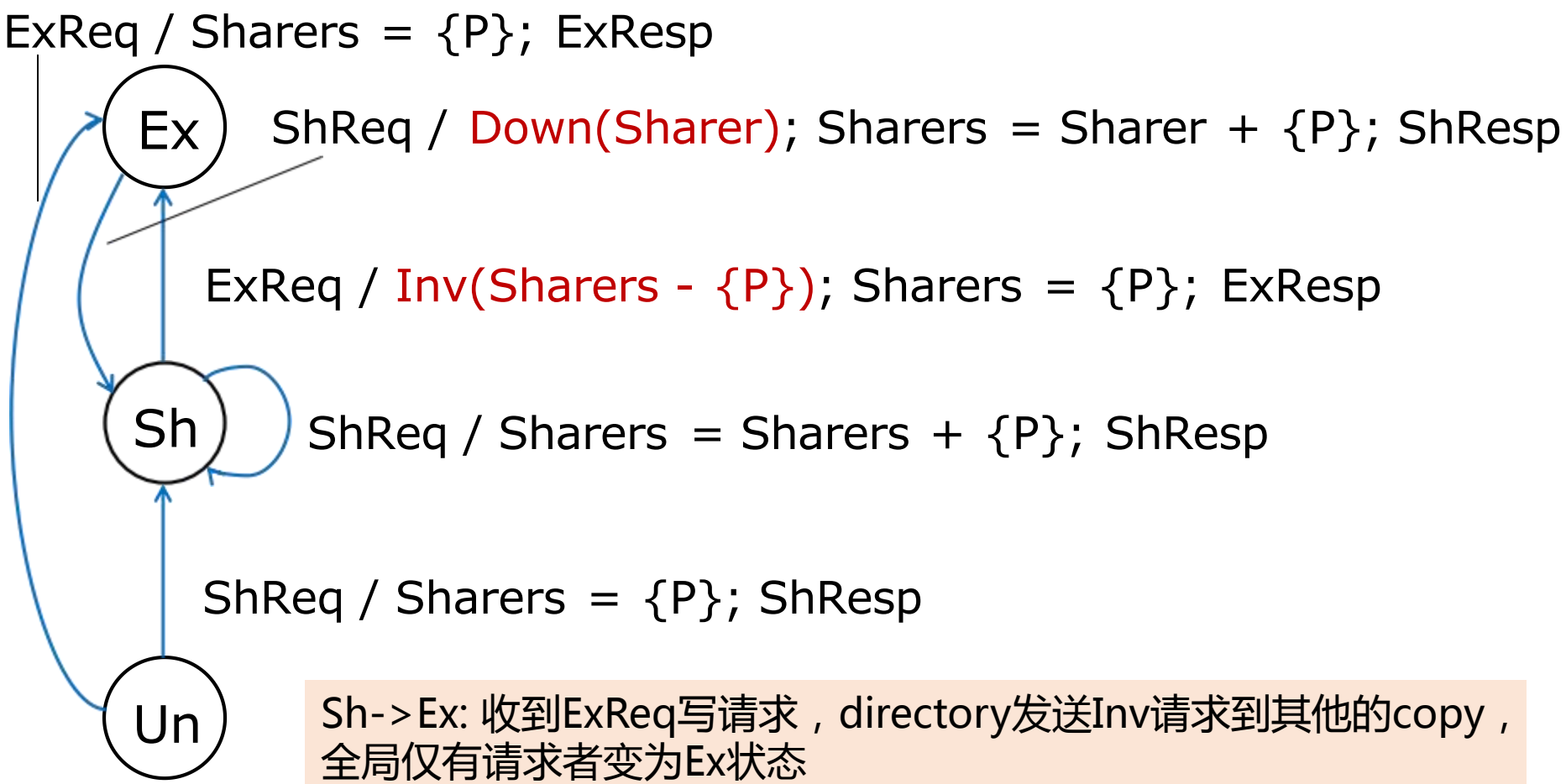
数据请求引发的状态变化:



Sh->Sh: 收到读请求，保持Sh状态不变，向请求者发送shares回复，将请求者加入sharers

MSI Protocol: Directory 状态变化

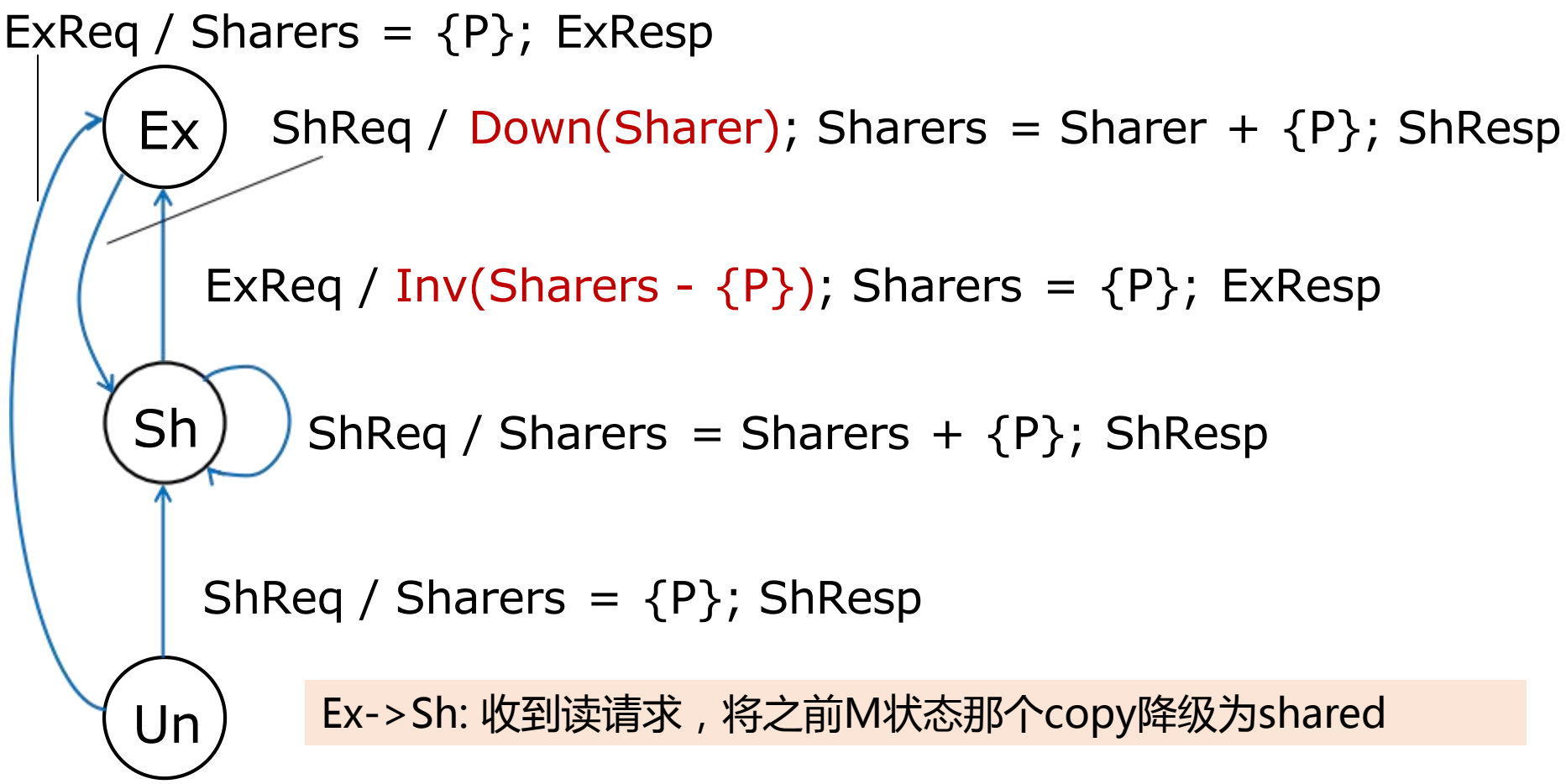
数据请求引发的状态变化:



Sh->Ex: 收到ExReq写请求，directory发送Inv请求到其他的copy，全局仅有请求者变为Ex状态

MSI Protocol: Directory 状态变化

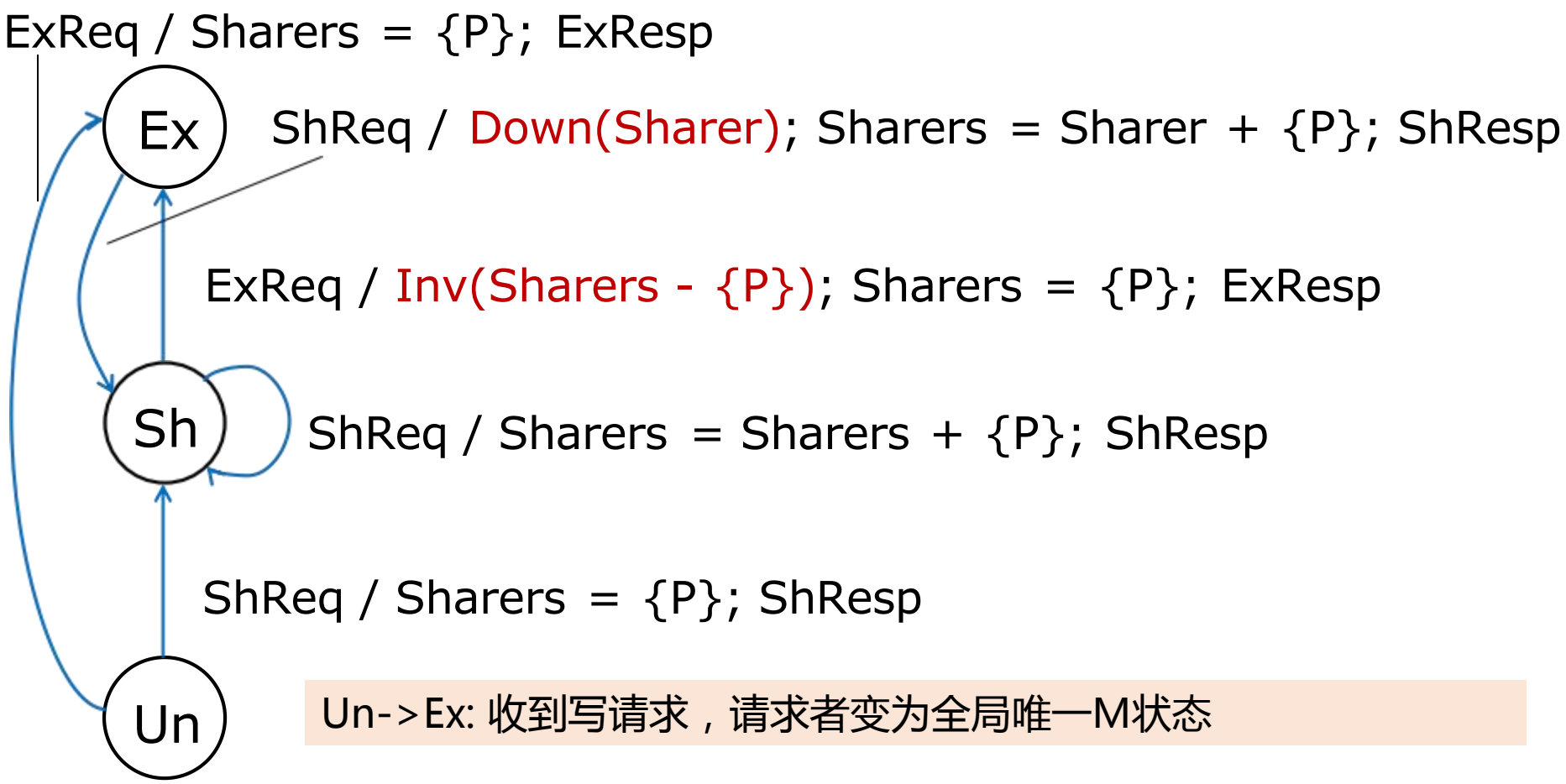
数据请求引发的状态变化:



Ex->Sh: 收到读请求，将之前M状态那个copy降级为shared

MSI Protocol: Directory 状态变化

数据请求引发的状态变化:



Un->Ex: 收到写请求，请求者变为全局唯一M状态

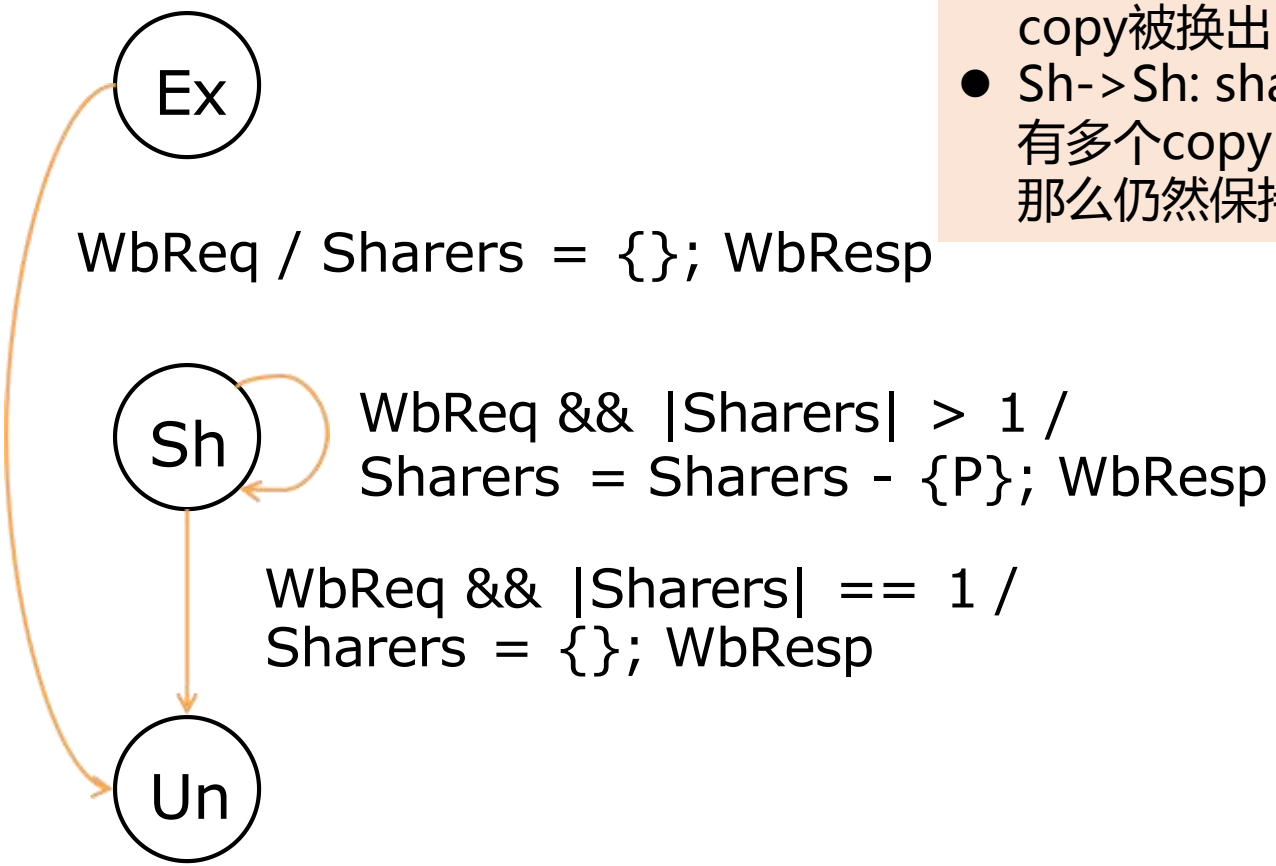
MSI Protocol: Directory 状态变化

数据请求引发的状态变化:

Un->Sh: 收到读请求，变为shared状态

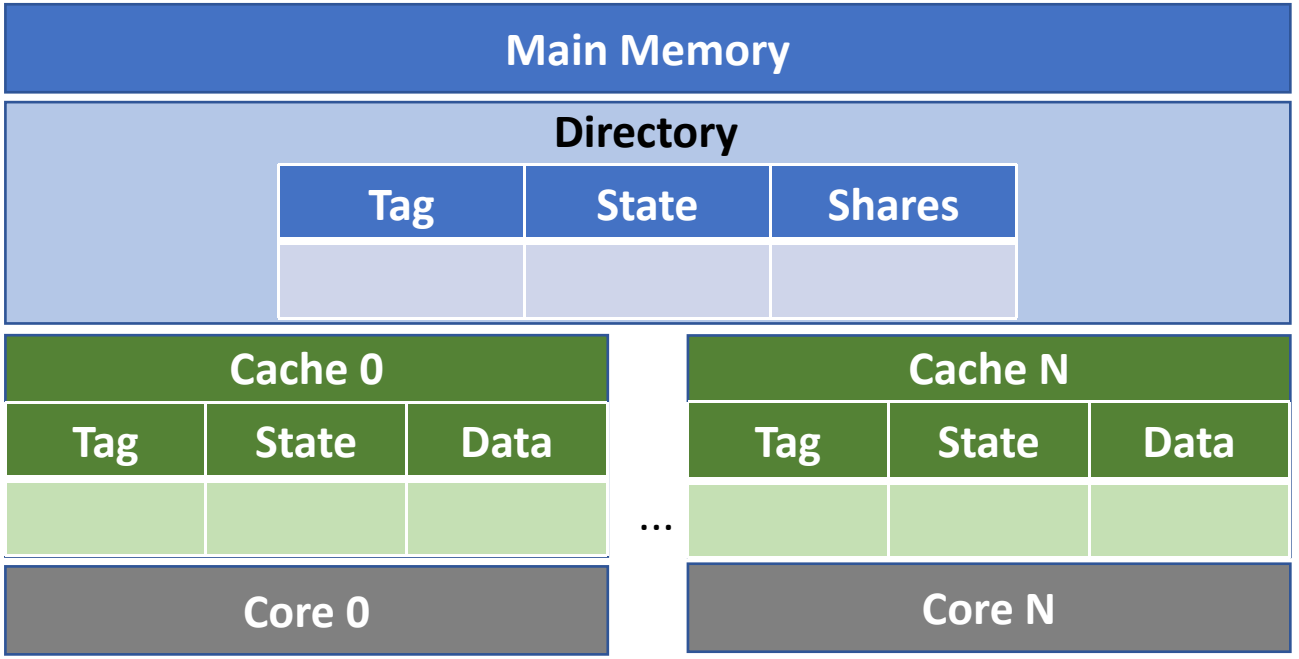
MSI Protocol: Directory 状态变化

writeback引发的状态变化:



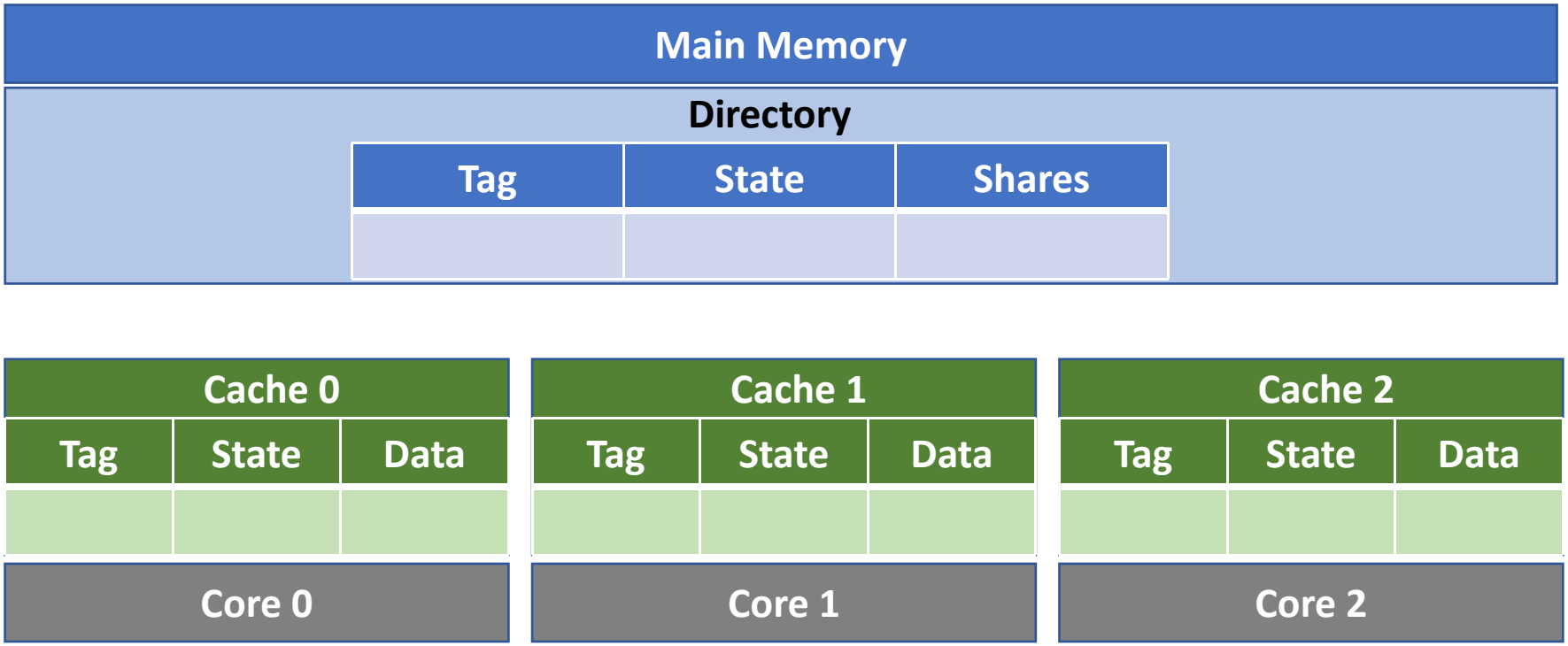
- Ex->Un: M状态的数据写回，意味着M状态的block被换出，全局没有copy了，所以变为un状态
- Sh->Un: shared状态数据写回，如果全局仅有1个copy，意味着唯一的copy被换出，所以也变为un状态
- Sh->Sh: shared状态写回，如果全局有多个copy，仅有1个copy被换出，那么仍然保持shared状态

基于Directory的MSI协议 - 示例

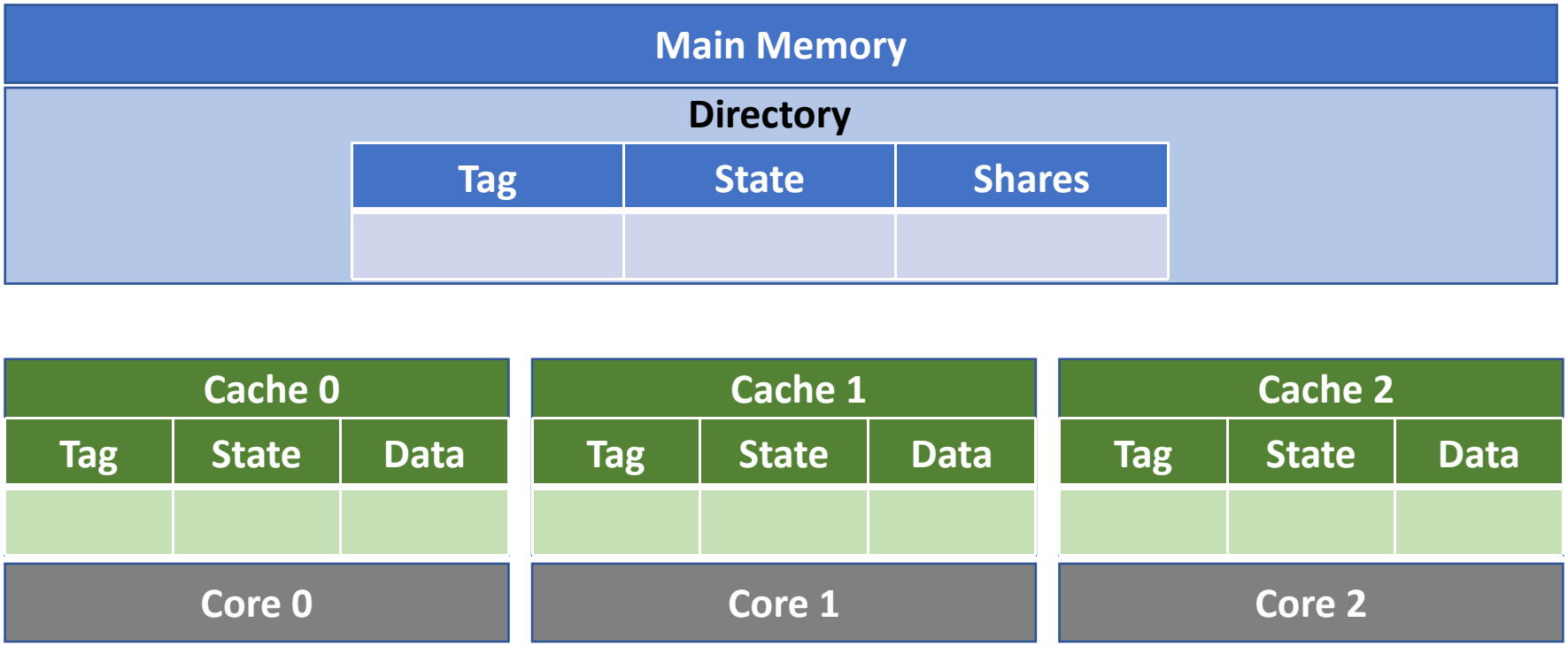


- **Cache 状态: Modified (M) / Shared (S) / Invalid (I)**
- **Directory 状态:**
  - **Uncached (Un)**: 没有任何copy
  - **Shared (Sh)**: 有一个或多个shared copy
  - **Exclusive (Ex)**: 有一个copy处于Modified状态

基于Directory的MSI协议 - 示例

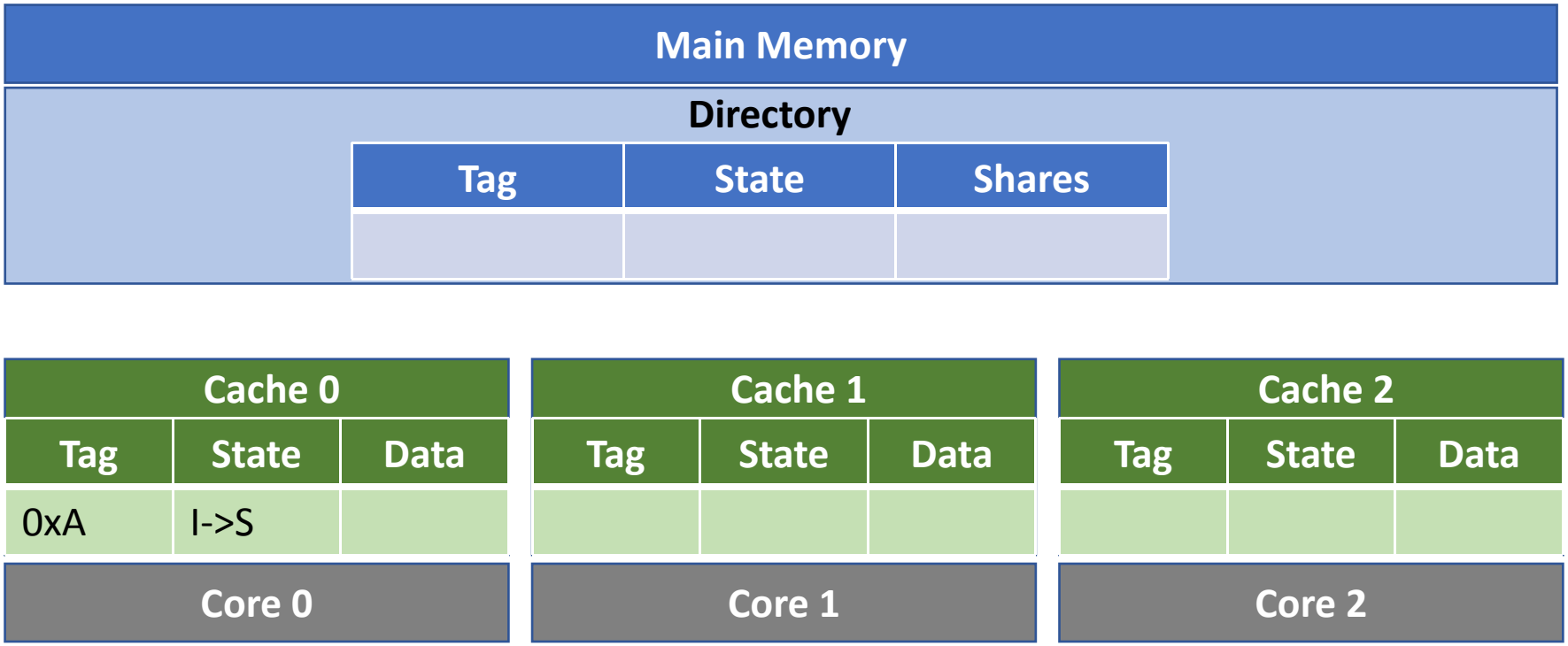


基于Directory的MSI协议 - 示例



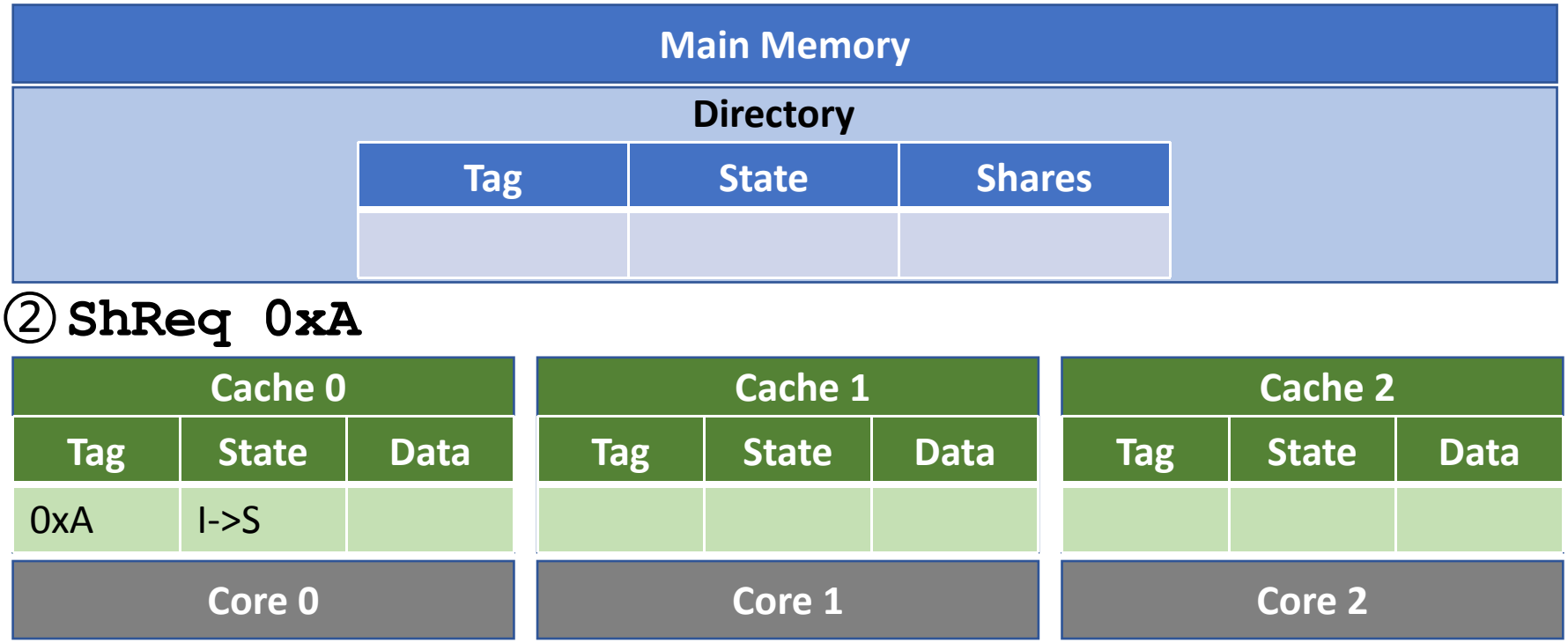
① `LD 0xA`

基于Directory的MSI协议 - 示例



① **LD 0xA**

基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例

Main Memory

Directory

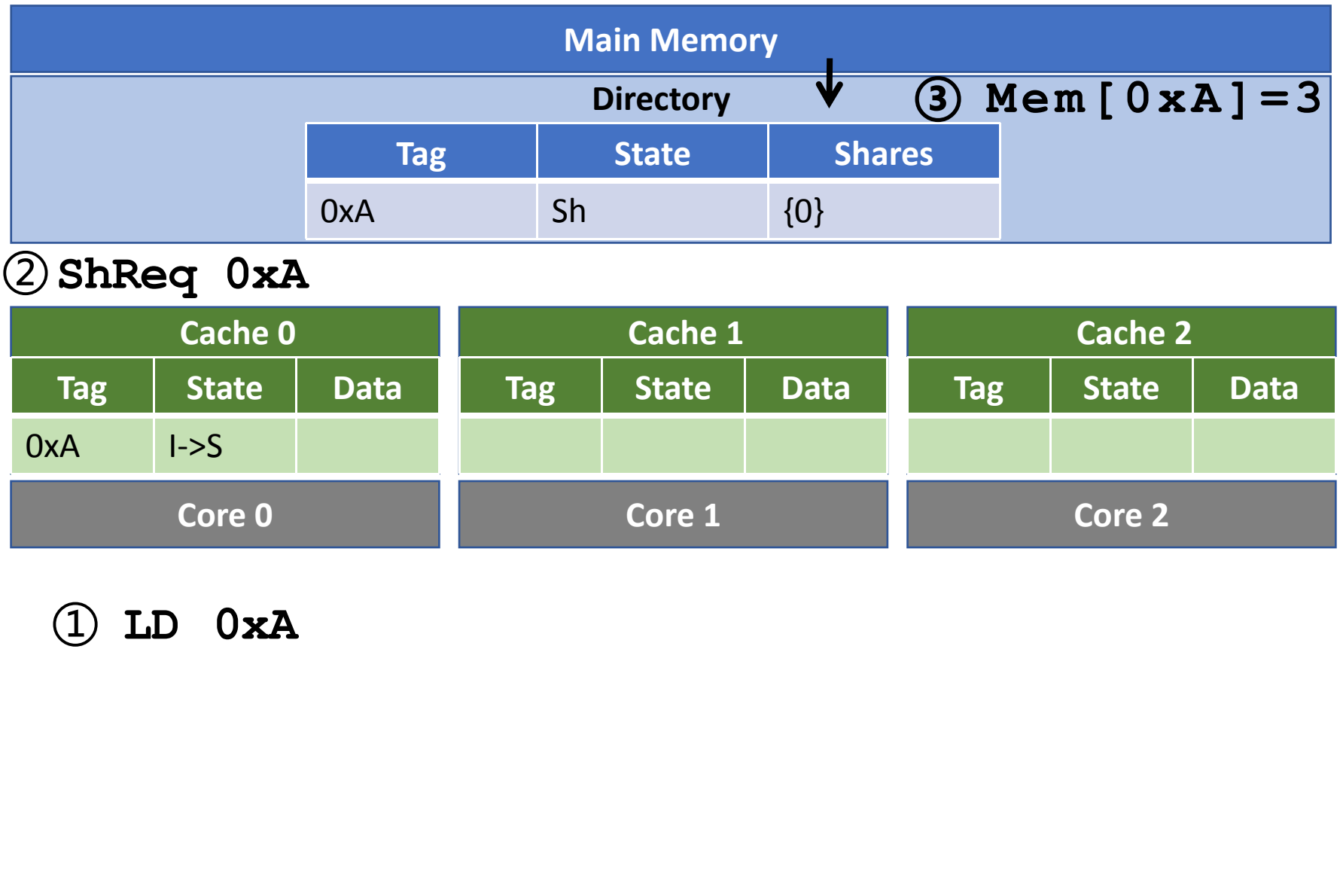
Tag	State	Shares
0xA	Sh	{0}

② **ShReq 0xA**

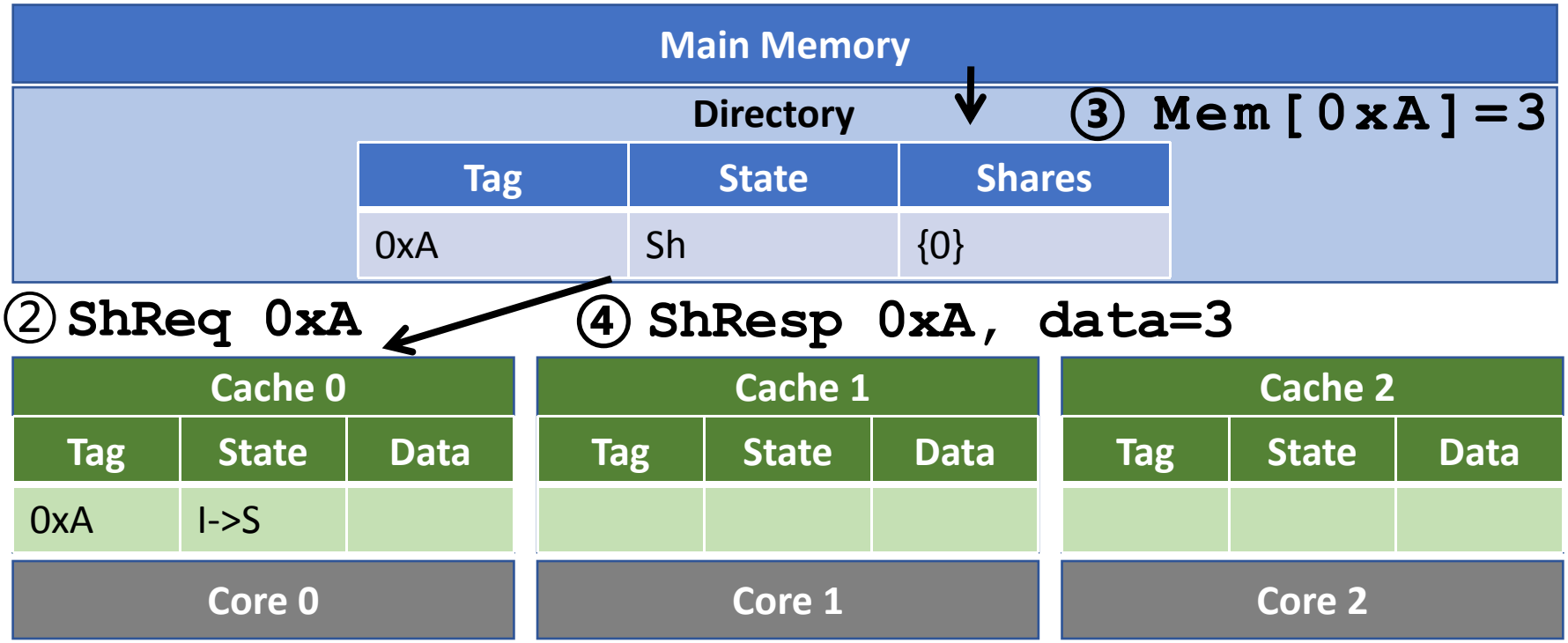
Cache 0			Cache 1			Cache 2		
Tag	State	Data	Tag	State	Data	Tag	State	Data
0xA	I->S							
Core 0			Core 1			Core 2		

① **LD 0xA**

基于Directory的MSI协议 - 示例

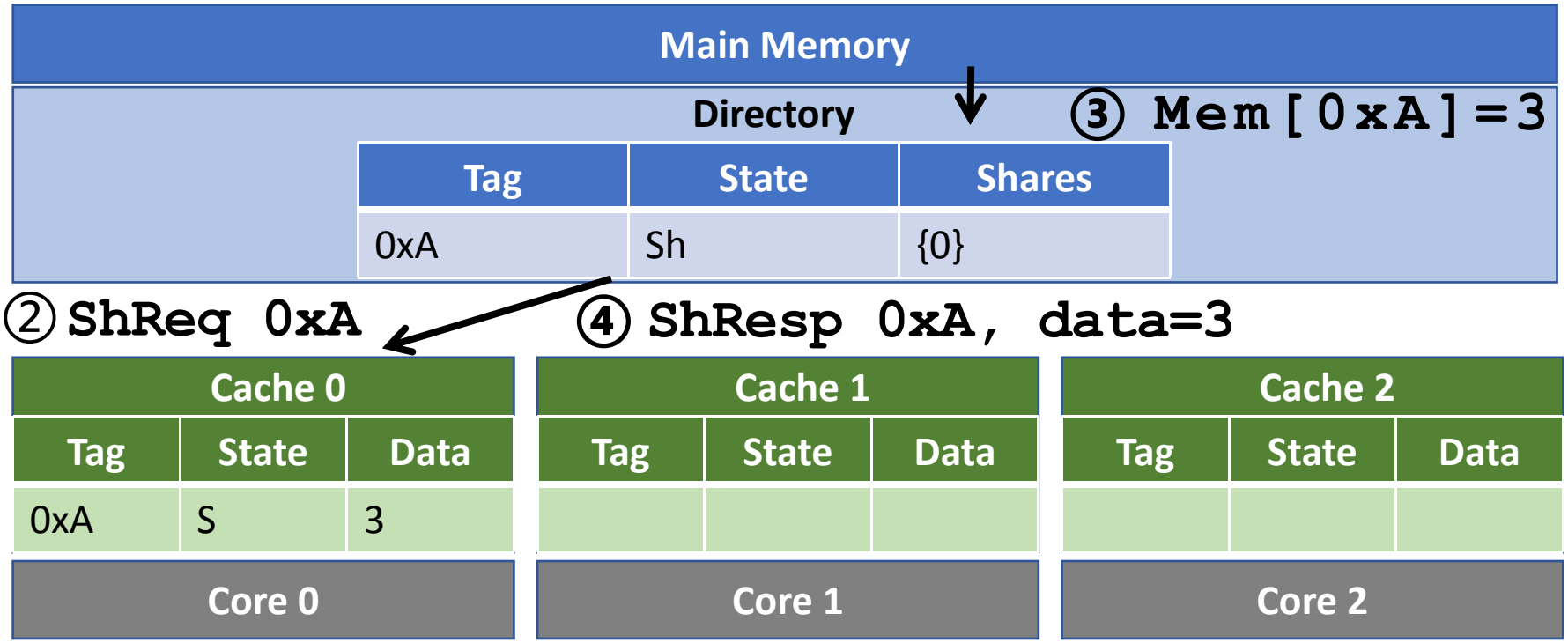


基于Directory的MSI协议 - 示例



① LD 0xA

基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例

Main Memory

Directory

Tag	State	Shares
0xA	Sh	{0}

Cache 0		
Tag	**State**	**Data**
0xA	S	3
Core 0		

Cache 1		
Tag	**State**	**Data**
Core 1		

Cache 2		
Tag	**State**	**Data**
Core 2		

基于Directory的MSI协议 - 示例

Main Memory

Directory

Tag	State	Shares
0xA	Sh	{0}

Cache 0		
Tag	**State**	**Data**
0xA	S	3
Core 0		

Cache 1		
Tag	**State**	**Data**
Core 1		

Cache 2		
Tag	**State**	**Data**
Core 2		

① LD 0xA

基于Directory的MSI协议 - 示例

Main Memory

Directory

Tag	State	Shares
0xA	Sh	{0}

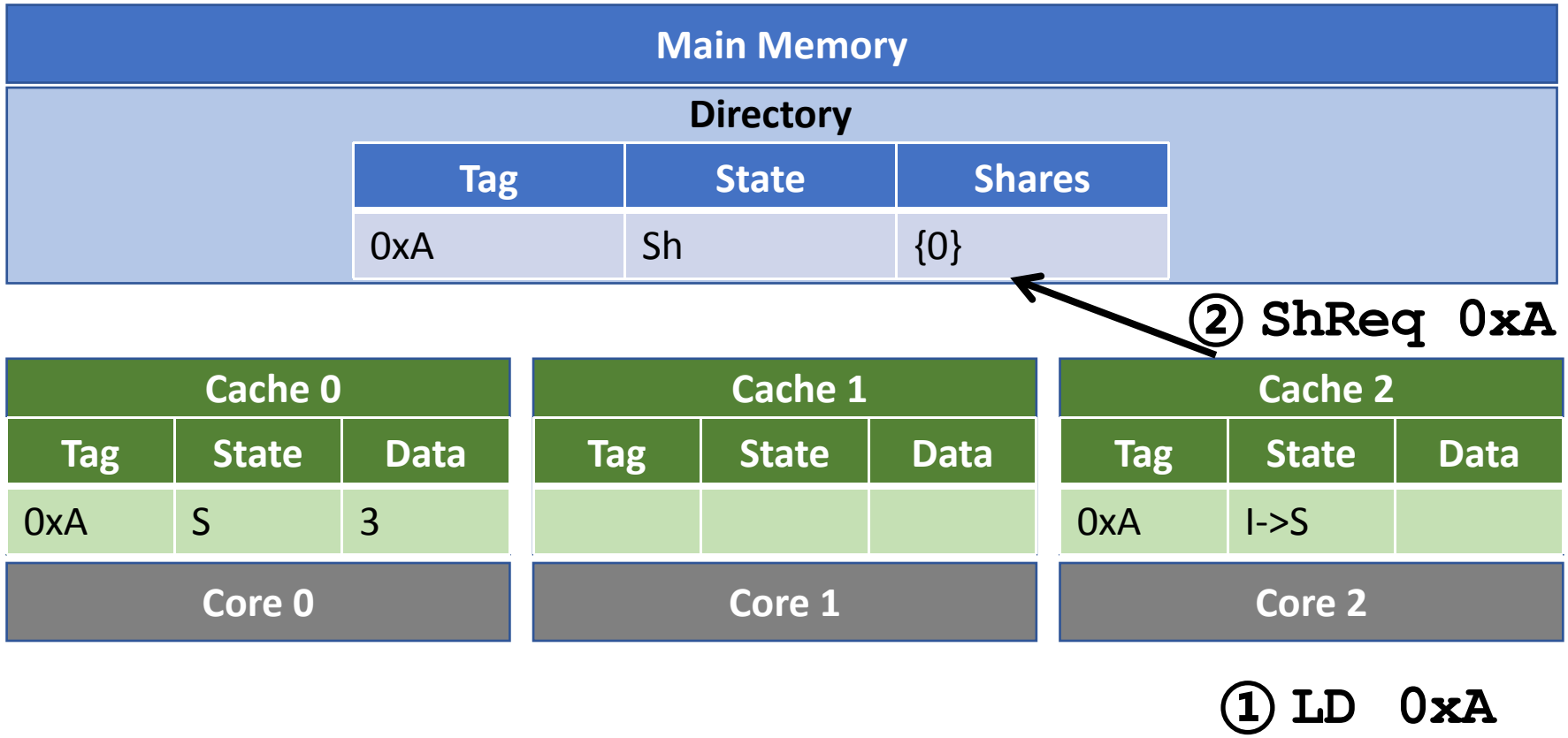
Cache 0		
Tag	**State**	**Data**
0xA	S	3
Core 0		

Cache 1		
Tag	**State**	**Data**
Core 1		

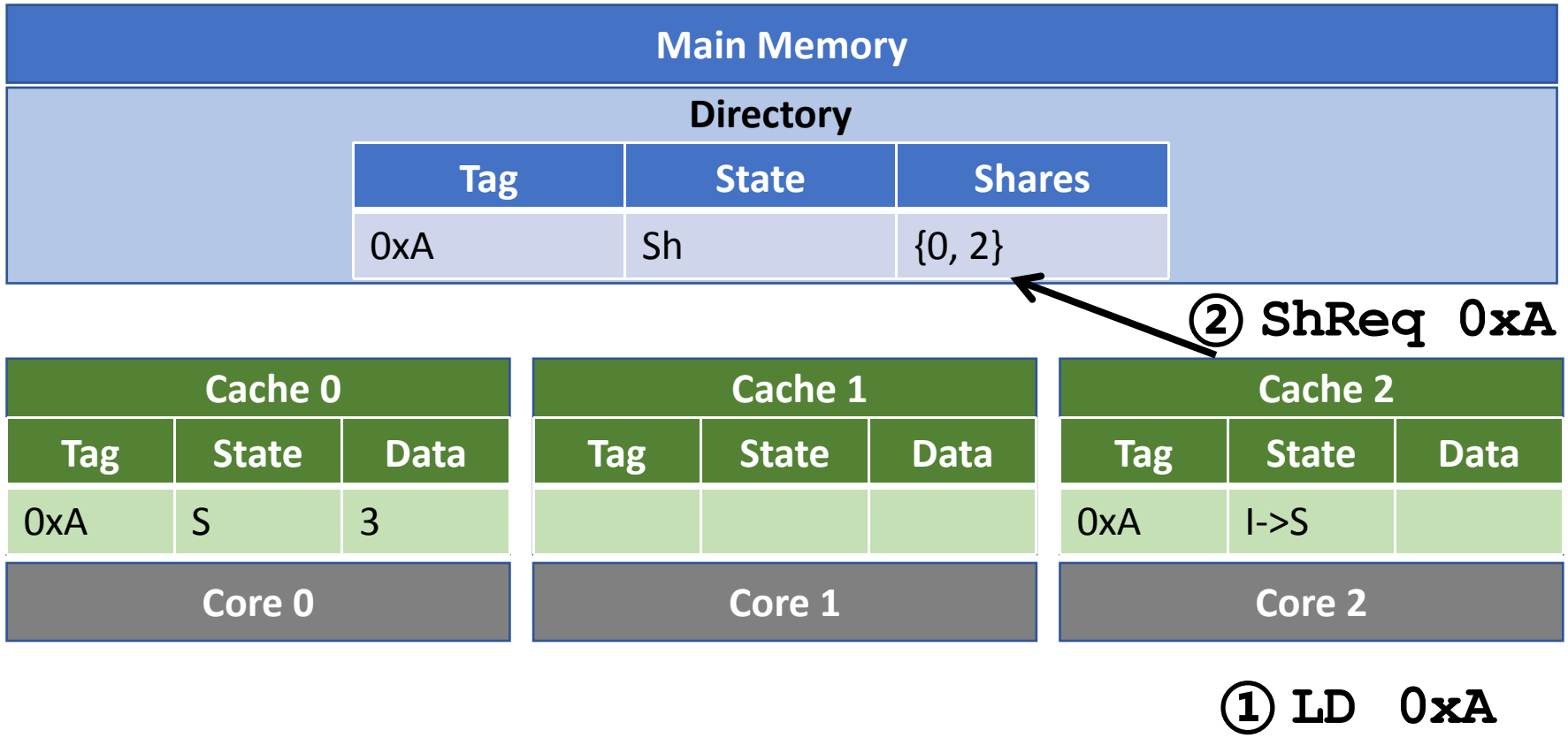
Cache 2		
Tag	**State**	**Data**
0xA	I->S	
Core 2		

① **LD 0xA**

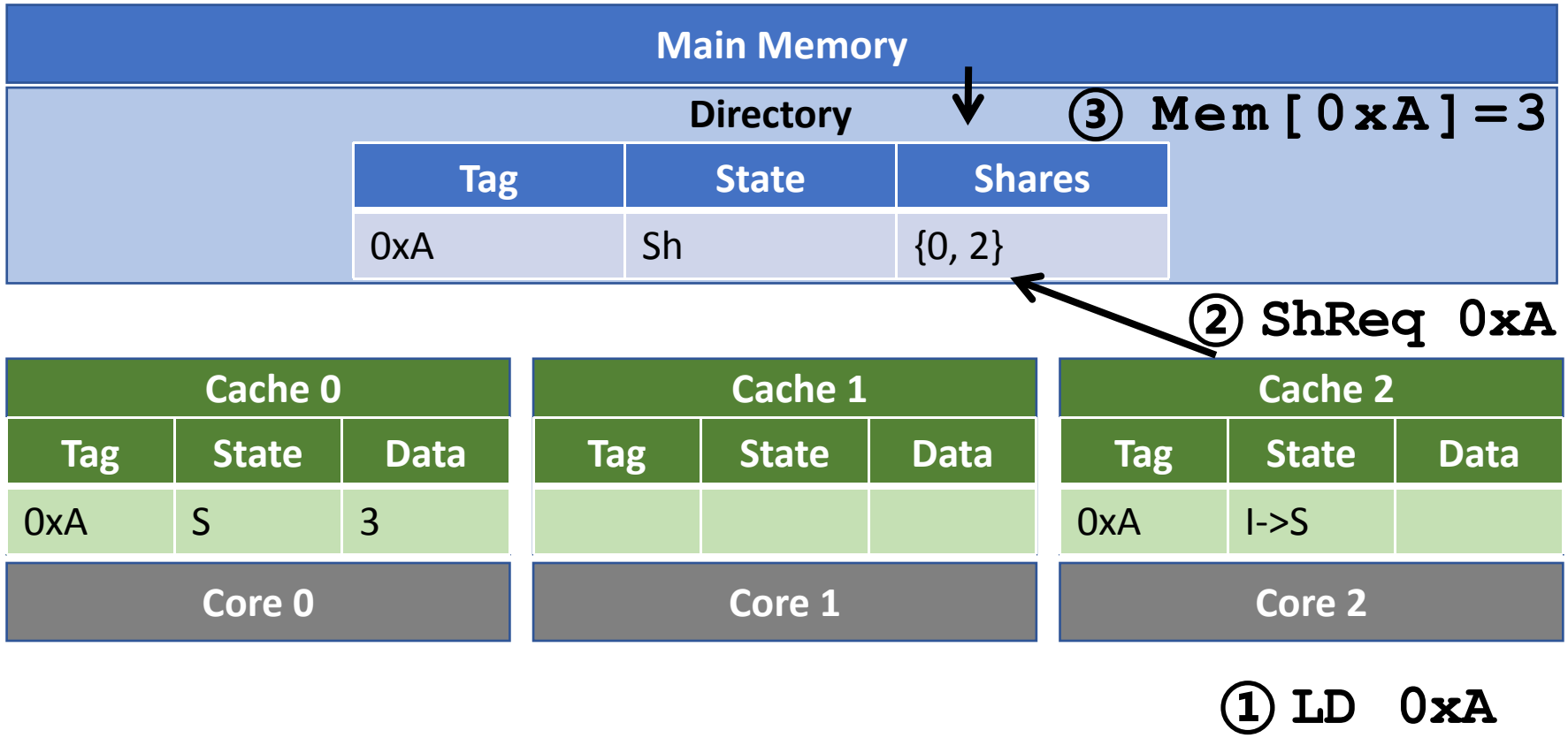
基于Directory的MSI协议 - 示例



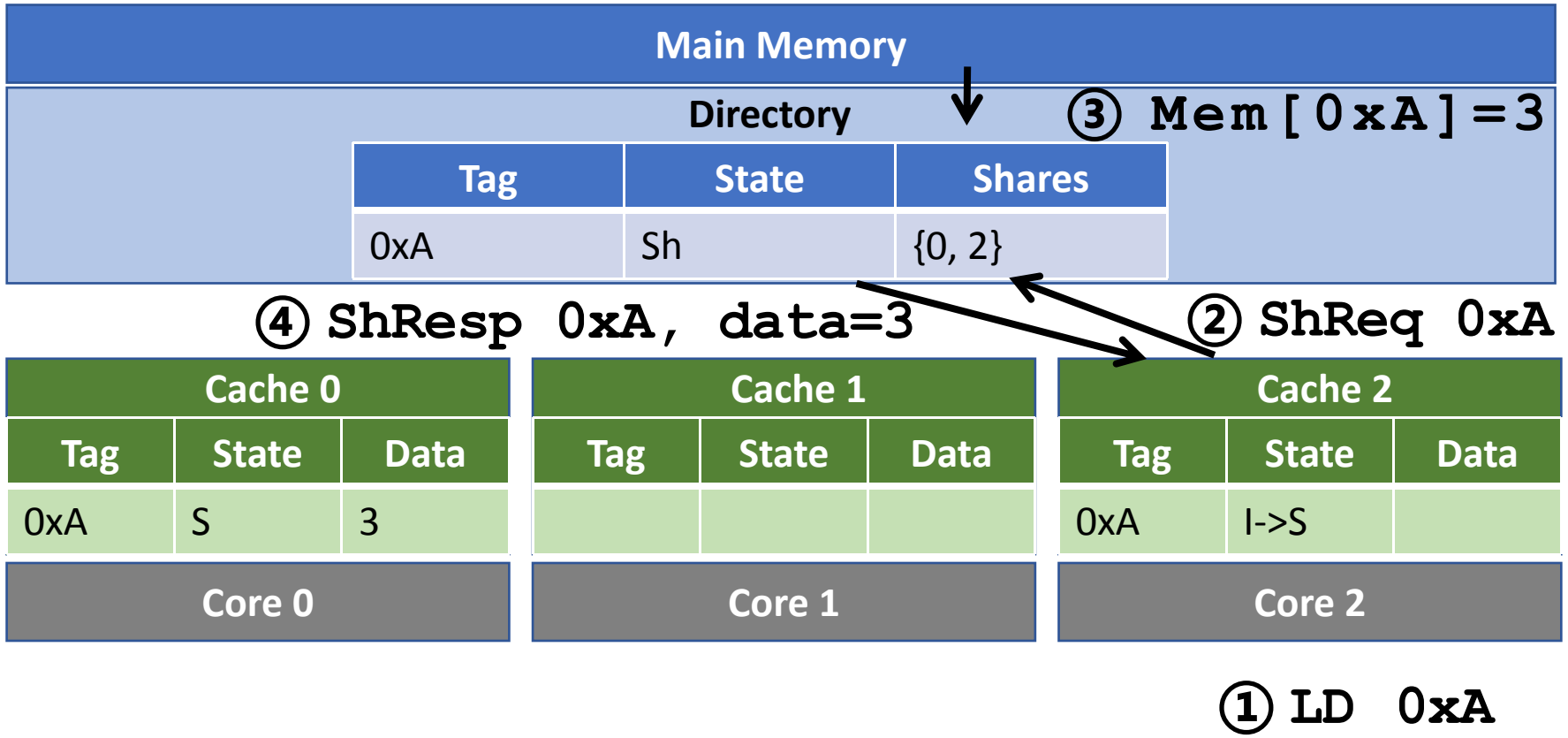
基于Directory的MSI协议 - 示例



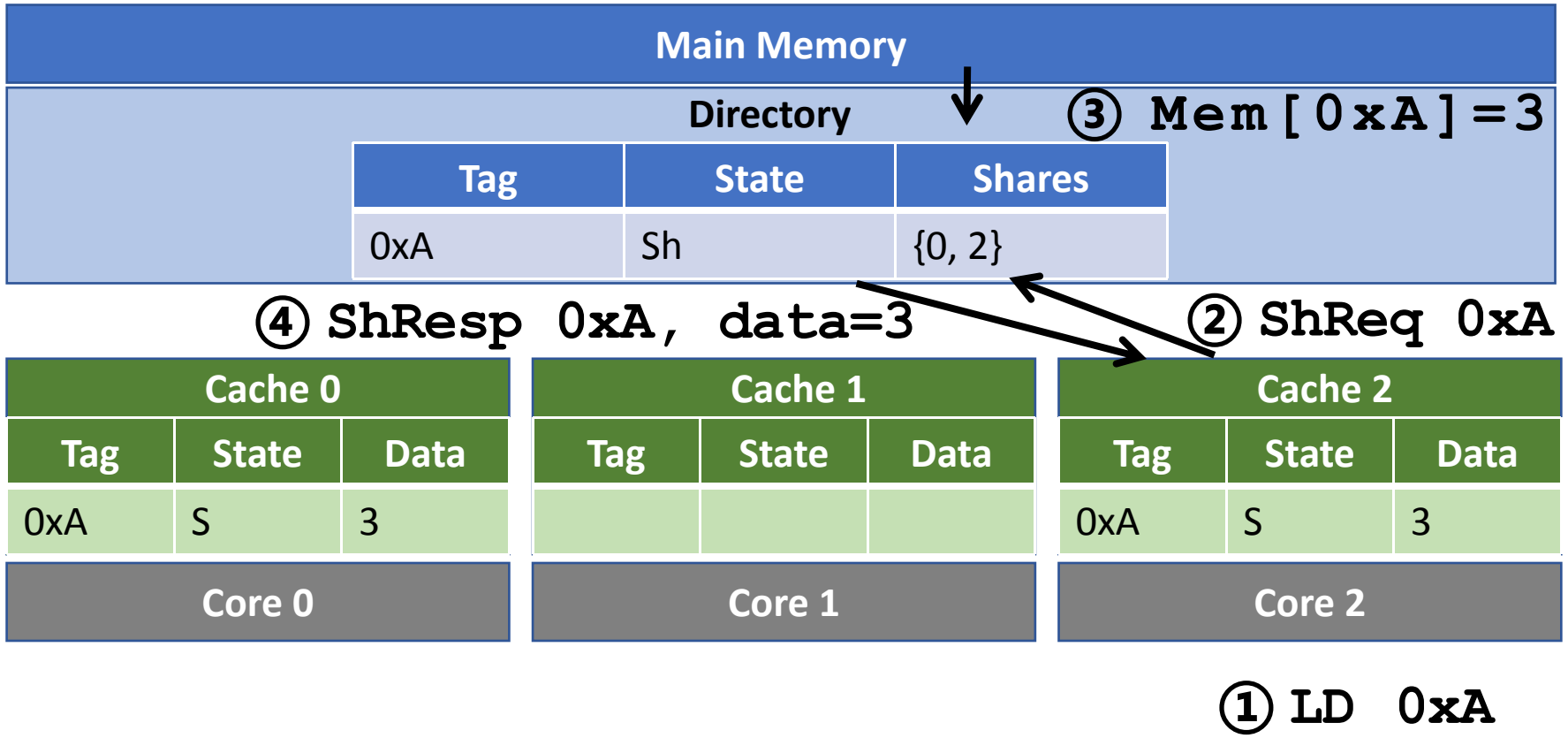
基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例

Main Memory

Directory

Tag	State	Shares
0xA	Sh	{0, 2}

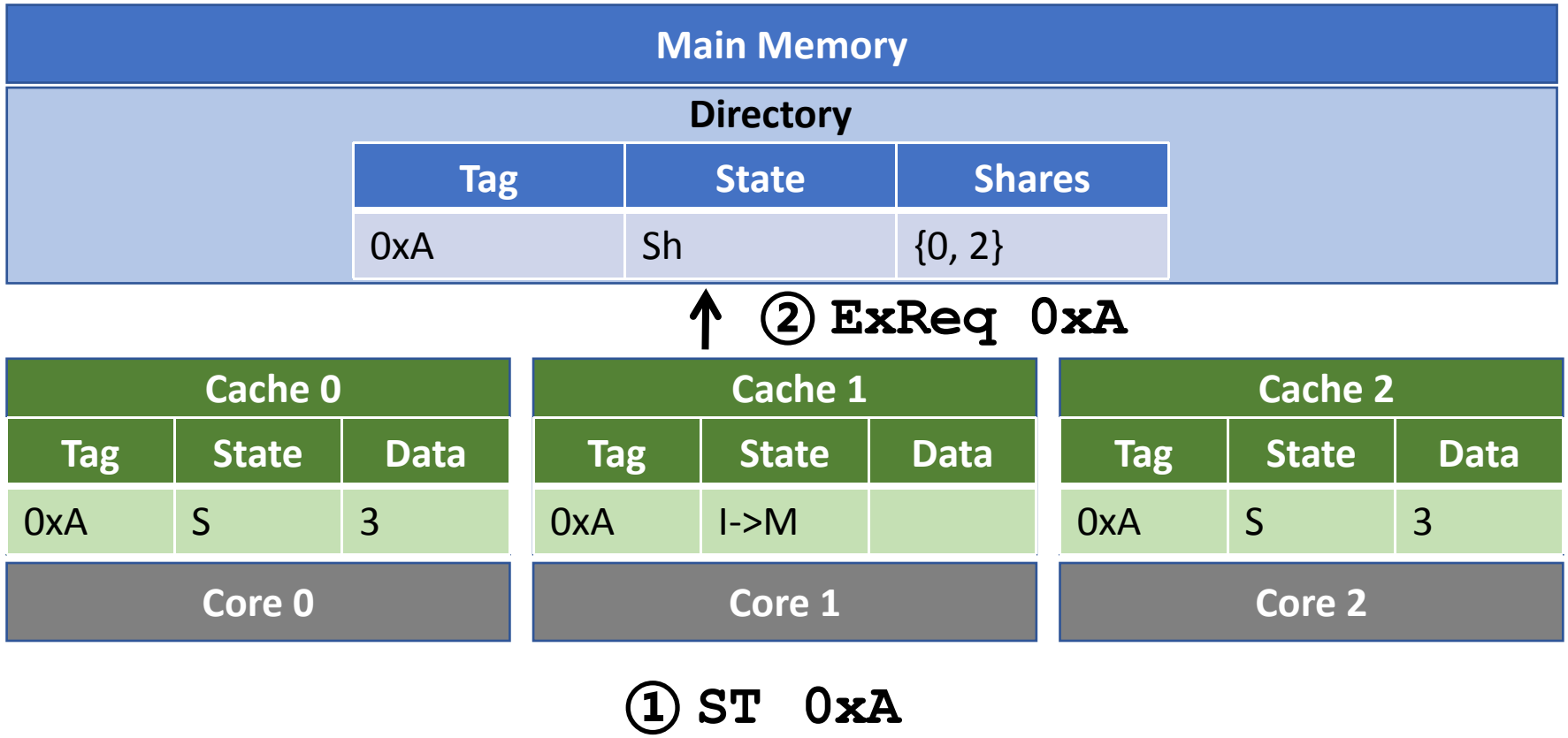
Cache 0		
Tag	**State**	**Data**
0xA	S	3
Core 0		

Cache 1		
Tag	**State**	**Data**
0xA	I->M	
Core 1		

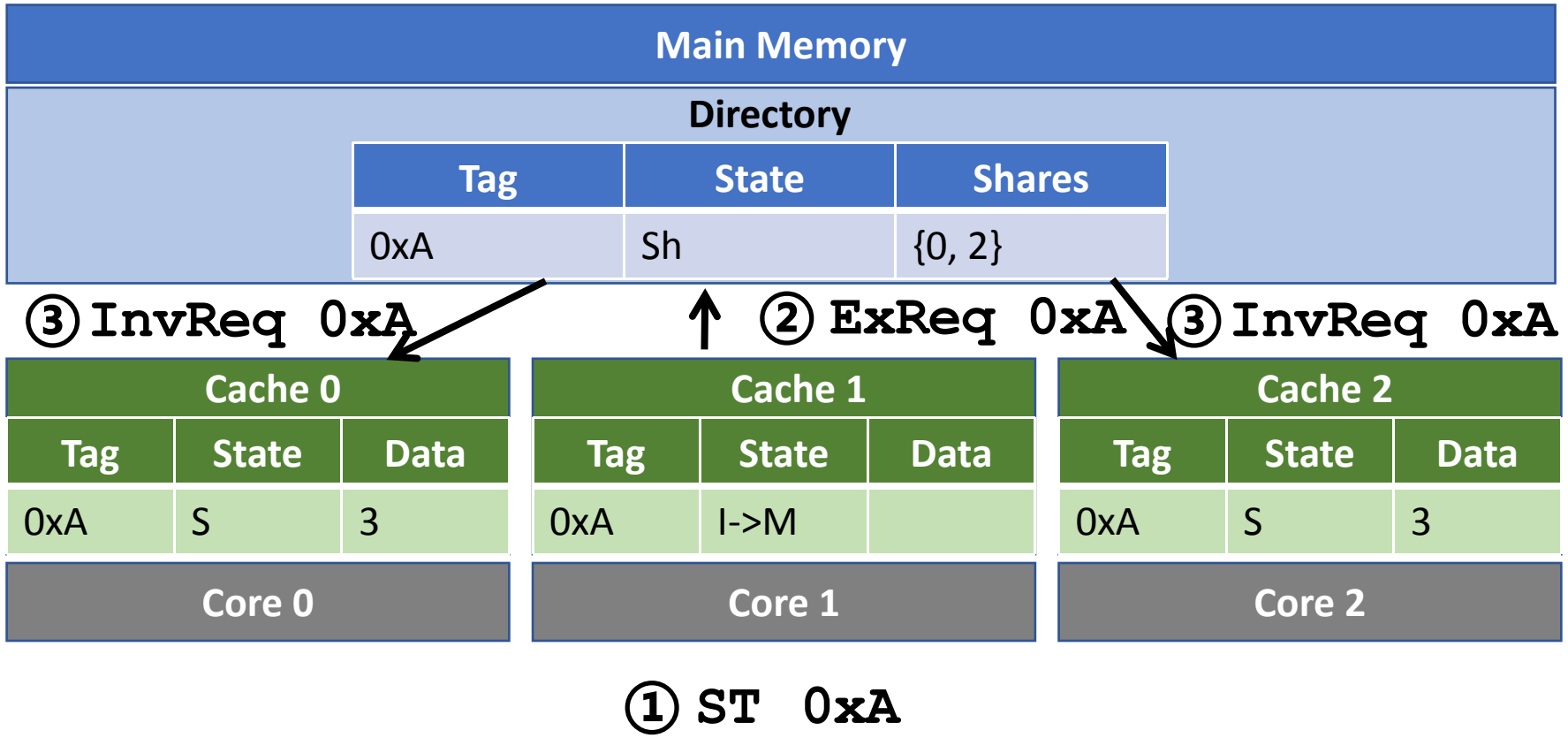
Cache 2		
Tag	**State**	**Data**
0xA	S	3
Core 2		

① **ST 0xA**

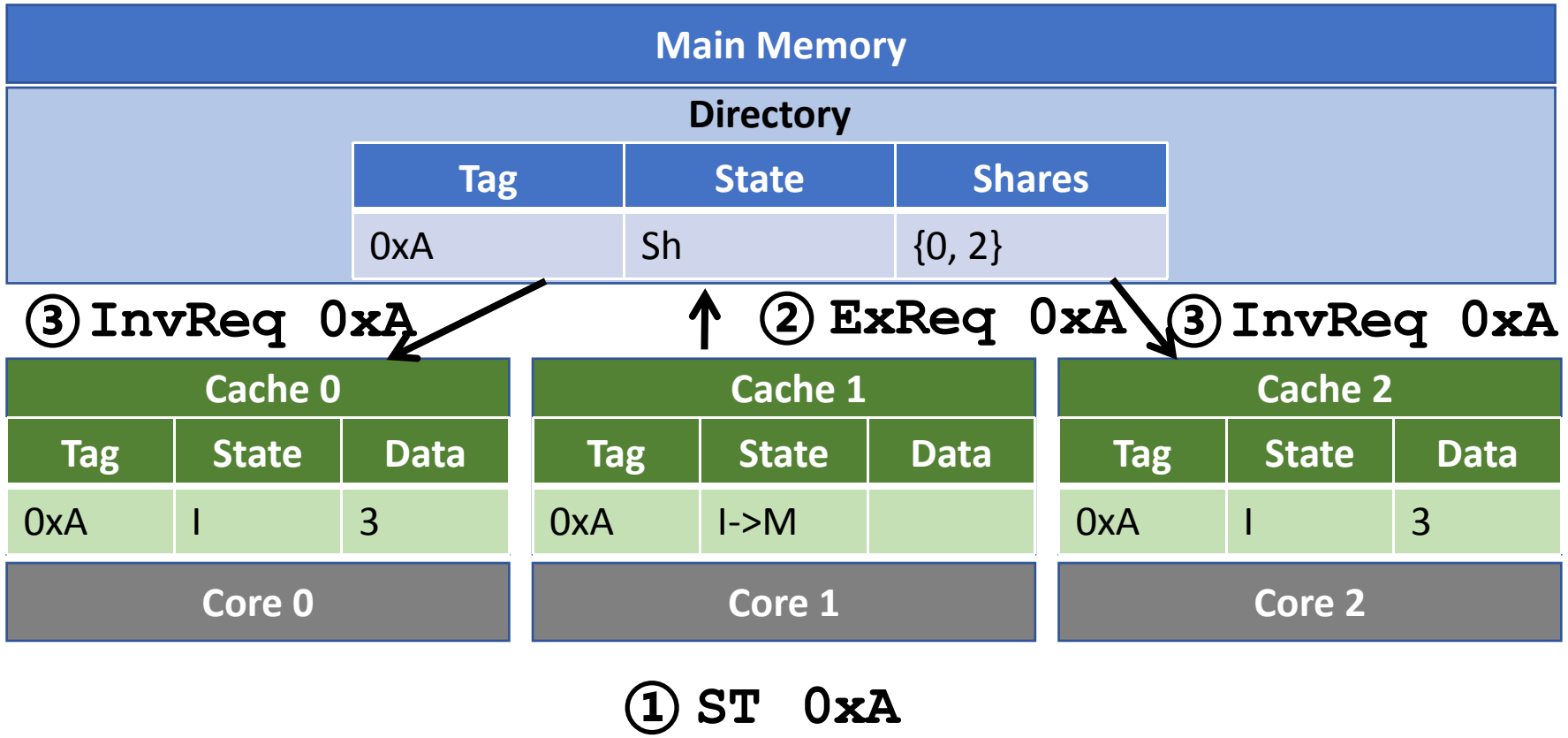
基于Directory的MSI协议 - 示例



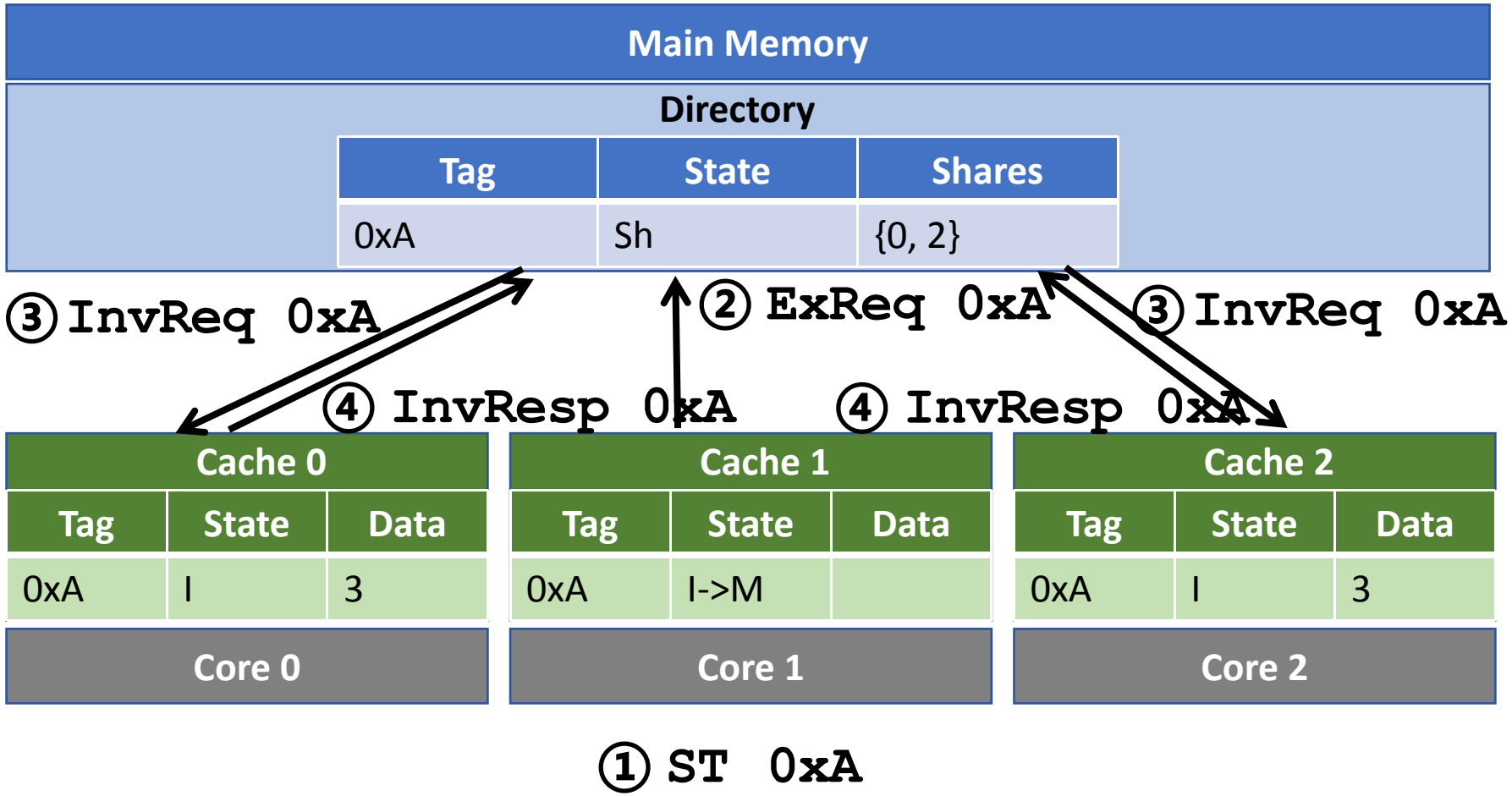
基于Directory的MSI协议 - 示例



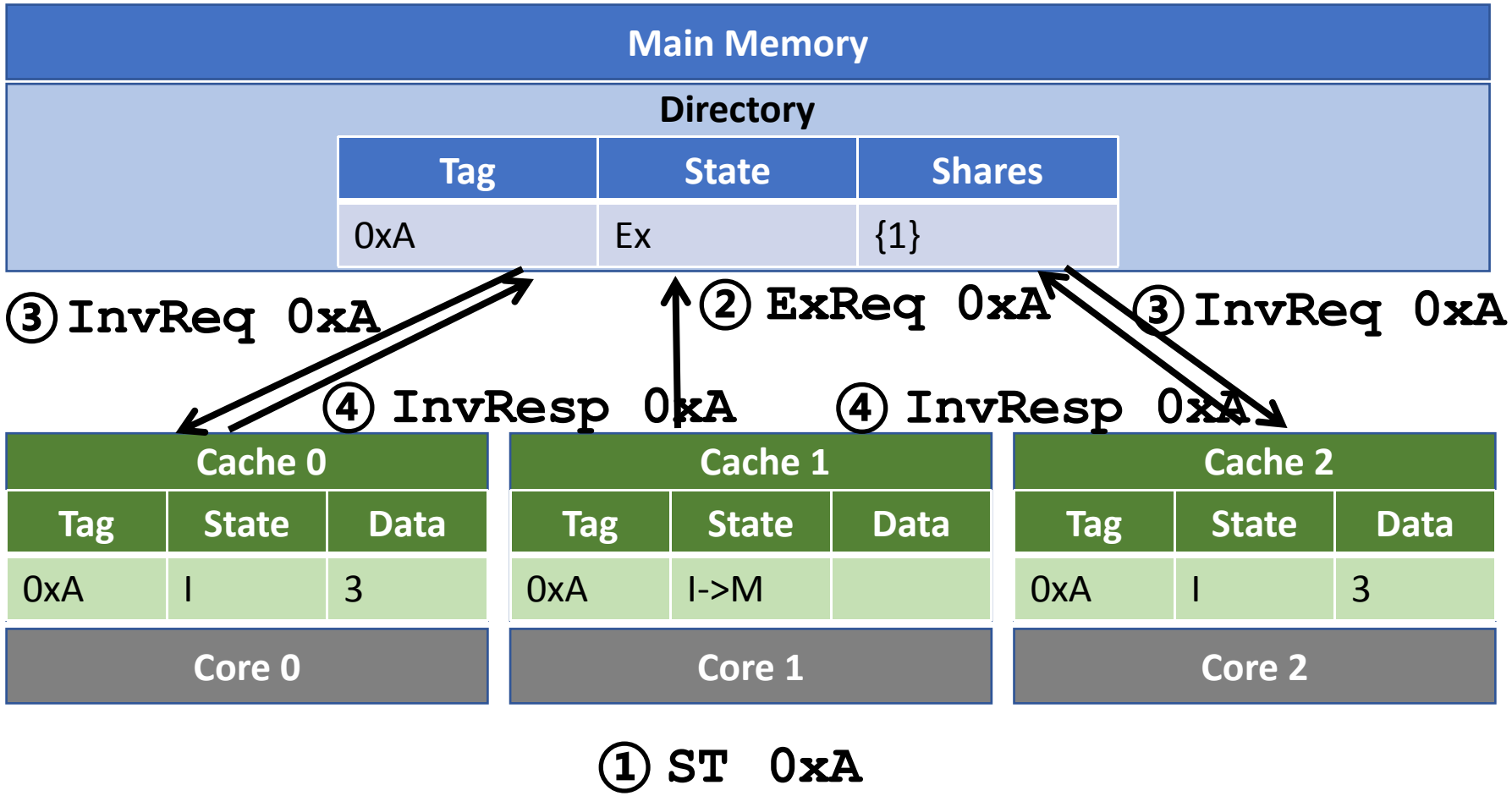
基于Directory的MSI协议 - 示例



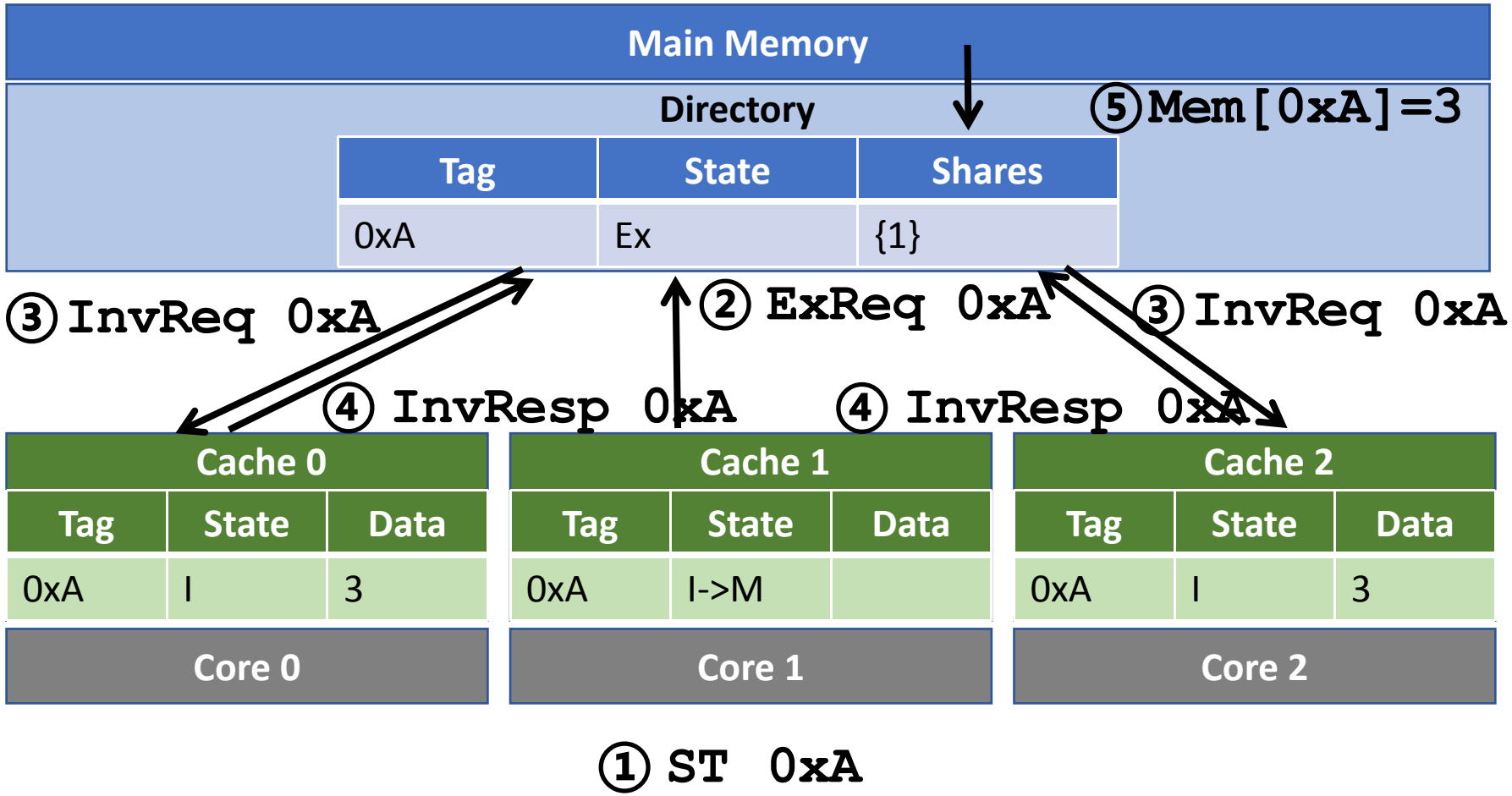
基于Directory的MSI协议 - 示例



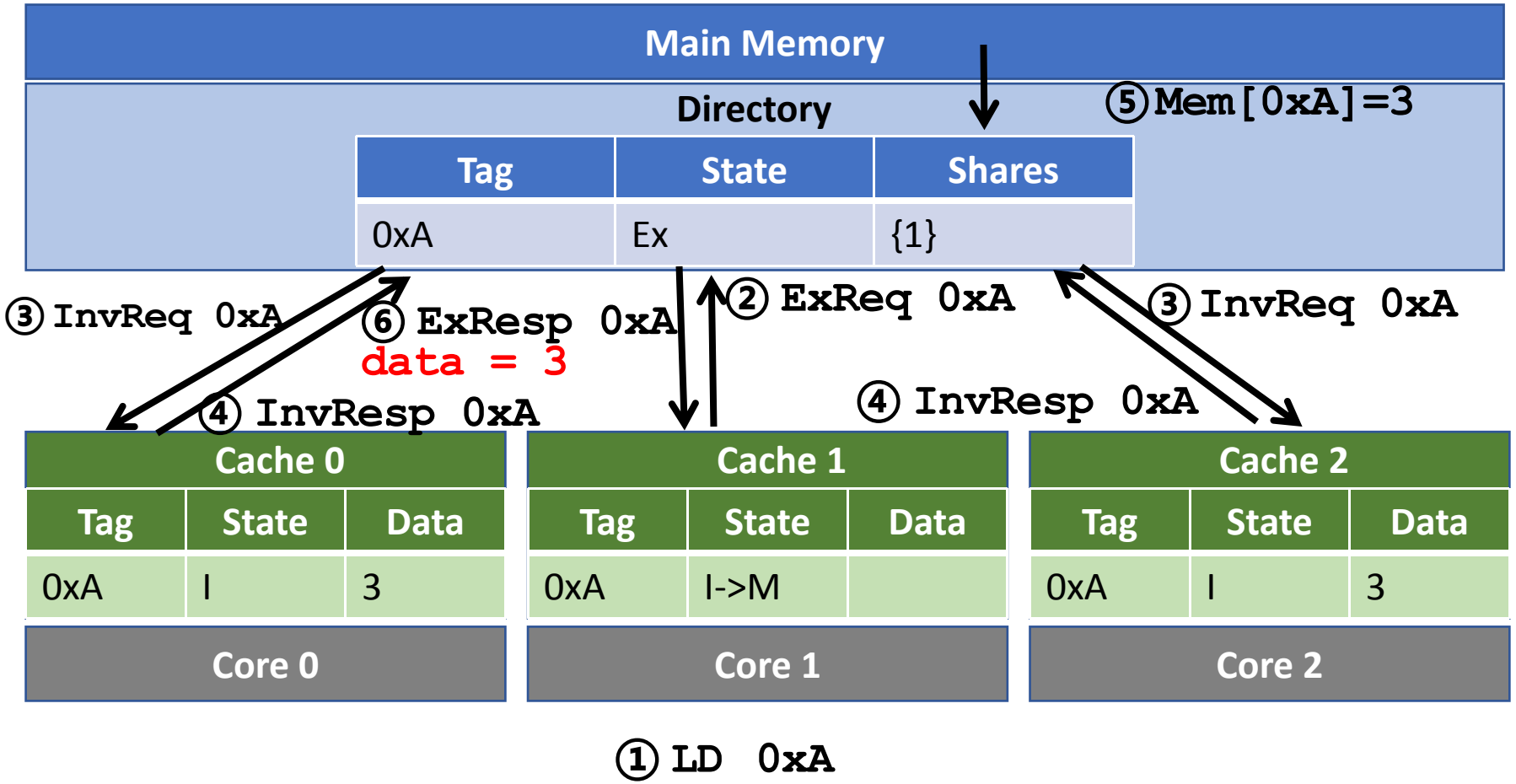
基于Directory的MSI协议 - 示例



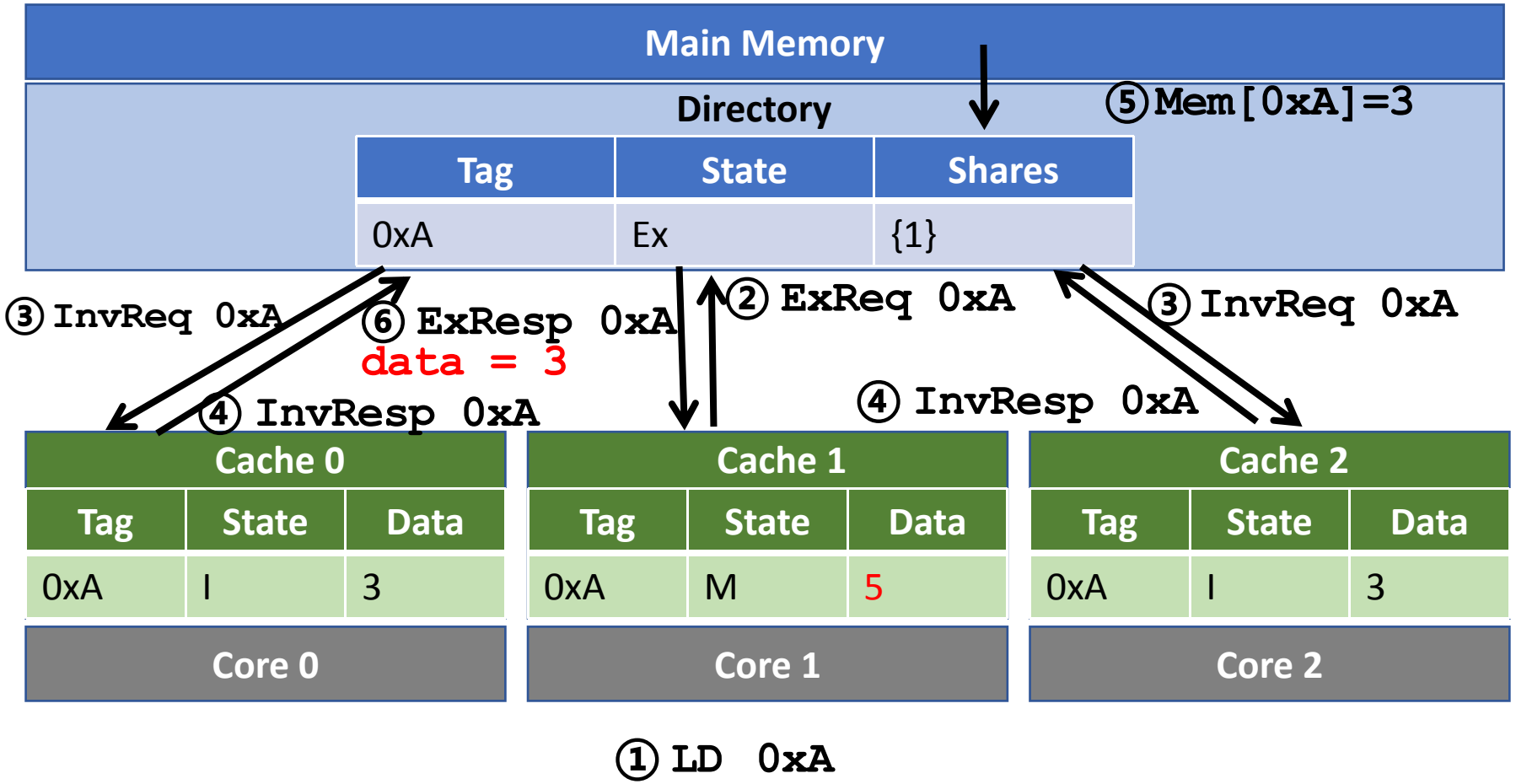
基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例

Main Memory

Directory

Tag	State	Shares
0xA	Ex	{1}

Cache 0		
Tag	State	Data
0xA	I	3

Core 0

Cache 1		
Tag	State	Data
0xA	M->I	5

Core 1

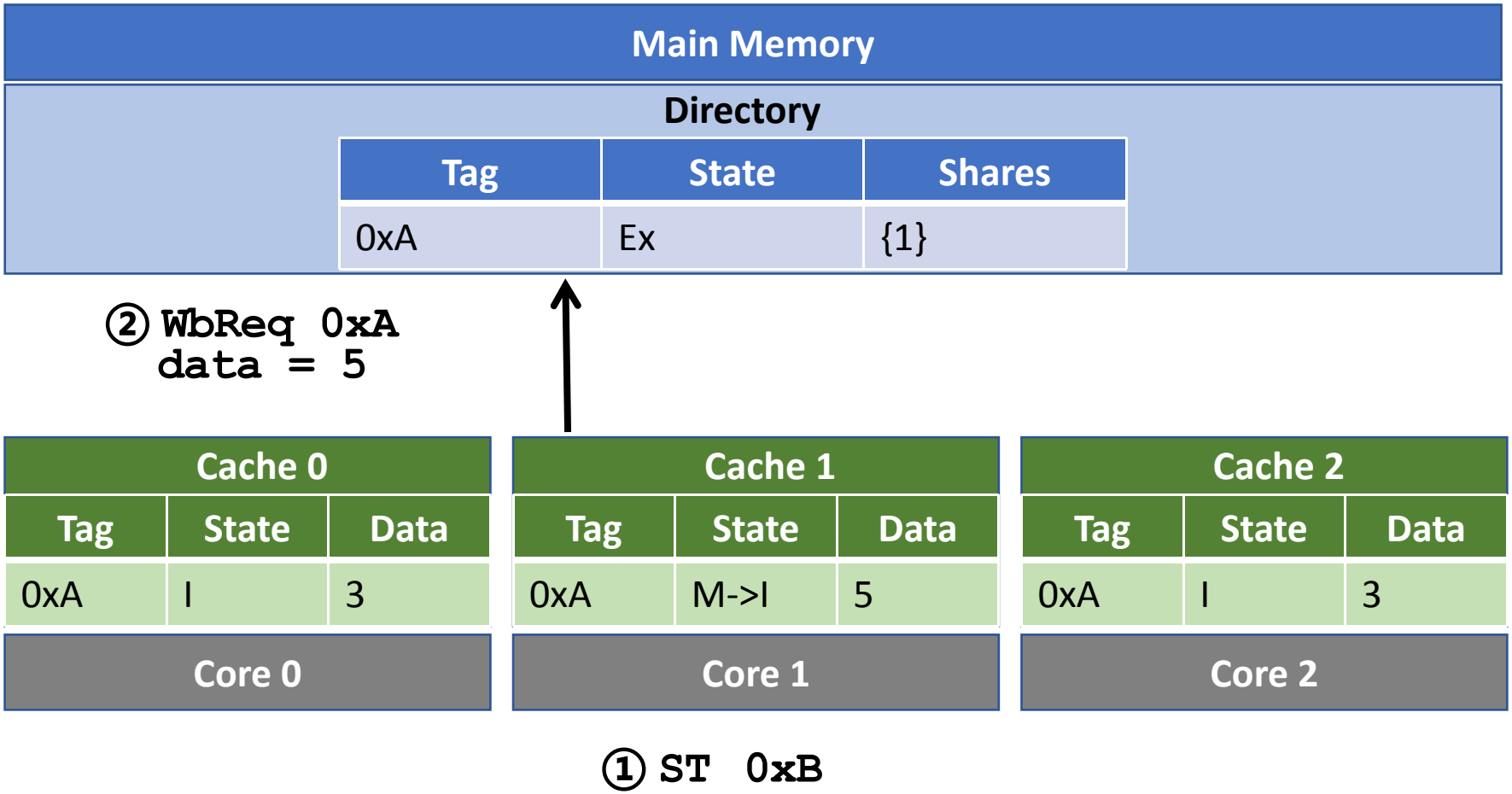
Cache 2		
Tag	State	Data
0xA	I	3

Core 2

① ST 0xB

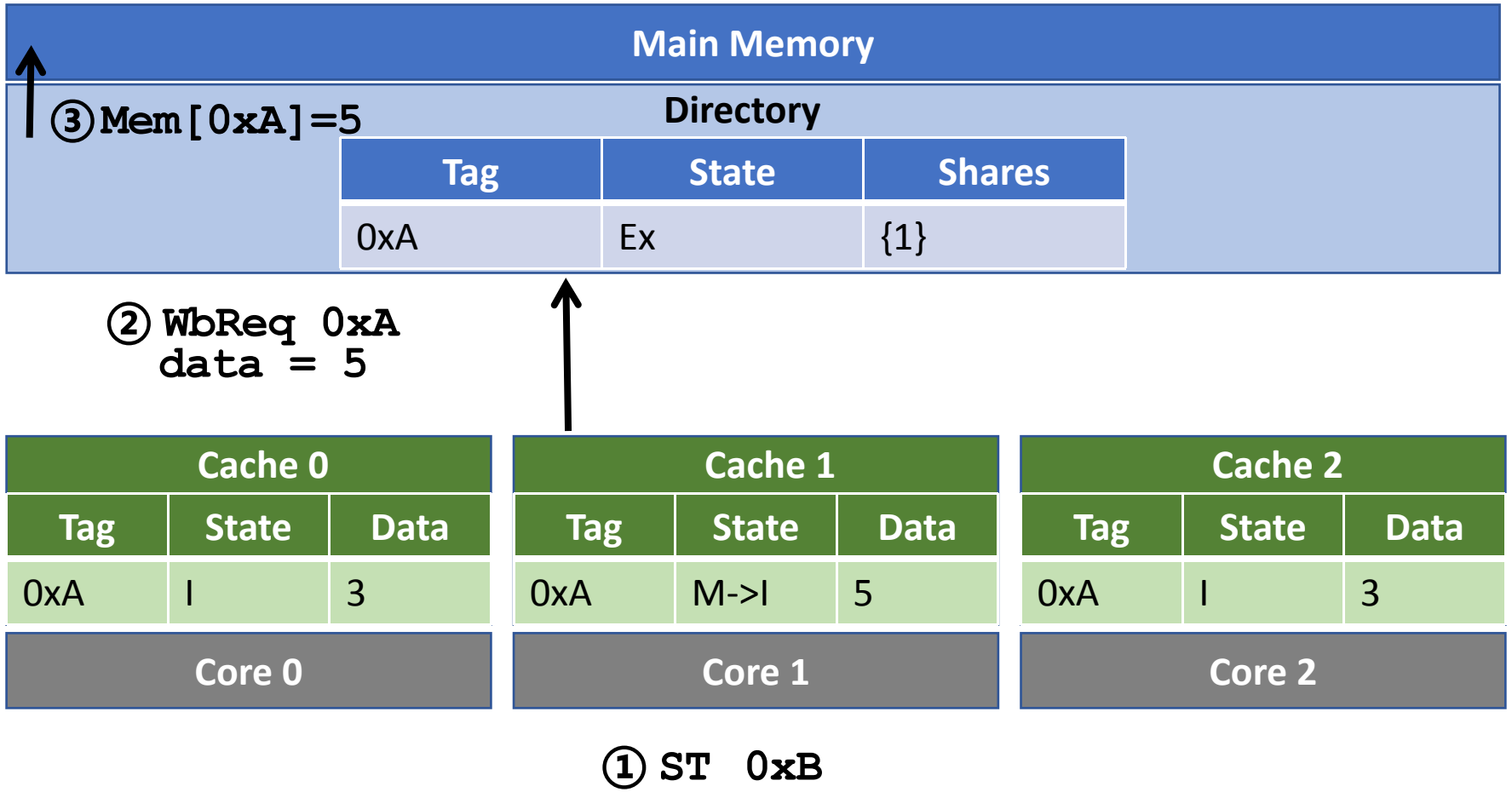
core 1又一次store，地址为0XB，将block 0x A替换

基于Directory的MSI协议 - 示例

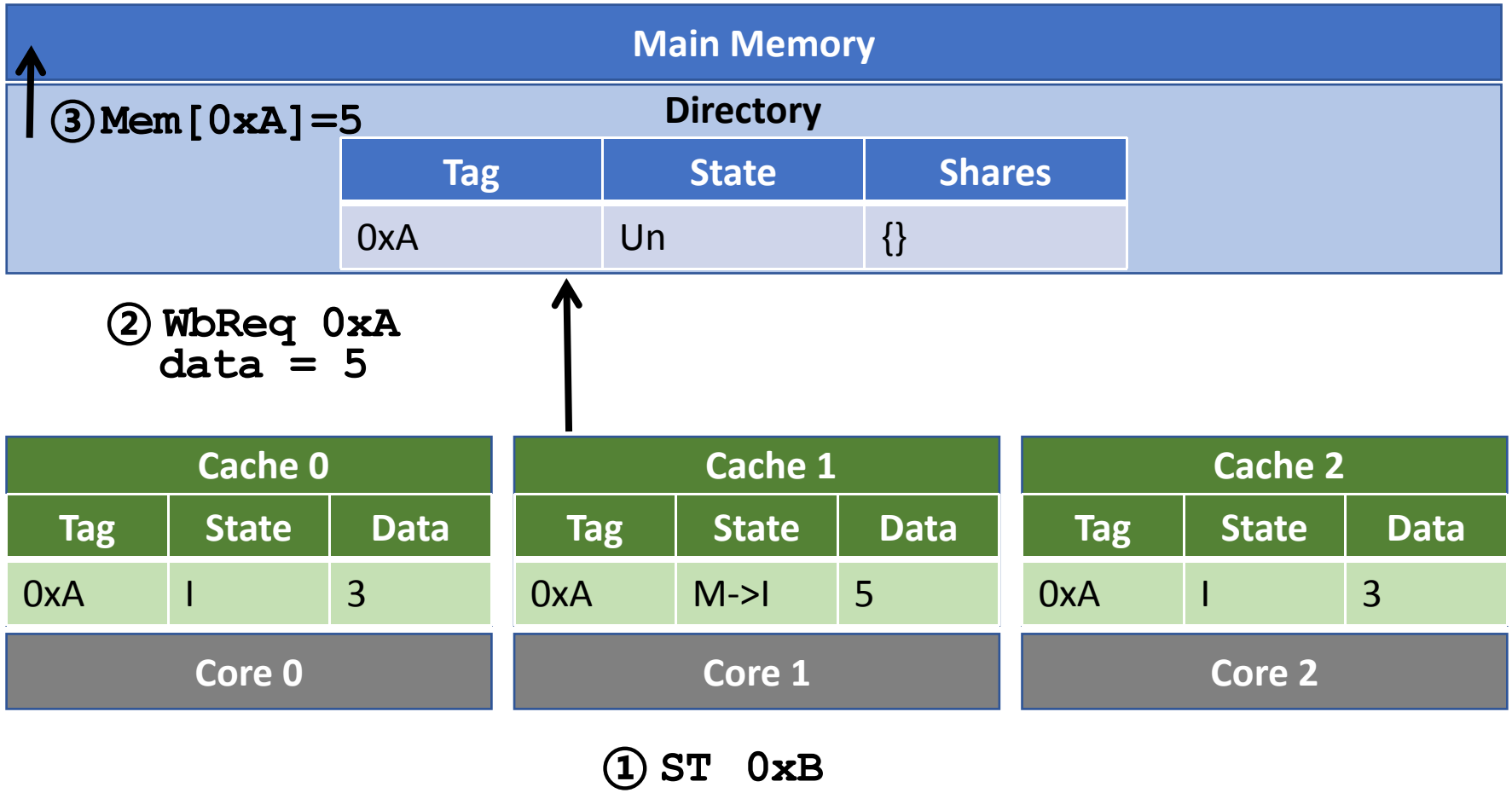


因为0xA数据是dirty的，先发送WB请求给directory，将数据写回内存

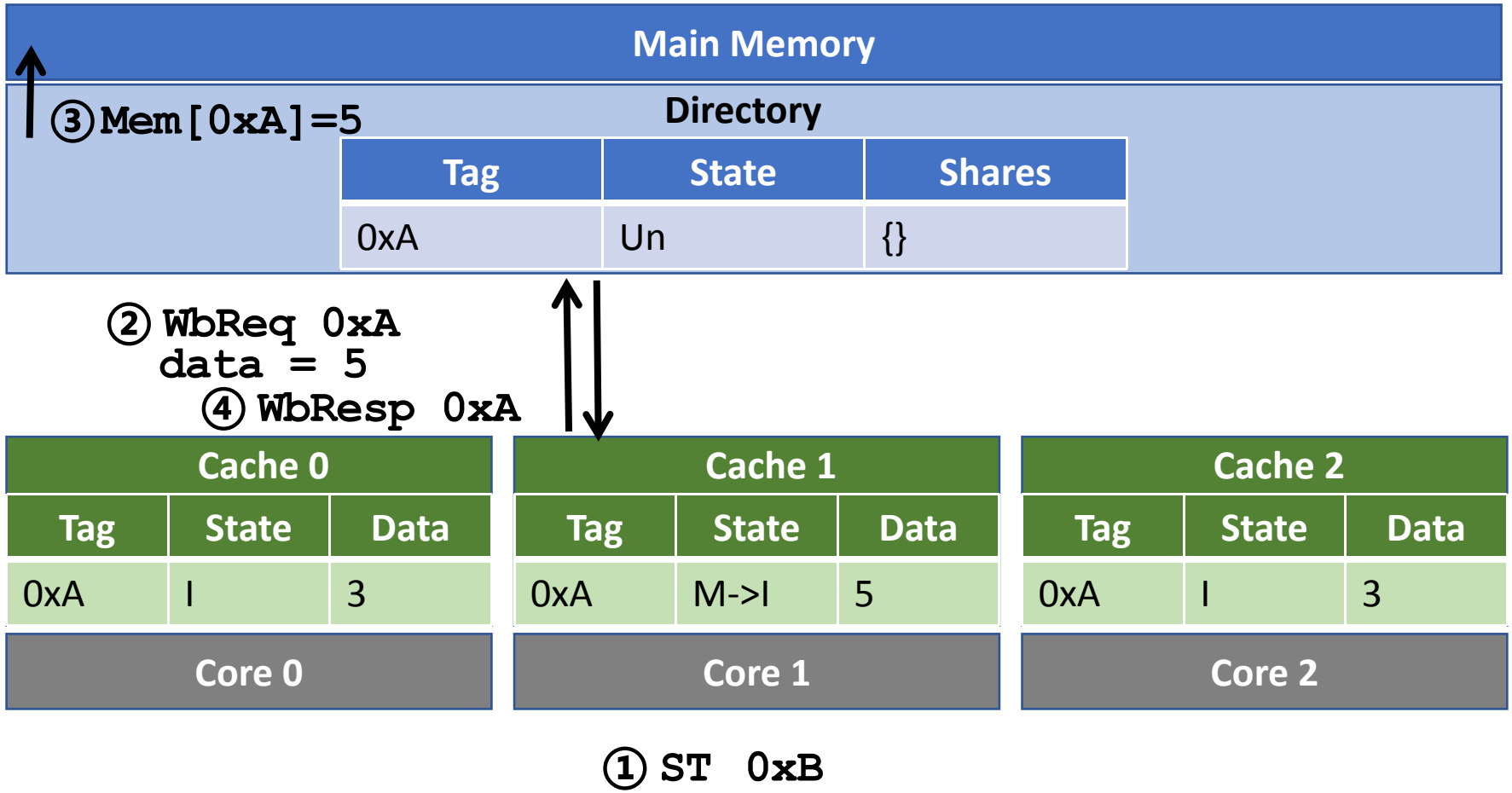
基于Directory的MSI协议 - 示例



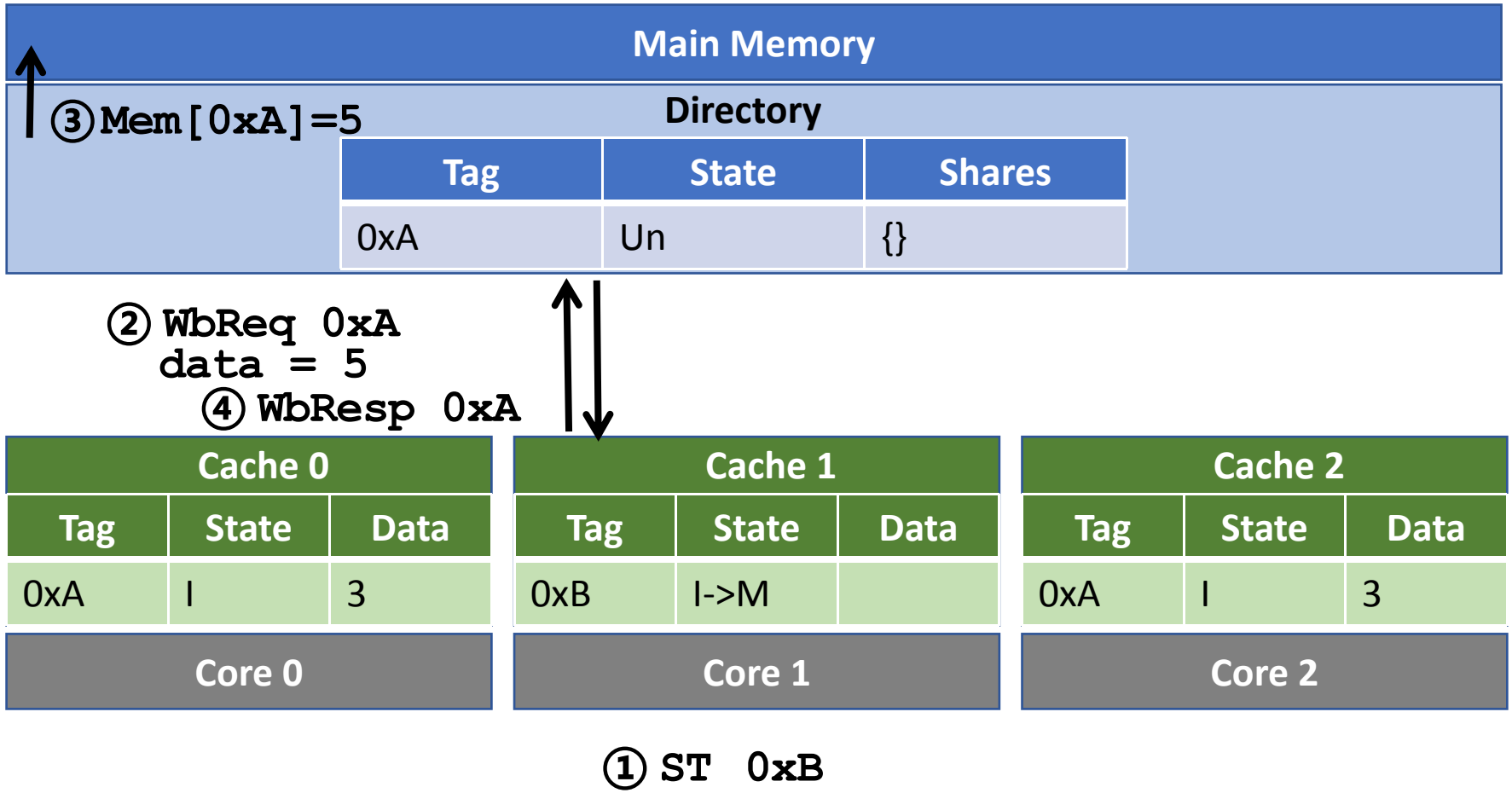
基于Directory的MSI协议 - 示例



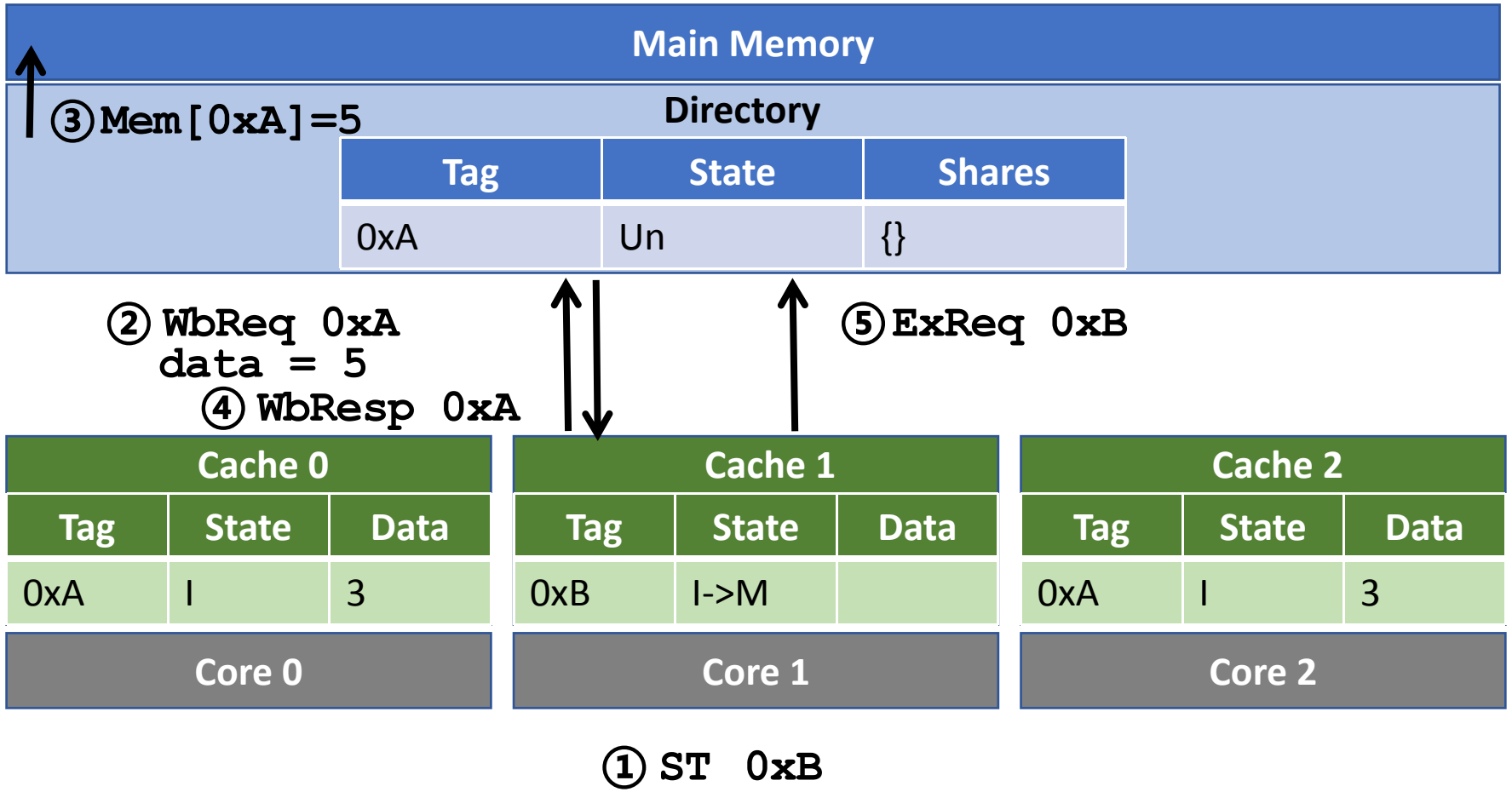
基于Directory的MSI协议 - 示例



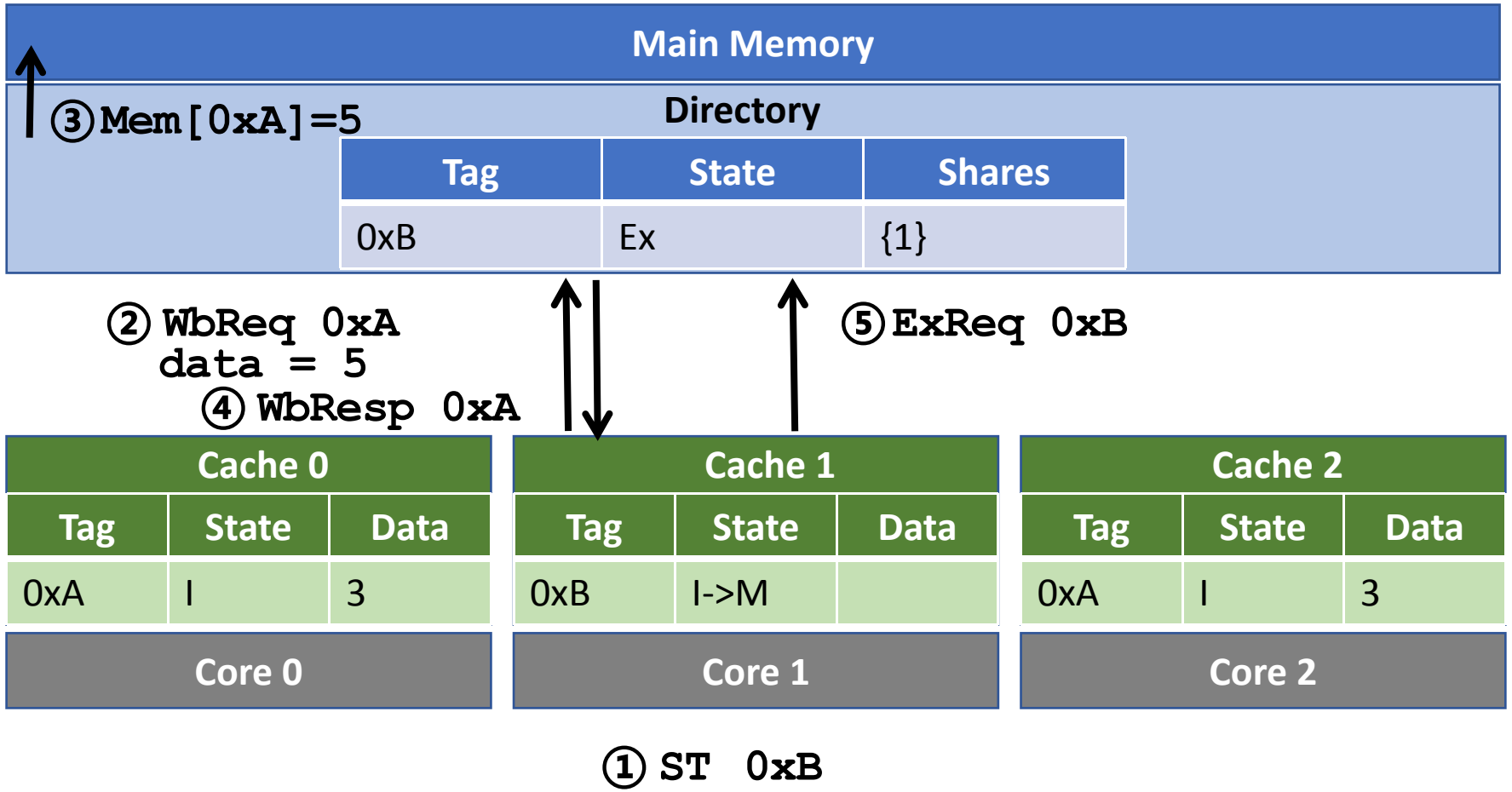
基于Directory的MSI协议 - 示例



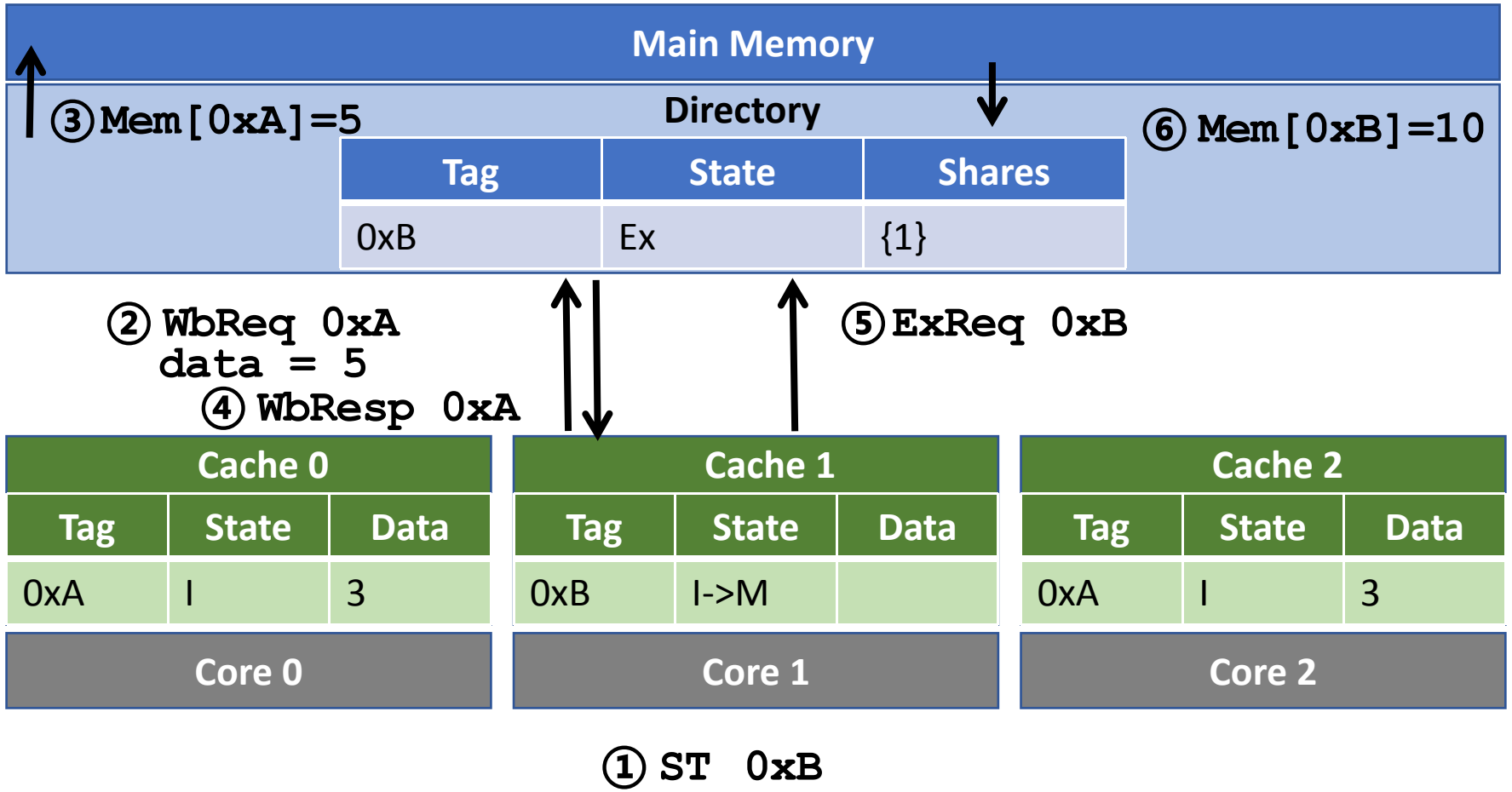
基于Directory的MSI协议 - 示例



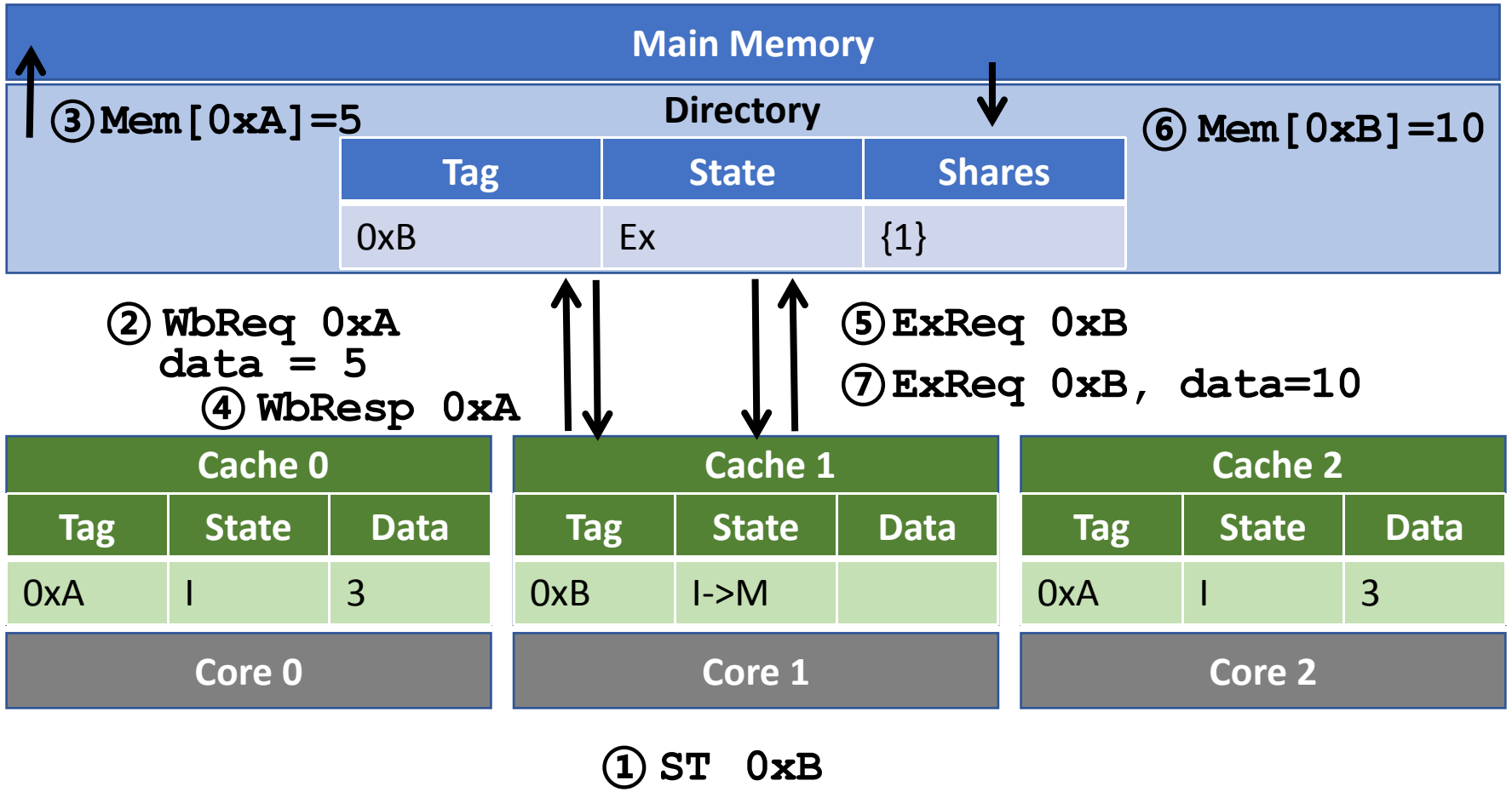
基于Directory的MSI协议 - 示例



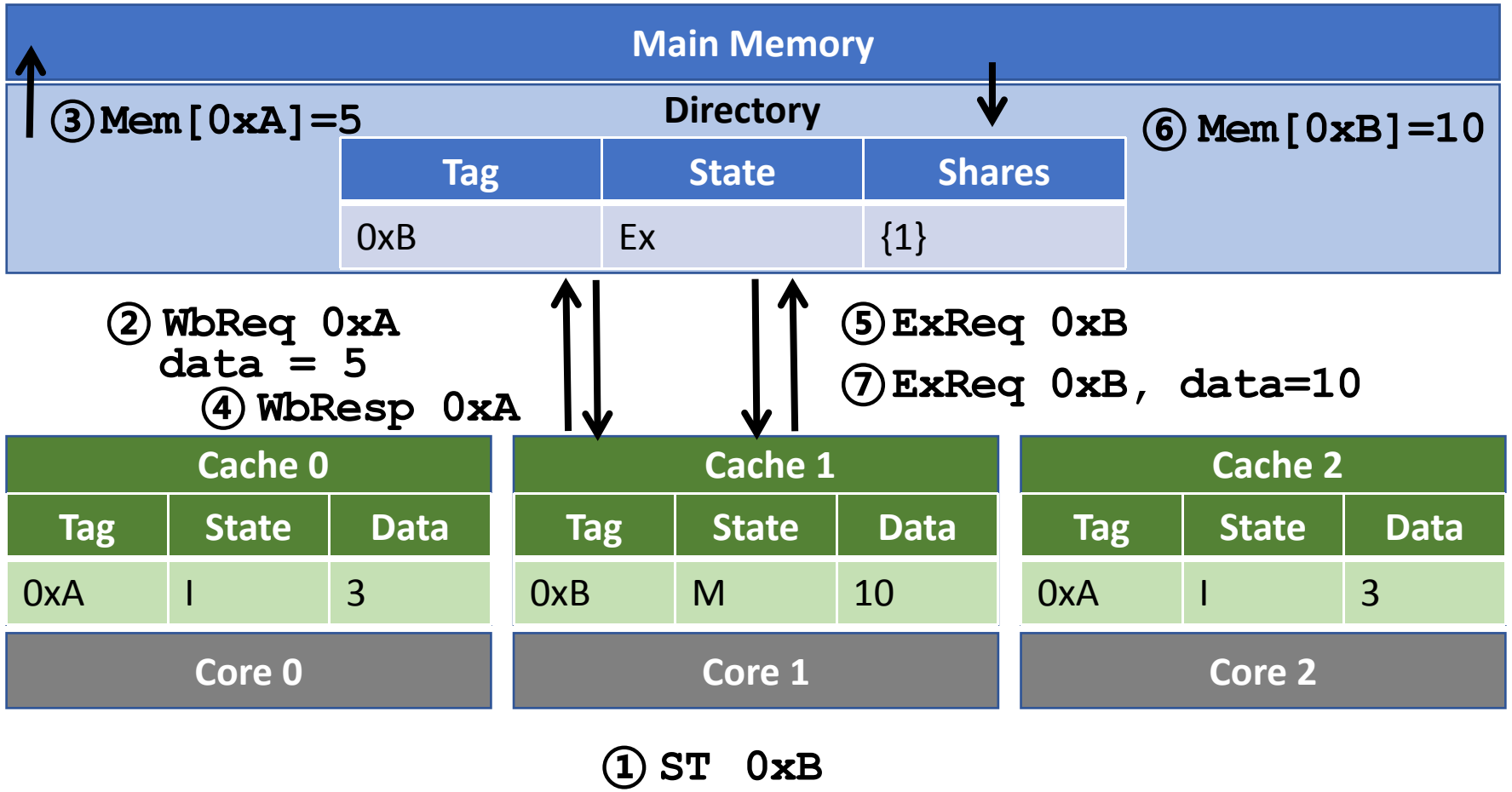
基于Directory的MSI协议 - 示例



基于Directory的MSI协议 - 示例



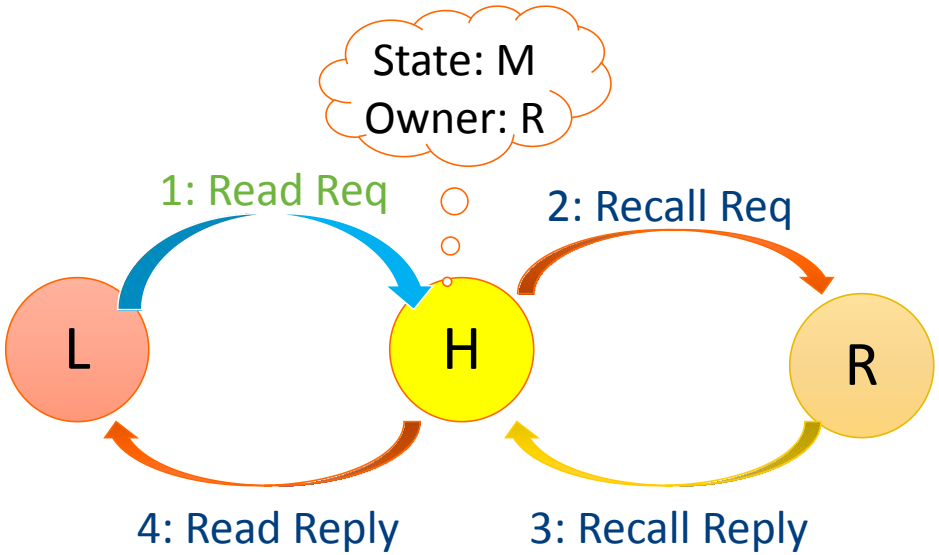
基于Directory的MSI协议 - 示例



这里写入的数据恰好也是10

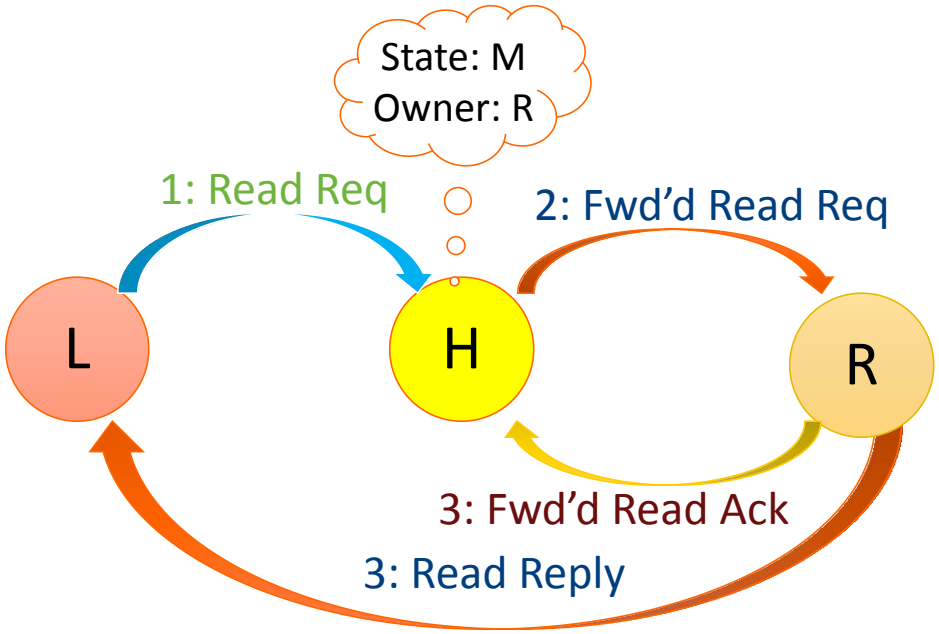
4-hop 实现方式

- L 读数据，发生miss
- H 是home node(记录数据状态)
- R 是owner, 并处于Modified状态



3-hop 实现方式

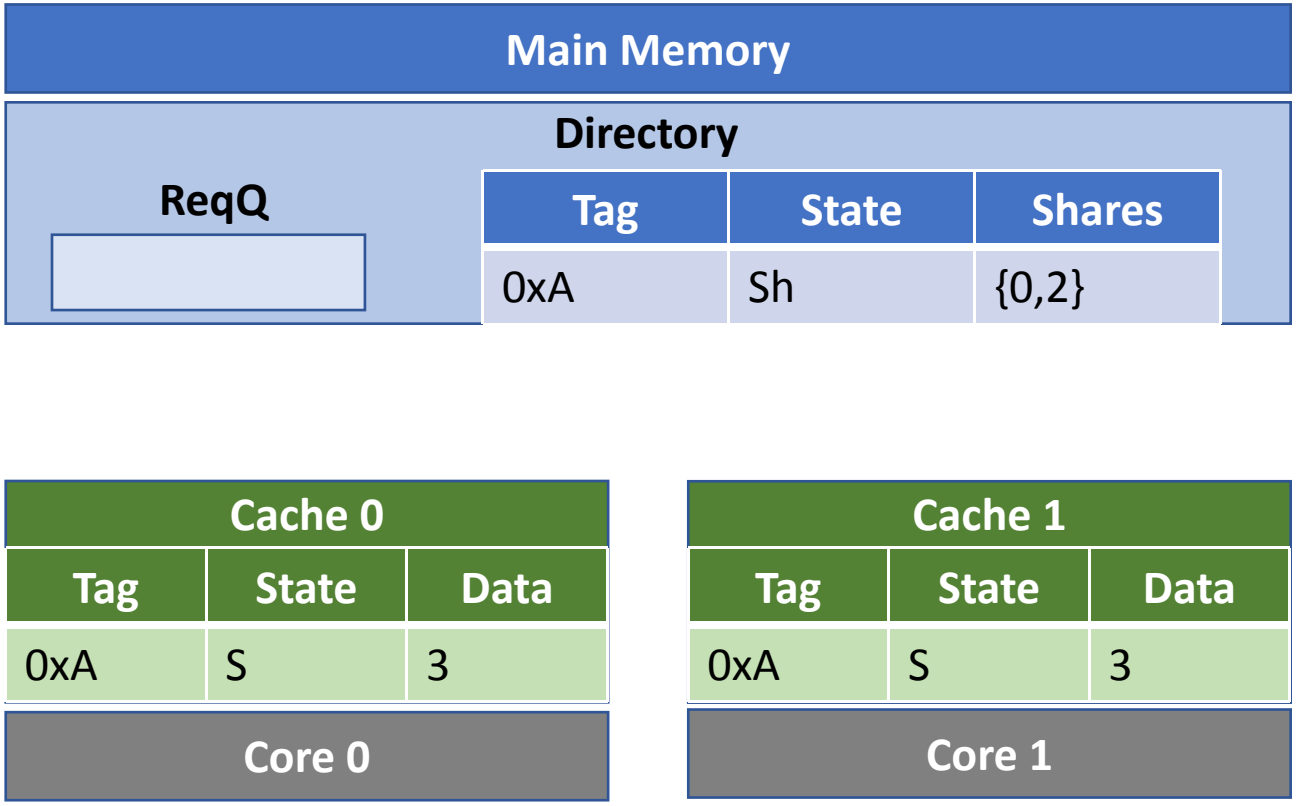
- L 读数据，发生miss
- H 是home node
- R 是owner, 当前处于Modified状态



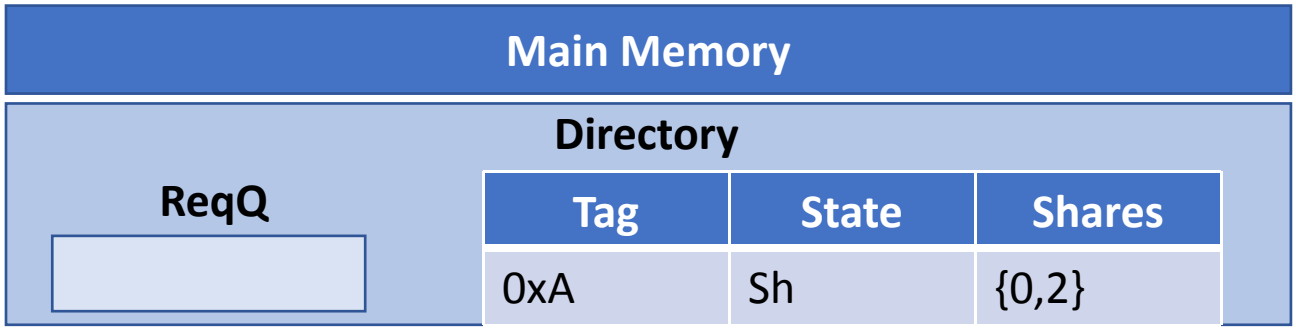
竞争问题

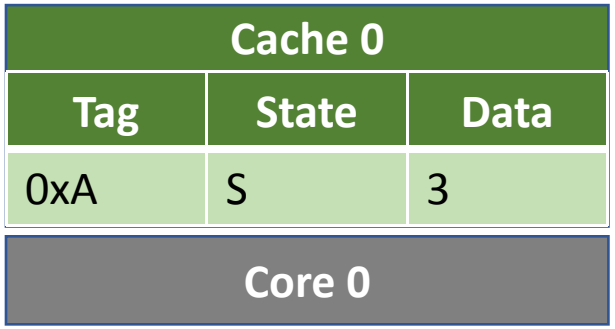
- **可能有并发的请求产生冲突**
  - 可能有多个请求同时访问相同的cache block!
- **需要解决冲突问题:**
  - 对某些请求不回应
    - 请求者再重新发送
  - 对请求进行排队，并按顺序处理
- **公平性**
  - 冲突发生时选优先处理哪个请求?
  - 互联网络的发送顺序、距离等因素都需要考虑

冲突示例

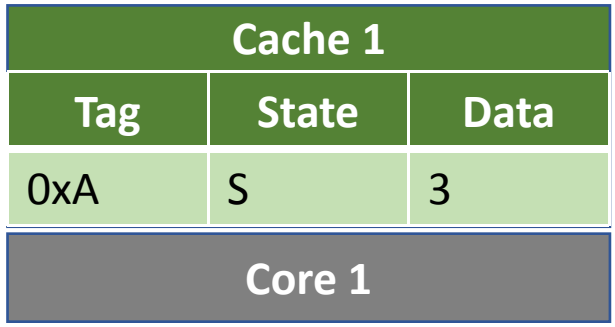

Main Memory
Directory
ReqQ
Tag
State
Shares
0xA
Sh
{0,2}
Cache 0
Tag
State
Data
0xA
S
3
Core 0
Cache 1
Tag
State
Data
0xA
S
3
Core 1

冲突示例



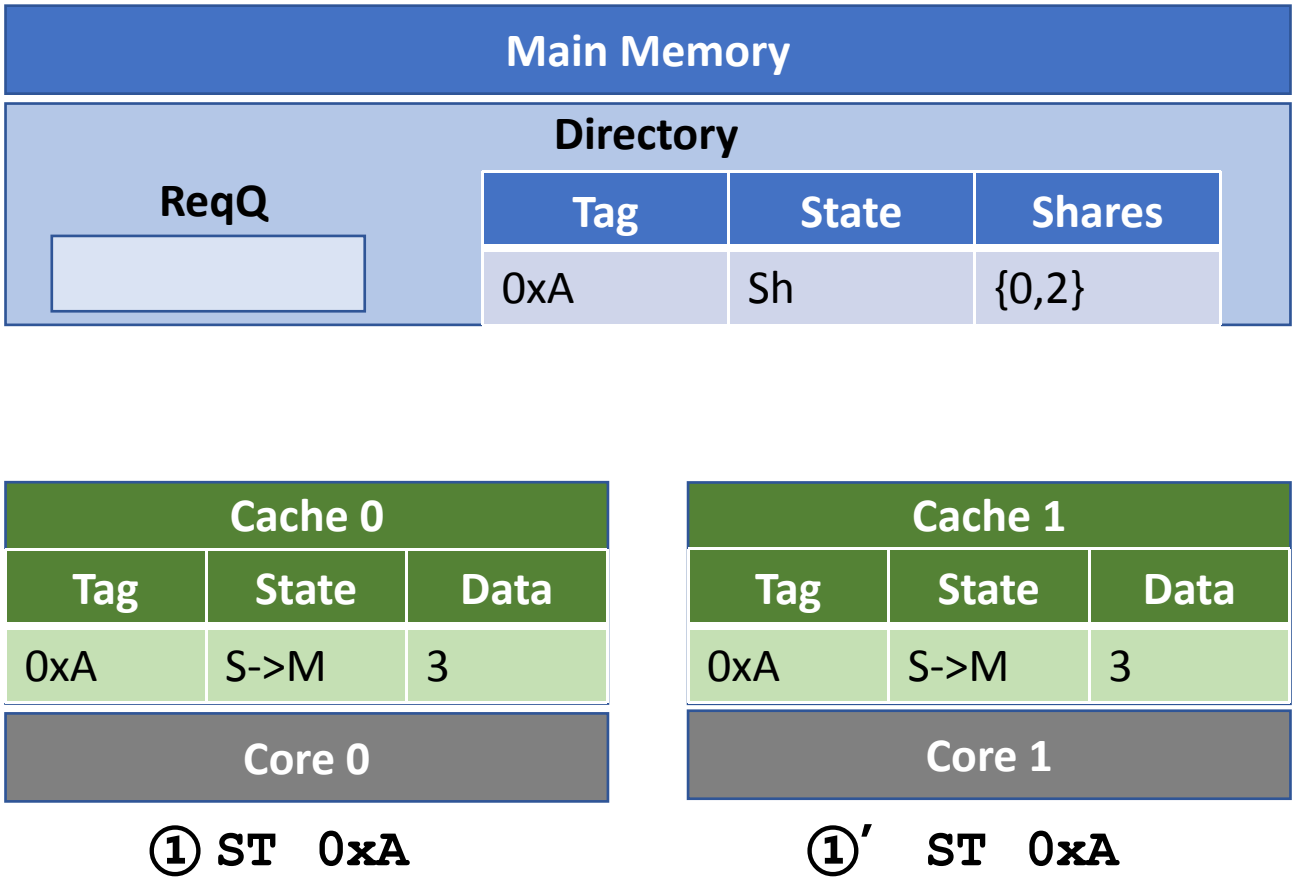


① ST 0xA

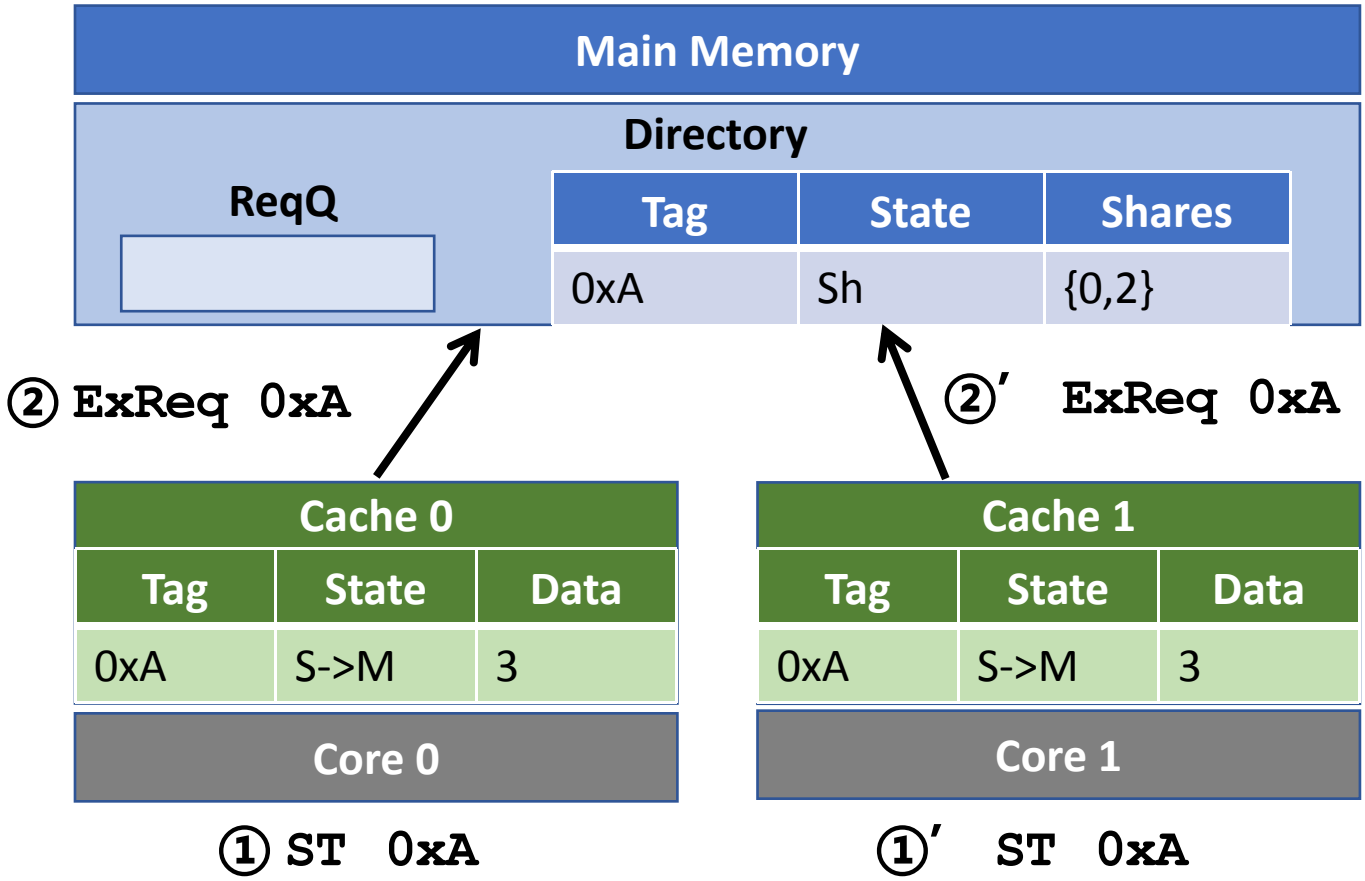


①′ ST 0xA

冲突示例

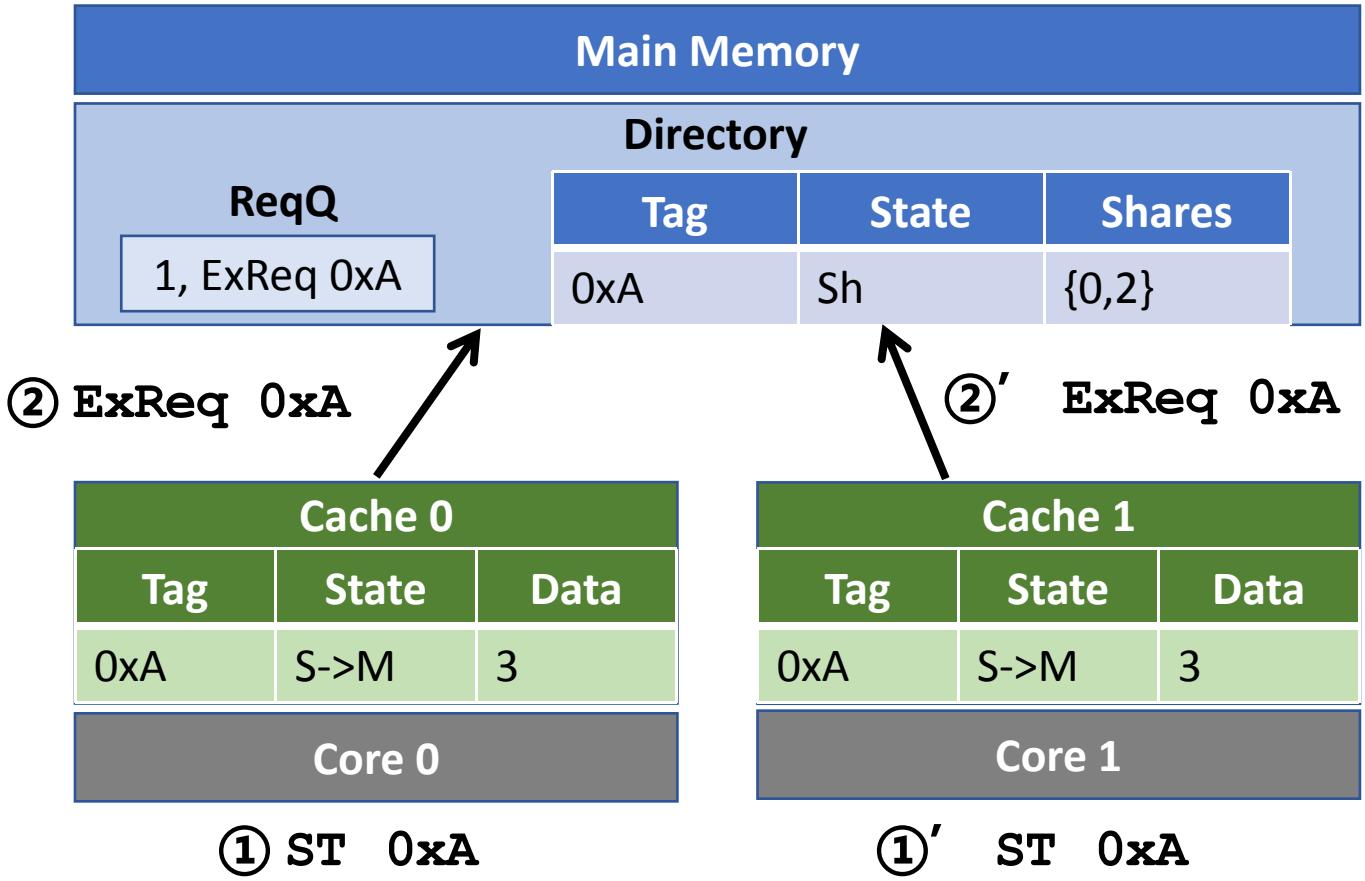

Main Memory
Directory
ReqQ
Tag
State
Shares
0xA
Sh
{0,2}
Cache 0
Tag
State
Data
0xA
S->M
3
Core 0
① ST 0xA
Cache 1
Tag
State
Data
0xA
S->M
3
Core 1
①′ ST 0xA

冲突示例



Caches 0 and 1 同时发送写请求 ExReqs

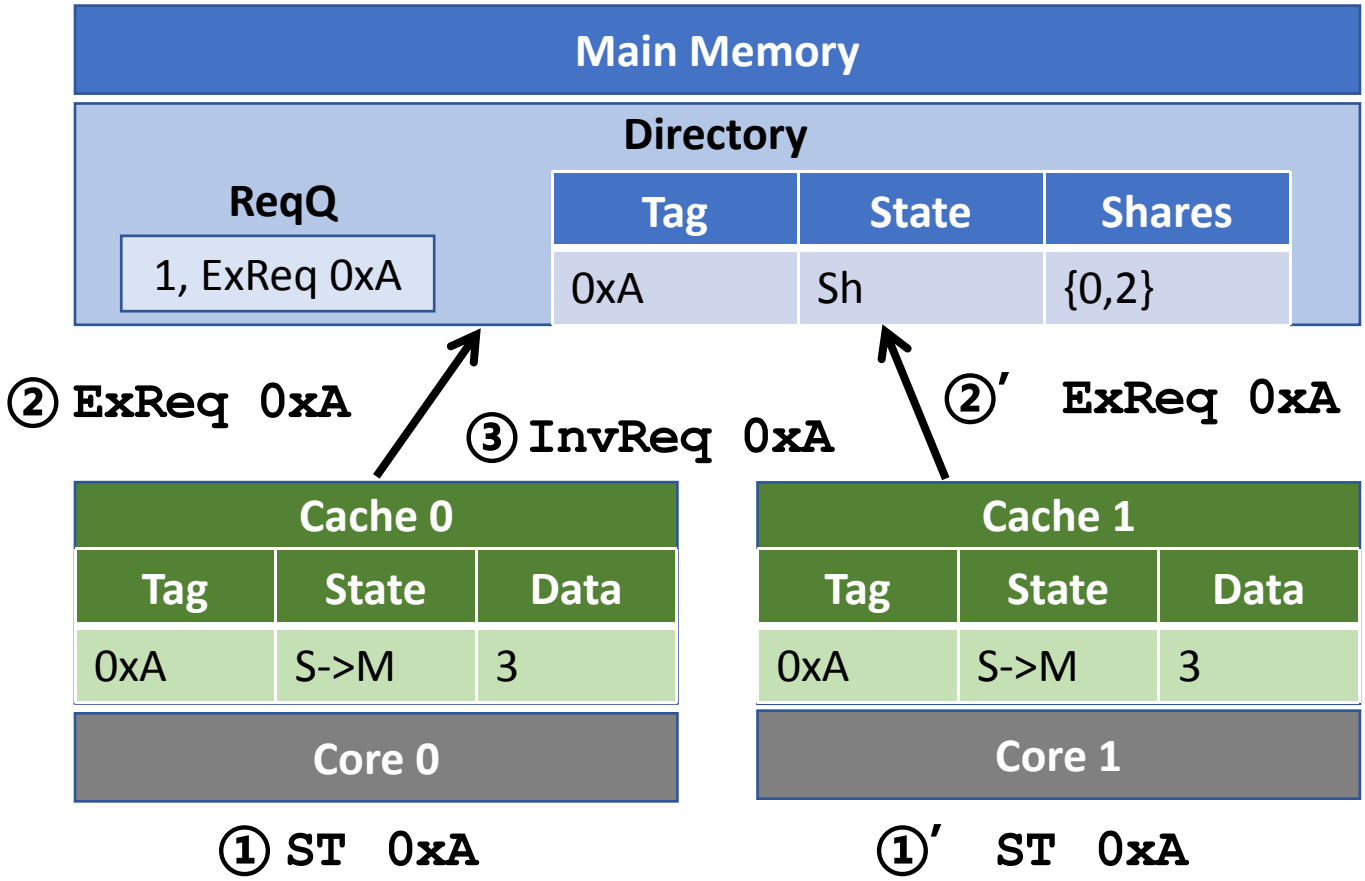
冲突示例



Caches 0 and 1 同时发送写请求 ExReqs

Directory处理cache 0′ s ExReq, 让cache 1排队

冲突示例

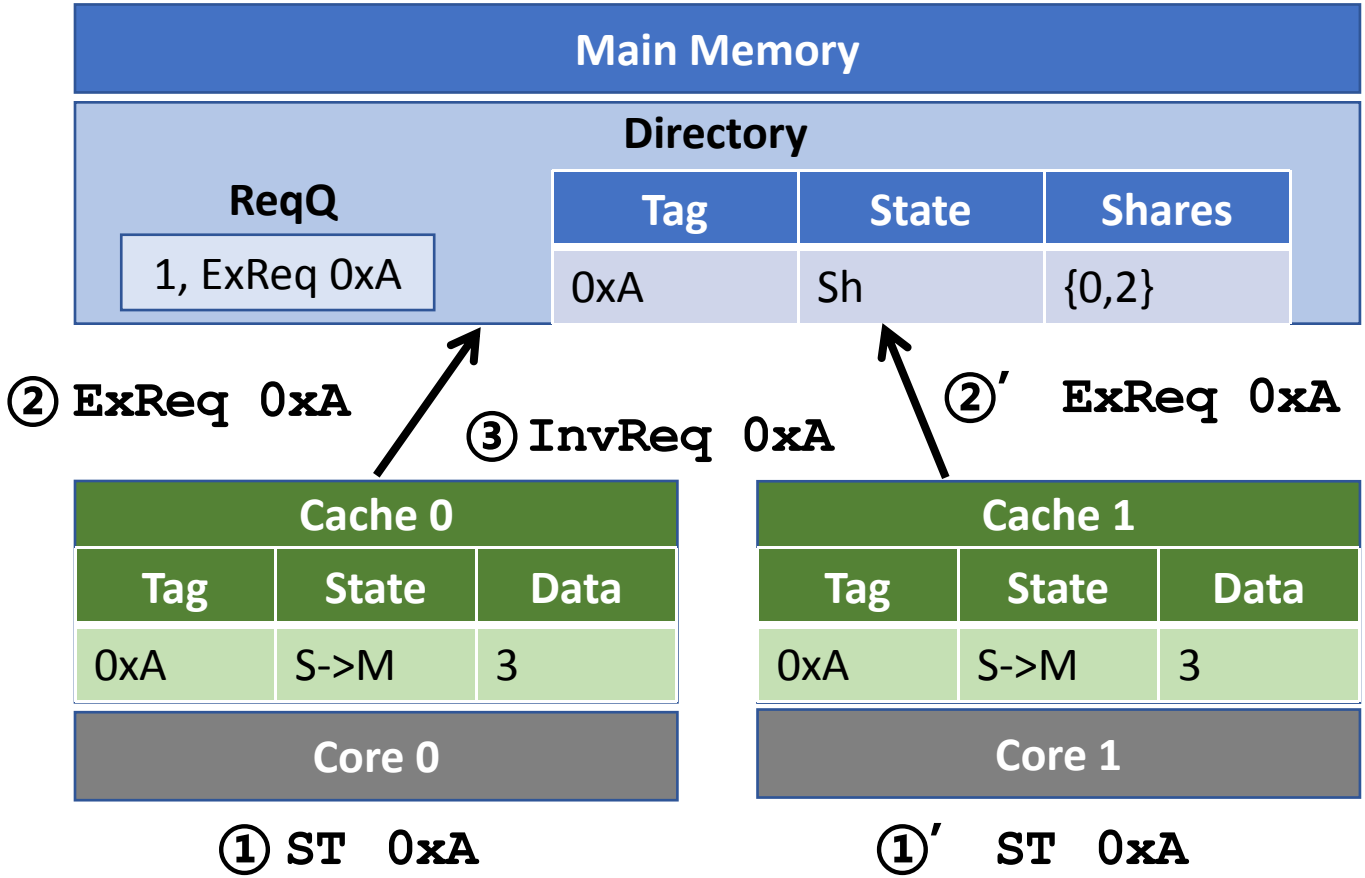


Caches 0 and 1 同时发送写请求ExReqs

Directory处理cache 0′ s ExReq, 让cache 1排队

Cache 1 期望得到ExResp,但收到InvReq!

冲突示例



Caches 0 and 1 同时发送写请求ExReqs

Directory处理cache 0′ s ExReq, 让cache 1排队

Cache 1 期望得到ExResp,但收到InvReq!

Cache 1 应该先回复InvResp,变为I状态；等directory再处理其ExReds时，再变为M状态

Snoopy vs. Directory Coherence

Snoopy

+ 跳数少, 延迟低
+ 全局序列化容易: 总线仲裁
+ 简单: 适用于基于总线的同构多处理器
- 依赖总线广播:
  → 单点处理，可扩展性差

Directory

- 跳数多，延迟高
- 需要额外硬件记录状态
- 协议和冲突处理比较复杂
+ 不需要广播
+ 可扩展性好